

THÈSE

présentée à l'

INSTITUT NATIONAL POLYTECHNIQUE DE LORRAINE

pour obtenir le grade de
DOCTEUR D'ÉTAT ÈS SCIENCES MATHÉMATIQUES

par

Radhia COUSOT

**FONDEMENTS DES MÉTHODES
DE PREUVE D'INVARIANCE ET DE FATALITÉ
DE PROGRAMMES PARALLÈLES**

Thèse soutenue le 15 novembre 1985 devant le jury :

Président : **C. Pair**

Rapporteur

Examineurs : **J.P. Jouannaud**

G. Roucairol

Rapporteur extérieur

M. Sintzoff

Rapporteur extérieur

J.P. Verjus

FONDEMENTS DES METHODES DE PREUVE D'INVARIANCE ET DE FATALITE DE PROGRAMMES PARALLELES

Radhia COUSOT

- 1. INTRODUCTION**
- 2. SEMANTIQUE OPERATIONNELLE**
- 3. PROPRIETES D'INVARIANCE ET DE FATALITE DES
PROGRAMMES**
- 4. PREUVES D'INVARIANCE**
- 5. PREUVES DE FATALITE**
- 6. CONCLUSION**

ANNEXES

- I. NOTATIONS MATHEMATIQUES**
- II. INDEX DES NOTATIONS MATHEMATIQUES**
- III. INDEX DES NOTATIONS INFORMATIQUES**

1. INTRODUCTION

1. INTRODUCTION

1.1 BREF HISTORIQUE DE NOTRE DEMARCHE

1.2 MOTIVATION ESSENTIELLE ET COMPARAISON AVEC D'AUTRES APPROCHES

1.3 RESUME SUCCINT ET PLAN DE LA THESE

1.3.1 NOTATIONS

1.3.2 SEMANTIQUE OPERATIONNELLE

1.3.3 PROPRIETES D'INVARIANCE ET DE FATALITE DES PROGRAMMES

1.3.4 PREUVES D'INVARIANCE

1.3.5 PREUVES DE FATALITE

1.3.6 CONCLUSION ET REFERENCES

1. INTRODUCTION

1.1 BREF HISTORIQUE DE NOTRE DEMARCHE

La motivation de ce travail remonte à nos premières réflexions en 1975 concernant les techniques de mise au point des programmes. Partant du problème de la détermination automatique de jeux d'essai, et après l'avoir généralisé, nous avons étudié les problèmes d'analyse sémantique des programmes (c'est-à-dire la détermination statique (à la compilation) de propriétés dynamiques (à l'exécution) des programmes). Plutôt que de chercher à établir un catalogue de méthodes d'analyse, nous avons préféré étudier comment construire une méthode d'analyse sémantique quelconque à partir d'une méthode de preuve en appliquant des techniques d'approximation (cf. 4.3.2.5). Ceci nous a conduit depuis 1979-80 à nous intéresser presque exclusivement aux méthodes de preuve. Comme nous avions du mal à comprendre très profondément les méthodes existantes, nous avons cherché à les formuler de manière aussi concise et rigoureuse que possible de façon à pouvoir les comparer et les généraliser. Il se pose alors très vite le problème de la justification puis de la construction d'une méthode de preuve à partir d'une sémantique et donc celui du choix de la méthode de définition de la sémantique des programmes. Ceci nous amène à l'étude des sémantiques de programmes qui est le point de départ de cette thèse, nous étudions ensuite les méthodes de preuve de propriétés d'invariance et de fatalité de programmes impératifs, séquentiels, nondéterministes ou parallèles.

1.2 MOTIVATION ESSENTIELLE ET COMPARAISON AVEC D'AUTRES APPROCHES

Il y a de très nombreuses façons d'aborder le problème des preuves de programmes et nous n'avons pas pu, ni voulu explorer toutes les voies de recherche possibles. Il nous semble que tous les problèmes relatifs aux preuves de programmes ne pourront faire de progrès significatifs que si les méthodes de preuve sont mieux comprises qu'elles ne le sont actuellement. Nous avons donc cherché à établir les fondements des méthodes de preuve de programmes ce qui nous a contraint à faire des choix voire des impasses :

- Avant de démontrer une propriété d'un programme il faut spécifier cette propriété. Nous n'étudions pas le problème de la spécification de propriétés de programmes et nous ne considérons que deux classes de propriétés à savoir l'invariance conditionnelle et la fatalité sous invariance.

- Les preuves ne sont pas encore entrées dans la pratique des programmeurs. Elles ne sont généralement appliquées formellement que sur de très petits programmes (quelques lignes) ou informellement sur de petits programmes (quelques pages). Il serait donc intéressant d'essayer de les appliquer sur des programmes moyens ou grands (quelques centaines voire milliers de pages) pour tirer de ces expériences des enseignements pratiques. Nos exemples seront toujours très courts et n'ont évidemment pas cet objectif, ils ne servent qu'à illustrer des notions abstraites. Parmi les difficultés pratiques qui rebutent les programmeurs, il y a le problème de la taille des preuves, le manque de temps, d'expérience et d'entraînement mais également le manque de connaissances (souvent réduites aux méthodes de Floyd et Dijkstra). Notre

contribution dans ce domaine vise plutôt à essayer d'élargir la panoplie des outils disponibles.

- Même pour des programmes simples, les preuves peuvent être difficiles et parfois complexes. Ceci conduit à l'idée que les preuves ne pourront être faites de manière efficace et rigoureuse que si ce sont les ordinateurs qui les font. Cette voie a beaucoup été explorée aux Etats-Unis sans rencontrer les succès espérés, à cause principalement des déficiences des démonstrateurs de théorèmes. Comme la part d'invention nécessaire pour faire des inductions est relativement faible au regard des très nombreuses conditions souvent simples qu'il faut vérifier, l'idée d'outils de preuve semi-automatiques est séduisante : l'ordinateur n'est plus utilisé pour faire mais pour aider à faire la preuve. On pourra penser à un simple aide-mémoire pour guider une preuve faite à la main. De tels systèmes interactifs sont généralement beaucoup plus ambitieux et cherchent à réduire les interventions nécessaires de l'utilisateur. Celui-ci intervient en programmant diverses stratégies possibles qui peuvent être utilisées pour tenter de faire la preuve ou bien de manière conversationnelle pour apporter la part d'invention nécessaire pour faire les inductions. Malheureusement, en cas d'échec de la preuve, les causes de l'échec sont difficilement présentables de manière synthétique à l'utilisateur. De ce fait, l'expérience a montré que les résultats deviennent vite incompréhensibles pour de programmes dépassant quelques lignes. Nous n'avons pas cherché à étudier un quelconque système d'aide à la preuve. Il nous semble que les systèmes existants ont bien souvent le défaut d'être basés sur une seule méthode qui ne marche évidemment pas à coup sûr (de même qu'en mathématiques un raisonnement peut être beaucoup plus simple qu'un autre même s'ils sont équivalents c'est-à-dire

qu'il est possible de passer formellement de l'un à l'autre). Notre apport dans ce domaine est donc essentiellement de présenter de manière uniforme des méthodes apparemment dissemblables. Dans le futur, ceci pourrait encourager à l'emploi d'une combinaison de méthodes plutôt que d'une seule.

- Notre formalisme pour étudier les méthodes de preuve est de nature sémantique plutôt que syntaxique. Il fait appel aux modèles ensemblistes et non aux systèmes logiques formels. Ceci va un peu à l'opposé des nombreux travaux actuels qui cherchent à formaliser les méthodes de preuve à l'aide de logiques de toutes sortes (algorithmiques, dynamiques, modales, temporelles, ...). Ces travaux reposent sur l'idée que l'utilisation de systèmes formels permettra une automatisation plus facile des preuves, (ce qui reste à démontrer). L'utilisation de systèmes formels introduit des problèmes supplémentaires (comme l'incomplétude syntaxique) qu'il nous semble utile d'éliminer dans un premier temps (par exemple pour ne pas cacher, quand il se pose, le problème plus fondamental de l'incomplétude sémantique). De plus ces logiques ne sont pas indépendantes du langage auquel elles s'appliquent et sont donc peut être à la fois beaucoup moins concises, moins abstraites et moins compréhensibles que la formalisation que nous proposons. Enfin, ces logiques d'emploi assez lourd nous semblent mal adaptées pour des preuves informelles qui semblent inévitables en pratique. Il reste que notre travail devrait pouvoir trouver son utilité comme base pour étudier ces logiques (du moins celles ayant trait à l'invariance et la fatalité).

- Comme les programmes sont difficiles à comprendre (et donc à prouver) une fois qu'ils sont terminés, il semble que l'étude de méthodes de construction de programmes (et des preuves concomitantes)

soit plus prometteuse que celle des preuves de correction *a posteriori*. En suivant cette démarche il reste qu'à chaque étape de la construction, il faut démontrer des propriétés relatives à cette étape. Pour ce faire, les méthodes de preuve de propriétés de programmes sont indispensables (et ce d'autant plus que nous proposons une notion abstraite du comportement d'un programme). D'autre part lors du passage d'une étape à la suivante (par exemple par transformation) un certain nombre de propriétés doivent être conservées sans que les preuves soient à refaire. Notre contribution dans ce domaine concerne l'étude de relations entre sémantiques qui conservent les propriétés d'invariance et de fatalité. Il s'agit bien évidemment de quelques résultats techniques et nous n'avons pas l'ambition de proposer une méthodologie de la programmation par transformation de programmes.

- Notre étude adopte un point de vue endogène c'est-à-dire que le comportement du programme est supposé complètement donné sans rien connaître de ce qui lui est extérieur : ce point de vue s'oppose au point de vue exogène où on s'intéresse seulement à une composante du programme (sous-programme, module, processus, etc.) dont le comportement dépend de causes externes et agit sur l'extérieur (dont la connaissance est en général imparfaite). Cette distinction ne nous paraît pas essentielle dans la mesure où l'état du système peut englober l'état du programme et la partie de l'état de l'extérieur ayant un intérêt pour la preuve.

La sémantique du système n'étant pas nécessairement close, on peut étudier des propriétés de systèmes dont l'évolution ne dépend pas uniquement de leur état courant mais dépend de leur histoire (et donc de causes extérieures inconnues et ignorées). Dans le même ordre d'idées, le fait que nous considérons des preuves

relatives à un programme donné, nous conduit à étudier la façon de décomposer cette preuve en fonction de la structure (des états ou des actions) du programme mais nous n'avons pas étudié le problème dual qui consiste à étudier comment la composition d'un programme à partir de parties induit une construction de la preuve du programme par composition des preuves des parties, (exemple de la méthode de Hoare).

Ayant brièvement présenté les voies de recherche qu'il nous aurait été possible d'explorer mais que nous n'avons pas choisies, nous résumons succinctement maintenant le contenu de cette thèse.

1.3 RESUME SUCCINT ET PLAN DE LA THESE

1.3.1 NOTATIONS

Nous utiliserons évidemment des notations mathématiques classiques (concernant la logique, les ensembles, les ordres, les ordinaux, les séquences et les cardinaux) qui sont résumées dans l'annexe I. Il est préférable de commencer par lire cette annexe, mais pour l'éviter nous donnons en annexe II un index des notations mathématiques qui pourra être consulté au fur et à mesure des besoins. Les notations informatiques sont introduites au fil du texte et résumées dans un index donné en annexe III.

1.3.2 SEMANTIQUE OPERATIONNELLE

Le chapitre 2 est consacré à l'étude de la sémantique opérationnelle des programmes.

Toutes les méthodes de preuve de programmes utilisent la notion de "pas de programme". Ceci conduit donc naturellement à formaliser la sémantique d'un programme par un système de transition (cf. 2.2) formé par un ensemble d'états (en général couple état mémoire - état contrôle), un ensemble d'actions, une caractérisation des états initiaux et une relation de transition qui pour toute action "a" caractérise les paires d'états "s" et "s'" telles que par exécution de l'action "a" on peut passer de l'état "s" dans l'état "s'".

La sémantique engendrée par ce système de transition (cf. 2.4) est formée par les ensembles d'états, d'actions et de traces engendrés par

ce système de transition. Une trace est une suite d'états séparés par des actions (conformément à la relation de transition) qui commence par un état initial et est infinie ou bien se termine par un état de blocage (sans successeur possible). Les traces représentent donc un calcul fini et achevé ou bien un calcul infini mais jamais un calcul en cours.

Malheureusement, toutes les sémantiques de programmes ne sont pas directement engendrées par un système de transition. Par exemple, les sémantiques de programmes parallèles équitables ne le sont pas car l'évolution du calcul ne dépend pas uniquement de l'état courant (qui ne contient pas toutes les informations nécessaires pour déterminer l'évolution future des calculs). Autrement dit, les systèmes de transition permettent de rendre compte de l'évolution des programmes quand elle ne dépend que de l'état courant mais pas quand elle dépend de l'histoire du calcul pour arriver dans cet état. Par soucis de généralité, nous sommes donc conduits à définir une sémantique (cf. 2.1) comme un ensemble d'états, un ensemble d'actions et un ensemble de traces quelconques.

Pour formaliser la notion de "pas d'exécution" d'un programme dont la sémantique est arbitraire nous définissons (cf. 2.3) la notion de système de transition engendré par une sémantique. Les transitions sont simplement celles qu'on peut observer le long d'une trace quelconque.

Pour faire des preuves de programmes il est souvent plus pratique de ne pas raisonner sur la sémantique du programme mais plutôt sur une sémantique qui lui est proche. De telles techniques sont souvent utilisées sans justification. Pour démontrer leur validité (aux chapitres 4 pour l'invariance et 5 pour la fatalité) il est nécessaire de les formaliser. Nous le faisons au moyen de relations entre sémantiques qui sont étudiées aux paragraphes 2.5 et 2.6.

Par exemple pour faire une preuve de correction partielle d'un programme parallèle, on peut ignorer l'hypothèse que son exécution est faiblement équitable (tout processus toujours activable est fatalement activé). Nous formaliserons la relation entre la sémantique non équitable et la sémantique équitable de ce programme au moyen d'une fermeture d'une sémantique par réduction aux traces équitables (cf. 2.6.4). Dans une preuve d'invariance, on peut ne pas tenir compte des branches du programme qui sont mortes. Pour le démontrer nous introduisons la notion de fermeture d'une sémantique par réduction aux états accessibles (cf. 2.6.3). Enfin, il est fréquent d'utiliser des variables auxiliaires pour démontrer la correction partielle de programmes parallèles. La relation entre la sémantique du programme et celle du programme transformé (contenant les variables auxiliaires) peut être définie par réduction des actions inobservables (pour éliminer les affectations aux variables auxiliaires) (cf. 2.5.4.2) et par concordance à une relation entre états près (pour éliminer les variables auxiliaires). Lorsque nous définirons une relation entre sémantiques, ceci induit une relation entre systèmes de transition (en considérant la relation entre les sémantiques qu'ils engendrent) dont nous étudions les propriétés.

Nous constatons au paragraphe 2.5 qu'en général, une sémantique et le système de transition qu'elle engendre ne donnent pas les mêmes informations. En effet, une sémantique est en général différente de sa rétraction par transitions c'est-à-dire de la sémantique engendrée par le système de transition qu'elle engendre. Nous disons que la sémantique est close dans le cas contraire. Ceci nous amène (cf. 2.6.8) à la caractérisation des sémantiques closes (à l'aide de divers opérateurs sur les sémantiques définis au paragraphe 2.6). Pour résumer, très intuitivement, une sémantique

est close si elle est fermée par fusion (cf. 2.6.5, le comportement futur de l'exécution dépend seulement de l'état courant qui a été atteint et non de la façon dont il a été atteint), réduite par élimination des traces préfixes stricts (cf. 2.6.6, l'arrêt de l'exécution en un état ne dépend que cet état) et fermée par limites (cf. 2.6.7, les limites des comportements finis sont des comportements infinis acceptables).

Nous passons au paragraphe 2.7 le problème de la spécification de la sémantique d'un programme. Quand la sémantique est close (cf. 2.7.1), on peut la faire aisément à l'aide d'un système de transition (qui peut être lui-même défini par induction sur la syntaxe du programme). Pour une sémantique non close, ce n'est pas possible directement. On peut toujours la faire indirectement en incluant un résumé de l'histoire des calculs dans les états et un contrôleur pour surveiller les transitions (cf. 2.7.2.2) ou bien spécifier la sémantique non close comme un sous-ensemble des préfixes des traces engendrés par un système de transition (cf. 2.7.2.1). Quand la sémantique est non close mais réduite par élimination des traces préfixes stricts, il suffit de considérer un sous-ensemble des traces engendrés par un système de transition (cf. 2.7.3).

Nous terminons ce chapitre par des exemples de définitions de la sémantique d'un langage de programmation (cf. 2.8). Il s'agit d'illustrer ce qui vient d'être dit sur les méthodes de spécification de sémantiques de programmes mais surtout de disposer d'exemples qui nous servent dans la suite pour illustrer la construction systématique de méthodes de preuves. Nous définissons la syntaxe et la sémantique opérationnelle des programmes séquentiels (cf. 2.8.1, qui sont composés d'affectations (ordinaires ou aléatoires), de conditionnelles

et d'itérations), de programmes parallèles asynchrones (cf. 2.8.2, qui sont composés de processus séquentiels partageant des données communes) auxquels nous ajoutons en 2.8.3 la possibilité de communiquer sur rendez-vous par envoi et réception de messages sur des canaux (avec possibilité de sélection entre plusieurs alternatives comme dans CSP ou ADA) et en 2.8.4 l'hypothèse d'exécution faiblement équitale. Finalement, nous ajoutons en 2.8.5 la possibilité de synchroniser les processus au moyen de sémaphores. Pour nous convaincre que la définition des sémaphores que nous avons utilisée est bien celle proposée à l'origine par Dijkstra, nous démontrons certaines propriétés des sémaphores qui ont été énoncées par Hoërmann et sont généralement tenues pour vraies sans justification. Enfin nous donnons une sémantique libérale des sémaphores qui peut être utilisée pour démontrer des propriétés d'invariance.

1.3.3 PROPRIETES D'INVARIANCE ET DE FATALITE DES PROGRAMMES

Au chapitre 3 nous définissons et illustrons très brièvement les propriétés de programmes que nous considérons dans cette thèse à savoir l'invariance (cf. 3.2) et la fatalité (cf. 3.3).

La propriété d'invariance conditionnelle (cf. 3.2.1) est la propriété d'invariance la plus générale que nous considérons. Nous disons que ψ est invariante sous condition ϕ pour une sémantique si pour toute trace de cette sémantique et tout état courant dans cette trace la relation ψ est vraie entre l'état initial et l'état

courant quand la relation ϕ a été vraie entre l'état initial et tous les états précédant l'état courant. Nous parlons d'invariance relationnelle (cf. 3.3.2) quand ϕ est toujours vraie et d'invariance assertionnelle (cf. 3.3.3) quand de plus ψ ne dépend pas de l'état initial.

Ces définitions recourent un grand nombre de propriétés classiques des programmes comme la correction partielle, l'absence d'erreurs à l'exécution, la non-termination, l'exclusion mutuelle, l'absence d'interblocages globaux permanents et des propriétés moins classiques comme les propriétés de précedence du genre l'état courant ne peut pas satisfaire ψ sans qu'un état précédent ait satisfait ϕ .

La propriété de fatalité sous invariance (cf. 3.3.1) est la propriété de fatalité la plus générale que nous considérons. Nous dirons que ψ est fatale sous invariance de ϕ pour une sémantique si pour toute trace de cette sémantique il existe un état (dit but) telle que la relation ψ soit vraie entre l'état initial et le but et la relation ϕ soit vraie entre l'état initial et tous les états qui précèdent le but. A nouveau, nous parlons de fatalité relationnelle quand ϕ est toujours vraie et de fatalité assertionnelle si de plus ψ ne dépend pas de l'état initial.

Ces définitions recourent également un grand nombre de propriétés classiques des programmes comme la terminaison, la correction totale, la garantie d'entrée en section critique ou de réponse à un signal, l'absence de famine d'un processus, etc.

D'autres propriétés des programmes peuvent également se ramener à l'invariance et à la fatalité par exemple en considérant les suffixes de la sémantique du programme ou d'autres relations entre sémantiques comme étudiées au chapitre 2.

1.3.4 PREUVES D'INVARIANCE

Le chapitre 4 est consacré aux fondements des méthodes de preuve de propriétés d'invariance de programmes séquentiels, non-déterministes ou parallèles.

Nous commençons par étudier au paragraphe 4.1 des relations entre sémantiques qui conservent l'invariance avec l'idée que pour démontrer une propriété d'invariance d'un programme relativement à une sémantique nous pouvons essayer de nous ramener à la preuve d'une propriété similaire relativement à une autre sémantique (généralement plus simple).

Par exemple, les propriétés d'invariance sont conservées pour des sémantiques concordantes à des relations ou fonctions entre états et/ou actions p.s. (cf. 4.1.1). Elles sont également conservées après réduction des états non-observables (cf. 4.1.2). La propriété la plus importante est que pour faire une preuve d'invariance pour une sémantique, il est toujours correct de raisonner sur le système de transition qu'elle engendre. Autrement dit les propriétés d'invariance sont conservées par rétraction de la sémantique par transitions (cf. 4.1.3). Cette méthode de preuve est correcte mais elle n'est en général pas sémantiquement complète c'est-à-dire qu'il se peut qu'une propriété

soit invariante pour une sémantique mais pas pour sa rétraction par transitions. Toutefois la méthode est sémantiquement complète quand la sémantique est fermée par fusion (cf. 4.1.3 et 4) et donc en particulier pour les langages considérés au paragraphe 2.8.

Au paragraphe 4.2 nous étudions les principes d'induction qu'on peut utiliser pour démontrer les propriétés d'invariance des programmes. Un principe d'induction décrit l'essence d'une méthode de preuve de manière très concise et abstraite.

Comme nous avons vu dans le paragraphe 4.1, nous rencontrons très fréquemment (mais pas toujours) des sémantiques closes. Nous commençons donc par étudier ce cas particulier (cf. 4.2.1). La méthode la plus connue pour démontrer des propriétés d'invariance de programmes est la méthode de Floyd, Naur et Hoare. Partant d'un exemple, nous inférons le principe d'induction de base pour cette méthode (cf. 4.2.1.1). Essentiellement celui-ci exprime la propriété assez évidente suivante : pour démontrer que la fermeture transitive réflexive t^* d'une relation de transition $\exists a \in A. t_a$ entraîne une relation invariante ψ ($\forall s, s' \in S. t^*(s, s') \Rightarrow \psi(s, s')$) il faut et il suffit qu'il existe un invariant I plus fort que ψ ($\forall s, s' \in S. I(s, s') \Rightarrow \psi(s, s')$) qui soit vrai quand l'état courant est un état initial ($\forall s \in S. I(s, s)$) et reste vrai pour tous les descendants possibles des états initiaux ($\forall s, s', s'' \in S, a \in A. (I(s, s') \wedge t_a(s', s'')) \Rightarrow I(s, s'')$). Dans ce chapitre nous traitons de l'invariance relationnelle en remarquant que la généralisation à l'invariance conditionnelle est triviale (cf. 4.2.1.1-2).

Nous étudions ensuite les variantes possibles de ce principe d'induction de base de façon à en dériver toutes les méthodes existantes plus quelques autres. Pour que l'étude soit systématique nous considérons des transformations de principes d'induction (cf. 4.2.1.2). Pour une propriété de la forme $([E(\underline{s}) \wedge \delta(\underline{s})] \Rightarrow \psi(\underline{s}, \underline{\delta}))$ relative à une relation entre états initiaux et finaux, il est possible de restreindre l'invariant aux états initiaux (cf. 4.2.1.2.1). Une autre transformation également triviale (cf. 4.2.1.2.2) consiste à remarquer que la condition de vérification $(\forall \underline{s}, \underline{s}' \in S, a \in A. [\exists \underline{\delta} \in S. I(\underline{s}, \underline{s}') \wedge t_a(\underline{s}, \underline{s}')] \Rightarrow I(\underline{s}, \underline{s}'))$ est équivalente à $(\forall \underline{s}, \underline{s}' \in S. I(\underline{s}, \underline{s}') \Rightarrow \neg [\exists \underline{\delta} \in S, a \in A. t_a(\underline{s}, \underline{s}') \wedge \neg I(\underline{s}, \underline{s}')])$. Autrement dit, nous pouvons utiliser une plus forte post-condition (comme Floyd pour l'affectation) ou bien une plus faible pré-condition (comme Hoare pour l'affectation). Une autre transformation (cf. 4.2.1.2.3) consiste à remarquer que nous pouvons raisonner sur les relations inverses $(\Gamma^* \Rightarrow \psi$ si et seulement si $(\Gamma^{-1})^* \Rightarrow \psi^{-1})$. Ceci nous permet de formaliser la méthode de Morris-Wegbreit dite "subgoal induction" qui consiste donc à appliquer la méthode de Floyd sur l'inverse du programme. Il est également possible (cf. 4.2.1.2.4) de remplacer l'invariant I par sa négation ce qui conduit à des preuves contrapositives par l'absurde (qui sont ignorées dans la littérature). Enfin (cf. 4.2.1.2.5), quand la propriété à démontrer est une assertion (au lieu d'une relation), nous pouvons utiliser une assertion (comme dans la méthode de Floyd-Naur) au lieu d'une relation invariante (comme dans la méthode de Manna).

Dans le paragraphe 4.2.1.3 nous déterminons tous les principes d'induction que nous pouvons dériver par les transformations ci-dessus, ce qui permet de retrouver toutes les méthodes classiques et de découvrir quelques autres.

Nous montrons ensuite que tous ces principes d'induction sont fortement équivalents (cf. 4.2.1.4) en ce sens que si nous avons

découvert l'invariant I qui convient pour un principe d'induction nous pouvons déterminer l'invariant I' qui convient pour faire la preuve avec tout autre principe d'induction. (Ceci m'empêche d'ailleurs pas que la preuve avec un principe d'induction soit plus facile qu'avec un autre). Ils sont également corrects (si l'invariant I satisfait les conditions de vérification alors φ est invariante) et sémantiquement complets (si φ est invariante, nous pouvons toujours trouver un invariant I satisfaisant les conditions de vérification (mais nous n'avons pas forcément de complétude syntaxique en ce sens que si le langage d'assertions est mal choisi, nous ne pouvons peut-être pas formuler I dans ce langage)).

Les résultats précédents se généralisent aux sémantiques non closes fermées par fusions (cf. 4.2.2) puisque dans ce cas une propriété est invariante pour cette sémantique si et seulement si elle l'est pour la réduction de cette sémantique par transitions.

Ceci n'est évidemment pas vrai pour une sémantique non fermée par fusions (cf. 4.2.3).

Si cette sémantique a été définie comme un sous-ensemble des préfixes des traces engendrés par un système de transition, il est correct mais pas sémantiquement complet d'utiliser les principes d'induction précédents pour ce système de transition. Pour être complets, nous proposons un principe d'induction utilisant des variables auxiliaires (dans la preuve mais pas dans le programme) permettant de cumuler des histoires. Ceci permet de tenir compte dans le principe d'induction proposé du fait que les successeurs possibles d'un état ne dépendent pas uniquement de cet état mais également de la façon dont il a été atteint (ce que nous savons quand nous cumuloons l'histoire).

Si cette sémantique non fermée par fusions a été définie par concordance avec une sémantique close (cf. 4.2.3.2), nous pouvons aisément nous ramener aux principes d'induction qui nous étudient pour les sémantiques closes.

De plus nous montrons que ces nouveaux principes d'induction se ramènent aux précédents quand la sémantique est fermée par fusions.

Enfin nous montrons (cf. 4.2.3.3) que les deux approches (cumul de l'histoire dans des variables auxiliaires ou utilisation d'une sémantique close concordante) sont fortement équivalentes.

Les principes d'induction tels que nous les avons proposés ne sont pas très pratiques à mettre en œuvre directement dans une preuve. Par exemple, dans la condition de vérification $(I(s, s) \wedge t_a(s, s')) \Rightarrow I(s, s')$, t_a serait une énorme formule définissant l'exécution d'un pas quelconque du programme. Il est donc préférable de décomposer cette condition de vérification complexe en une conjonction de conditions de vérification plus simples correspondant par exemple chacune à un pas élémentaire du programme. C'est l'objet du paragraphe 4.3 qui porte sur la construction systématique d'une méthode de preuve d'invariance à partir d'une sémantique opérationnelle et d'un principe d'induction par décomposition de l'invariant global en invariants locaux.

Plutôt que d'imaginer empiriquement une méthode de preuve pour un langage de programmation puis de démontrer sa correction et sa complétude sémantique a posteriori, nous proposons de construire la méthode de preuve de manière systématique. La démarche (cf. 4.3.1) consiste tout d'abord à définir la sémantique opérationnelle

à l'aide d'un système de transition (cf. 4.3.1.1), puis à définir la propriété invariante à démontrer (cf. 4.3.1.2) ce qui permet de choisir le principe d'induction adéquat (cf. 4.3.1.3) parmi ceux précédemment proposés. Ce principe d'induction fait intervenir un invariant global (portant sur les états du programme) alors qu'on préfère généralement des invariants locaux (qui portent par exemple sur les états du programmes correspondant à chaque point du programme, ce qui permet par exemple dans la méthode de Floyd de les associer en commentaire au point du programme auquel ils correspondent). Les invariants locaux étant choisis (cf. 4.3.1.4), il faut définir leur sémantique (cf. 4.3.1.5) c'est-à-dire définir l'invariant global qui correspond à des invariants locaux et inversement. La détermination de la méthode de preuve consiste alors à dériver les conditions de vérification correspondantes. Ceci (cf. 4.3.1.7) en remplaçant le système de transition par sa définition et l'invariant global par les invariants locaux dans le principe d'induction qui a été choisi. Il ne reste ensuite qu'à simplifier pour obtenir les conditions de vérification élémentaires. La méthode obtenue est correcte par construction. Il faut ensuite vérifier qu'elle est sémantiquement complète (cf. 4.3.1.8). Cette vérification peut être inutile ou simplifiée selon la nature de la relation entre invariant global et invariants locaux. C'est pourquoi nous remarquons au paragraphe 4.3.1.6.1 qu'en général, l'ensemble des invariants locaux forme un treillis qui correspond au treillis des invariants globaux (qui sont des sous-ensembles de l'ensemble des paires d'états). Nous étudions ensuite (cf. 4.3.1.6.2) diverses propriétés possibles des correspondances entre invariants locaux et globaux (correspondances monotones, (demi- ou quasi-) correspondances de Galois (injectives, surjectives), isomorphismes complets).

Dans le paragraphe suivant 4.3.2, nous donnons des exemples de construction de méthode de preuve en commençant par la

construction d'une méthode de preuve de non-termination, d'absence d'erreurs à l'exécution et d'invariance globale, par l'absurde pour les programmes séquentiels (cf. 4.3.2.1). Comme ces méthodes ne sont pas classiques, nous les illustrons par des exemples simples.

Ensuite (cf. 4.3.2.2), nous étendons la méthode de Morris-Wegbreit (dite "subgoal induction" pour démontrer la correction partielle de programmes séquentiels) aux programmes parallèles (comme Owicki-Gries ont étendu la méthode de Floyd-Naur-Hoare aux programmes parallèles asynchrones) et nous la généralisons à d'autres propriétés d'invariance. Nous commençons par considérer la correction partielle des programmes séquentiels (cf. 4.3.2.2.1-4) puis l'invariance globale (alors que Morris-Wegbreit croyaient que ce n'était pas possible). Nous abordons ensuite la correction partielle de programmes parallèles asynchrones (cf. 4.3.2.2.2) où nous retrouvons la décomposition de la preuve en une preuve séquentielle par processus et une preuve d'absence d'interférences. Cette méthode étant nouvelle nous des exemples d'application (comme le calcul parallèle asynchrone de $n!$).

Ayant remarqué que dans les méthodes "en avant" (à la Floyd) l'invariant décrit ce qui a été fait (relation entre l'état initial et l'état courant) alors que pour les méthodes "en arrière" (à la Morris-Wegbreit) l'invariant décrit ce qui reste à faire (relation entre l'état courant et l'état final) et que ces deux types d'informations peuvent être très utiles pour aider à la compréhension du programme, nous proposons un principe d'induction (cf. 4.3.2.2.5-3) combinant les avantages de ces deux méthodes (mais où la preuve présente évidemment des redondances).

Nous généralisons ensuite cette méthode aux programmes parallèles synchrones pour la preuve d'absence d'interblocages

globaux permanents (cf. 4.3.2.2.3), la preuve d'exclusion mutuelle (cf. 4.3.2.2.4) et la preuve de non-termination (cf. 4.3.2.2.5).

Nous tentons ensuite (cf. 4.3.2.2.6) d'expliquer pourquoi la méthode de Morris-Wegbreit n'a pas eu le même succès que la méthode de Floyd. La raison principale nous semble bien mise en évidence dans le cas des programmes parallèles où les mêmes invariants peuvent être utilisés, pour les méthodes "à la Floyd", pour démontrer la correction partielle, l'absence d'erreurs à l'exécution, l'absence d'interblocages, l'exclusion mutuelle etc., alors que ce n'est pas le cas pour les méthodes "à la Morris-Wegbreit". Pour remédier à cet inconvénient, nous proposons d'utiliser un principe d'induction combinant l'induction en avant et en arrière.

Nous terminons cette série d'exemples en construisant une méthode de preuve de propriétés d'invariance pour les programmes parallèles communicants (cf. 4.3.2.3). Pour toutes ces méthodes que nous avons construites, nous avons donné une preuve de correction et de complétude sémantique.

Les méthodes de preuve de propriétés d'invariance pour les programmes parallèles connues dans la littérature (Aschcroft, Hoare, Howard, Keller, Lamport, Magunkiewicz, Newton, Owicki-Gries, ...) sont souvent difficiles à comparer à cause des formalismes souvent très différents qui sont utilisés pour les présenter. L'objectif du paragraphe 4.3.2.4 est de montrer qu'elles dérivent toutes du même principe d'induction (qui est à la base de la méthode de Floyd) et ne diffèrent que par la façon de décomposer l'invariant global utilisé dans ce principe d'induction en invariants locaux associés à des points du programme.

Par exemple, la méthode de Ashcroft et de Keller consiste à utiliser un seul invariant global (cf. 4.3.2.4.1). Ce fut la première généralisation de la méthode de Floyd aux programmes parallèles mais elle a l'inconvénient qu'il y a une seule condition de vérification qui n'est pas décomposée en conditions élémentaires.

A l'inverse, la méthode de Ashcroft-Manna consiste à utiliser un invariant local associé à chaque état de contrôle (cf. 4.3.2.4.3). Dans ce cas, la décomposition est par contre trop fine, et par conséquent le nombre de conditions de vérification est trop grand.

Un premier compromis dans la méthode de Owicki-Gries consiste à utiliser un invariant local sur les variables, associé à chaque point (et non plus à chaque état) de contrôle du programme (cf. 4.3.2.4.3). Mais cette méthode est sémantiquement incomplète.

Pour y remédier, nous pouvons suivre Newton et Lamport et utiliser des invariants locaux portant sur l'état de contrôle et les variables, associés à chaque point de contrôle du programme (cf. 4.3.2.4.4).

Nous pouvons également comme l'ont proposé Owicki-Gries utiliser des invariants locaux portant sur les variables du programme et sur des variables auxiliaires, associés à chaque point de contrôle du programme (cf. 4.3.2.4.5). Nous montrons que la méthode est correcte et sémantiquement complète (en définissant la sémantique auxiliaire, qui peut toujours être utilisée pour faire la preuve, à une réduction des états non observables et à une fonction des états p.s.). Dans cette sémantique auxiliaire, les états de contrôle sont simplement simulés par des variables, ce qui montre que les variables auxiliaires ne servent dans la méthode de Owicki-Gries qu'à simuler l'état de contrôle dont les invariants locaux sont indépendants).

Avec ces décompositions, le nombre de conditions de vérification est proportionnel au produit des longueurs des processus du programme parallèle alors qu'en pratique on souhaite que le nombre de conditions de vérification croisse linéairement avec la taille du programme. C'est le cas avec une méthode proposée par Lamport qui consiste à utiliser des invariants locaux portant sur l'état de contrôle et les variables, associés à chaque processus du programme parallèle.

Il est possible enfin, d'utiliser une information redondante (comme dans des méthodes proposées par Hoare, Howard, ...) sous la forme d'un invariant global et d'invariants locaux associés à divers points du programme (cf. 4.3.2.4.7).

Cette profusion de méthodes nous amène à les classer selon le principe d'induction sous-jacent et selon la finesse de la décomposition de l'invariant global en invariants locaux (cf. 4.3.2.4.8).

Pour conclure ce paragraphe, il nous semble que les exemples que nous avons donnés montrent que le choix de la finesse de la décomposition de l'invariant global en invariants locaux ne devrait pas être fixé une fois pour toutes dans une méthode de preuve. Il est bien préférable de choisir cette décomposition en fonction du problème à traiter. La formalisation des méthodes de preuve d'invariance que nous avons proposée permet de le faire sans difficultés.

Nous terminons ce chapitre sur les preuves d'invariance par le paragraphe 4.3.2.5 consacré à l'analyse sémantique de programmes. Nous le faisons parce que ce problème (qui rappelons le, consiste

à déterminer statiquement et automatiquement des invariants pour un programme) a motivé le travail que nous présentons ici. Nous le faisons surtout pour montrer que les résultats obtenus se généralisent sans peine aux programmes parallèles. La présentation est très brève. Nous rappelons simplement comment faire une analyse sémantique "en avant" (cf. 4.3.2.5.1, déterminer un sous-ensemble des descendants des états initiaux), comment faire une analyse sémantique "en arrière" (cf. 4.3.2.5.2, déterminer un sous-ensemble des ascendants des états finaux) et comment faire une analyse combinée "avant-arrière" (cf. 4.3.2.5.3, déterminer un sous-ensemble des états qui sont à la fois descendants des états initiaux et ascendants des états finaux). Comme le formalisme utilisé est très général et qu'il englobe les programmes parallèles nous nous contentons de donner quelques exemples pour montrer l'application des méthodes d'analyse sémantique à ce type de programmes.

1.3.5 PREUVES DE FATALITE

Le chapitre 5 est consacré à l'étude des méthodes de preuve de propriétés de fatalité des programmes séquentiels et parallèles.

Un certain nombre de résultats obtenus au chapitre précédent (comme les transformations de principes d'induction, la décomposition de l'invariant global en invariant locaux, ...) s'appliquent également ici (avec les légères adaptations qui pourraient être nécessaires). Pour éviter les répétitions, nous ne reprenons pas ces mêmes idées dans ce chapitre même si elles s'appliquent.

Ce chapitre comprend deux paragraphes importants, le paragraphe 5.2 consacré aux principes d'induction "à la Floyd" et le paragraphe 5.3 consacré aux principes d'induction "à la Burstall". En fait nous aurions pu rédiger différemment en présentant 5.2 en quelques phrases comme un cas particulier de 5.3. Nous avons choisi d'aller du particulier (5.2) au général (5.3) de façon à refléter l'évolution historique mais surtout pour graduer les difficultés. Pour éviter les redites nous présentons en 5.2 un certain nombre de résultats (comme les principes d'induction pour les sémantiques mon closes, ...) qui ne seront pas repris en 5.3 car leur généralisation ne nous a pas semblé présenter des difficultés une fois que l'idée a été donnée en 5.2.

Nous commençons l'étude des méthodes de preuve de fatalité par celle des relations entre sémantiques qui conservent la fatalité (cf. 5.1). Malheureusement, les résultats positifs sont beaucoup moins nombreux que pour l'invariance. Sans chercher l'exhaustivité, nous montrons (cf. 5.1.1) que les propriétés de fatalité sont conservées par inclusion de sémantiques (mais dans un sens seulement car par exemple il n'est pas toujours possible de démontrer une propriété de fatalité d'un programme parallèle synchrone en raisonnant sur la sémantique libérale des sémaphores) et que (cf. 5.1.2) les propriétés de fatalité sont conservées pour des sémantiques concordantes à des relations entre états et actions pris (dans les deux sens, sous certaines conditions).

Dans le paragraphe 5.2, nous étudions les preuves de fatalité par des principes d'induction généralisant la méthode de Floyd.

Nous commençons par rappeler en 5.2.1 la méthode de Floyd (dite des assertions invariants et de l'ordre bien fondé) pour démontrer la correction totale des programmes séquentiels.

Ceci nous permet d'en extraire le principe d'induction de base pour démontrer les propriétés de fatalité des sémantiques cluses (cf. 5.2.2).

Nous étudions ensuite une série de principes d'induction équivalents au principe d'induction de base (cf. 5.2.3) qui reflètent quelques unes des variantes possibles de la méthode de Floyd. Par exemple, il n'est pas nécessaire d'associer une fonction de terminaison à tous les points de contrôle du programme mais seulement aux points de coupure des boucles. L'utilisation de bons-ordres n'est pas obligatoire puisque les relations bien-fondées suffisent et sont quelquefois plus commodes. La fonction de terminaison peut être remplacée par une variable auxiliaire (dans la preuve mais qui n'apparaît pas nécessairement dans le programme) qui décroît strictement à chaque pas. Cette variable auxiliaire peut toujours être choisie comme un ordinal, etc.

Nous abordons ensuite le problème de la correction et de la complétude sémantique des principes d'induction à la Floyd qui précèdent (cf. 5.2.4).

Nous remarquons en 5.2.5 que si la propriété fatale est une relation entre les états initiaux et finaux il faut en général, que la fonction de terminaison porte sur l'état courant mais également

sur l'état initial (ce que beaucoup d'ouvrages introductifs ignorent. Ils présentent donc une méthode sémantiquement incomplète).

Il est également intéressant de caractériser les relations bien-fondées (ou de manière équivalente les ordinaux) qui sont nécessaires pour faire des preuves de fatalité basées sur les principes d'induction généralisant la méthode de Floyd (cf. 5.2.6). Pour ce faire, nous disons que le non-déterminisme d'une sémantique est m -borné si le cardinal de l'ensemble des successeurs d'un état quelconque pour la relation de transition qu'elle engendre est strictement inférieur à m . En particulier le non-déterminisme est fini (Dijkstra dit "borné") si tout état a un nombre fini de successeurs possibles. Nous montrons que pour une sémantique close dont le non-déterminisme est m -borné, il est toujours possible de faire des preuves de fatalité avec des relations bien-fondées dont l'ordre est inférieur à m^+ (où $m^+ = \omega$ si $m < \omega$, $m^+ = m$ quand m est un cardinal régulier sinon m^+ qui est le plus petit cardinal strictement supérieur à m). Cette limite est stricte quand m est régulier. Comme cas particulier, nous obtenons que la méthode de Knuth-Luckham-Suzuki (qui consiste à utiliser un compteur strictement incrémenté à chaque tour de boucle et dont la valeur est bornée) n'est pas sémantiquement complète quand le non-déterminisme n'est pas fini (et ne peut donc pas être généralisé au cas des programmes parallèles équitables).

Le paragraphe 5.2.7 est consacré à la décomposition des conditions de vérification. Le cas général ayant été étudié en 4.3, nous nous contentons d'illustrer quelques décompositions (de façon à montrer comment les méthodes de Lamport et Owicki-Grues initialement conçues pour les preuves de correction partielle peuvent être étendues aux preuves de correction totale).

N'ayant abordé jusqu'ici que le cas des sémantiques closes, le cas des preuves de fatalité pour les programmes parallèles équitables étaient exclus. c'est pourquoi nous étudions au paragraphe 5.2.8 les principes d'induction "à la Floyd" pour démontrer les propriétés de fatalité de sémantiques non closes.

Comme pour l'invariance, nous pouvons définir la sémantique non close par concordance avec une sémantique close à une fonction de états pris (cf. 5.2.8.1). Dans ces conditions, n'importe lequel des principes d'induction introduits pour les sémantiques closes est utilisable. Ceci revient, par exemple pour un programme parallèle équitable, à raisonner sur un programme transformé qui incorpore un contrôleur d'exécution assurant l'équité.

Une autre approche (cf. 5.2.8.2) peut être utilisée quand nous spécifions la sémantique non close par un sous-ensemble des préfixes des traces engendrés par un système de transition. Elle consiste à cumuler l'histoire des calculs dans une variable auxiliaire. Ceci permet, quand la sémantique n'est pas fermée par limites de ne pas imposer que la fonction de terminaison décroisse à chaque pas mais seulement aux points de coupure qui ne sont pas déterminés statiquement comme dans la méthode de Floyd mais dynamiquement c'est-à-dire en fonction de l'histoire des calculs. Comme cas particulier nous retrouvons la méthode de Pnueli-Lehmann-Stavi pour démontrer la correction totale de programmes parallèles faiblement équitables et qui consiste essentiellement à appliquer la méthode de Floyd mais avec la possibilité que la fonction de terminaison ne décroisse pas tant qu'un processus est activable sans être activé. De plus, quand la sémantique n'est pas réduite par élimination des traces préfixes stricts, l'histoire est utilisée pour s'assurer que la

but est atteint pour les traces finies avant la fin de la trace (qui peut ne pas être un état sans successeur). Enfin, quand la sémantique n'est pas fermée par fusion, l'invariant doit être vrai pour tous les états qui peuvent être atteints en suivant le préfixe d'une trace où le but n'est jamais atteint mais pas forcément pour tous les préfixes obtenus par transitions successives. Là encore, le cumul de l'histoire dans une variable auxiliaire est utile.

Enfin, nous montrons au paragraphe 5.3.8.3 que les deux approches (utilisation d'une sémantique auxiliaire incluant un contrôleur d'exécution ou bien utilisation de variables auxiliaires pour cumuler l'histoire) sont équivalentes.

Au paragraphe 5.3.1, nous présentons la méthode des annotations intermittentes de Burstall à l'aide d'exemples puis nous en déduisons le principe d'induction de base formalisant de manière très concise cette méthode. Nous démontrons que ce principe de preuve est correct. La question de la complétude sémantique est plus complexe. En utilisant l'induction transfinie (plutôt que finie puisque le principe d'induction généralise la méthode de Burstall au monde terminisme infini) nous montrons que ce principe d'induction est sémantiquement complet sous une condition suffisante (mais pas nécessaire) sur la sémantique et la propriété de fatalité. Cette condition exprime qu'un état ne peut pas être un but sur une trace et appartenir à un préfixe d'une autre trace le long duquel le but n'a pas été atteint. Cette condition est évidemment vérifiée dans le cas de Burstall qui considère des programmes de terministes mais également pour des généralisations aux programmes monde terministes (Prucci, Apt-Delparte, ...) où sont considérées des propriétés de fatalité unaires ne dépendant pas des états initiaux.

Lorsqu'on considère des propriétés de fatalité unaires, les relations entre les valeurs initiales et finales des variables des programmes ne peuvent être exprimées qu'en considérant un programme transformé dans lequel les valeurs initiales sont affectées à des variables auxiliaires. Outre la transformation du programme sous raison fondamentale, l'utilisation de variables auxiliaires est en un sens trop souple parce que nous pouvons relier des états intermédiaires quelconques lors d'un calcul et même mémoriser toute l'histoire du calcul. Une telle liberté d'utilisation de variables auxiliaires n'est pas dans l'esprit de la méthode proposée par Burstall et des exemples donnés par Manna-Waldinger où les lemmes démontrés par induction sur les données sont toujours de la forme :

"if sometime $\phi(x_1, \dots, x_m) \wedge x_1 = a_1 \wedge \dots \wedge x_m = a_m$ at l then
 sometime $\psi(x_1, \dots, x_m, x_1, \dots, x_m)$ at l' "

(où $x_1, \dots, x_m$ sont les variables du programme et $a_1, \dots, a_m$ leurs valeurs symboliques respectives au point l du programme). Ceci s'exprime dans notre principe d'induction de base par l'utilisation de propriétés de fatalité binaires (mieux qu'en imposant des restrictions adéquates sur l'utilisation des variables auxiliaires qui dépendraient de la syntaxe des programmes). Cependant, nous faisons la conjecture que même pour les programmes déterministes, il existe des propriétés de fatalité pour lesquelles l'utilisation d'assertions binaires n'est pas sémantiquement complète.

Cette conjecture nous conduit en 5.3.2 à généraliser la méthode des assertions intermittentes de Burstall d'une part en utilisant l'induction transfinie (pour traiter le monde déterminisme non borné) et des assertions intermittentes ternaires (permettant d'exprimer des lemmes d'une forme plus générale "if sometime $\phi(x_1, \dots, x_m, z_1, \dots, z_n) \wedge x_1 = a_1 \wedge \dots \wedge x_m = a_m$ at l then sometime $\psi(x_1, \dots, x_m, z_1, \dots, z_n, a_1, \dots, a_m, x_1, \dots, x_m)$ at l' " où $z_1, \dots, z_n$ (respectivement $a_1, \dots, a_m$) désignent les valeurs des variables au point d'entrée (respectivement au point l)

du programme). Nous démontrons que ce principe d'induction généralisé est correct et sémantiquement complet.

De ce principe, nous dérivons au paragraphe 5.3.3 toute une série de principes d'induction de plus en plus abstraits et concis. Par exemple, il est intéressant de considérer un nombre transfini et non plus fini d'assertions intermittentes (que nous pouvons représenter de manière finie au moyen de variables auxiliaires). Ceci étend la méthode de Burstall de façon à incorporer la méthode de Floyd et permet d'utiliser des assertions intermittentes binaires et non plus ternaires (en prenant formellement un lemme différent pour chaque état initial). Parmi ces principes d'induction il en est un qui formalise l'idée de Schwarz que la méthode de Burstall consiste à démontrer des théorèmes par induction mathématique à partir d'axiomes spécifiant l'effet des commandes élémentaires du programme. Les principes d'induction les plus abstraits permettent une meilleure compréhension de la méthode de Burstall (par exemple nous montrons que "l'évaluation symbolique" et l'"induction sur les données" peuvent être comprises de manière unifiée et réduite à une induction sur les calculs). Ces généralisations successives introduisent plus de souplesse dans l'écriture des preuves mais pas de puissance supplémentaire puisque nous démontrons que tous les principes de preuve considérés sont corrects et sémantiquement complets donc équivalents. Comme les principes d'induction les plus abstraits peuvent paraître très éloignés de la méthode de Burstall, nous donnons quelques exemples pour montrer le contraire.

Le principe d'induction "à la Floyd" comporte une induction le long des traces d'exécution tandis que le principe d'induction "à la Burstall" comporte la combinaison d'une induction le long de parties de traces d'exécution (en relation avec "l'évaluation symbolique" de Burstall)

et d'une récursivité (en relation avec "l'induction sur les données" de Burstall). Le principe d'induction "à la Floyd" correspond au cas particulier du principe d'induction "à la Burstall" où la récursivité n'est pas utilisée. Comme conséquence immédiate, le principe d'induction "à la Burstall" est sémantiquement complet puisque nous avons démontré précédemment que celui "à la Floyd" l'est. Cette remarque explique également pourquoi la méthode de Burstall est mieux adaptée que la méthode de Floyd pour démontrer la correction totale de programmes itératifs obtenus par élimination de la récursivité (la raison étant qu'il est possible de conserver la récursivité dans la preuve). Beaucoup plus important est le fait que le principe d'induction "à la Burstall" offre des possibilités de décomposer une preuve d'un théorème de fatalité en preuves indépendantes de lemmes plus simples, qui n'existent pas avec le principe d'induction "à la Floyd" qui nécessite une preuve globale (cf. 5.3.5).

L'argument de complétude sémantique pour le principe d'induction "à la Burstall" n'est pas pleinement satisfaisant parce que le style des preuves permises est fixé. Les utilisateurs de la méthode de Burstall ont besoin d'un résultat de complétude plus fort puisqu'ils aimeraient savoir si les lemmes qu'ils ont l'intention d'utiliser dans leurs preuves peuvent toujours être choisis librement. Une réponse affirmative est donnée au paragraphe 5.3.4 (avec la condition nécessaire et suffisante que chaque lemme doit concerner une propriété qui est fatale pour le programme mais aussi relativement aux autres lemmes qui sont utilisés dans sa preuve).

Une certaine polémique a entouré la comparaison des méthodes de Floyd et Burstall (Manna-Waldinger, Gies, ...), les uns affirmant qu'il y a des programmes qui ont une preuve "naturelle" par la méthode

de Burstall et pas avec la méthode de Floyd, les autres travaillant suffisamment la preuve avec la méthode de Floyd jusqu'à donner une impression de simplicité. (La plupart des exemples fournis (comme la version itérative de la fonction d'Ackermann) étaient obtenus par élimination de la récursivité et nous en donnons un (cf. 5.4-1) qui est simple, semble convaincant et n'est pas de cette nature). Comme contribution à ce débat nous démontrons au paragraphe 5.4 que toute preuve obtenue par une méthode peut se réécrire systématiquement en une preuve par l'autre méthode. La preuve est assez technique et longue mais intuitivement la transformation entre les deux preuves est très similaire dans un sens à l'élimination de la récursivité dans les programmes et dans l'autre sens à la présentation récursive de programmes itératifs, (nous ne prétendons donc pas que ces transformations préservent le "matériel" des preuves).

Ayant montré que la méthode de Floyd est en ces particularités de la méthode de Burstall (après les généralisations adéquates que nous avons faites), il reste néanmoins que les présentations classiques des preuves par ces deux méthodes à l'aide d'assertions invariantes d'une part et d'assertions intermittentes d'autre part sont suffisamment dissemblables pour qu'il soit difficile d'uniformiser ces deux méthodes. C'est pourquoi nous introduisons, au paragraphe 5.5, la notion de chaîne de preuve.

L'idée de présenter graphiquement les preuves de programmes par des diagrammes acycliques fut introduite par Lamport et développée ultérieurement par Owicki-Lamport et Manna-Pnueli. Cependant, ces méthodes n'étaient pas sémantiquement complètes à cause d'un certain

nombre de instructions, principalement l'impossibilité de faire des inductions infinies. Notre formalisation est plus générale du fait qu'elle consiste à introduire des chartes de preuve qui sont bien structurées, peuvent éventuellement présenter des cycles et peuvent être utilisés récursivement pour les preuves par induction sur les données. Après avoir défini la notion de charte de preuve (cf. 5.5.1) nous démontrons la correction et la complétude sémantique de la méthode en montrant qu'elle correspond à l'utilisation d'un principe d'induction "à la Bantall" (cf. 5.5.2). Nous donnons ensuite quelques exemples de présentation de preuves par chartes (cf. 5.5.3), pour montrer que les preuves "à la Floyd" peuvent se présenter très naturellement par des chartes et également pour montrer que les chartes de preuve sont très utiles pour démontrer des propriétés de fatalité de programmes parallèles asynchrones. Enfin, les idées développées au paragraphe 5.2 concernant les preuves de fatalité pour les sémantiques non closes, s'appliquent directement. Nous le montrons simplement par des exemples en étendant les preuves de fatalité par chartes de preuves au cas des programmes parallèles faiblement équitables puis à celui des programmes parallèles synchrones.

1.3.6 CONCLUSION ET REFERENCES

Le chapitre 6 est une brève conclusion. Les références sont données à la fin de chaque chapitre.

2. SEMANTIQUE OPERATIONNELLE

2. SEMANTIQUE OPERATIONNELLE

2.1 DEFINITION DE LA SEMANTIQUE PAR UN ENSEMBLE DE TRACES COMPLETES

2.1.1 TRACES

2.1.2 SEMANTIQUE

2.2 DEFINITION DES SYSTEMES DE TRANSITION

2.3 SYSTEME DE TRANSITION ENGENDRE PAR UNE SEMANTIQUE

2.4 SEMANTIQUE ENGENDREE PAR UN SYSTEME DE TRANSITION

2.5 RELATIONS ENTRE SEMANTIQUES ET ENTRE SYSTEMES DE TRANSITION

2.5.1 INCLUSION DE SEMANTIQUES ET DE SYSTEMES DE TRANSITION

2.5.2 EQUIVALENCE DE SYSTEMES DE TRANSITION

2.5.3 CONCORDANCE ENTRE SEMANTIQUES ET ENTRE SYSTEMES DE TRANSITION A DES RELATIONS ENTRE ETATS ET/OU ACTIONS PRES

2.5.3.1 Concordance à une fonction des états près

- 2.5.3.2 Concordance à l'annulation des états près
- 2.5.3.3 Concordance à l'annulation des actions près

2.5.4 REDUCTION DE SEMANTIQUES

- 2.5.4.1 Réduction des états inobservables
- 2.5.4.2 Réduction des actions inobservables

2.6 FERMETURES DE SEMANTIQUES

- 2.6.1 FERMETURE D'UNE SEMANTIQUE PAR PREFIXES
- 2.6.2 FERMETURE D'UNE SEMANTIQUE OU D'UN SYSTEME DE TRANSITION PAR SUFFIXES
- 2.6.3 FERMETURE D'UNE SEMANTIQUE OU D'UN SYSTEME DE TRANSITION PAR REDUCTION AUX ETATS ET/OU ACTIONS ACCESSIBLES
- 2.6.4 FERMETURE D'UNE SEMANTIQUE PAR REDUCTION AUX TRACES EQUITABLES
- 2.6.5 FERMETURE D'UNE SEMANTIQUE PAR FUSIONS
- 2.6.6 REDUCTION D'UNE SEMANTIQUE PAR ELIMINATION DES TRACES PREFIXES STRICTS
- 2.6.7 FERMETURE D'UNE SEMANTIQUE PAR LIMITES
- 2.6.8 RETRACTION D'UNE SEMANTIQUE PAR TRANSITIONS, SEMANTIQUE CLOSE

2.7 SPECIFICATION D'UNE SEMANTIQUE A L'AIDE D'UN SYSTEME DE TRANSITION

- 2.7.1 SPECIFICATION D'UNE SEMANTIQUE CLOSE A L'AIDE DU SYSTEME DE TRANSITION QUI L'ENGENDRE

2.7.2 SPECIFICATION D'UNE SEMANTIQUE NON CLOSE

2.7.2.1 Spécification par un système de transition et une condition sur les préfixes des traces qu'il engendre

2.7.2.2 Spécification par concordance avec une sémantique close

2.7.3 SPECIFICATION D'UNE SEMANTIQUE NON CLOSE REDUITE PAR ELIMINATION DES TRACES PREFIXES STRICTS ET FERMEE PAR FUSIONS

2.7.3.1 Spécification par un système de transition et une condition sur les traces qu'il engendre

2.7.3.2 Spécification par concordance avec une sémantique close

2.8 EXEMPLE DE DEFINITION DE LA SEMANTIQUE D'UN LANGAGE DE PROGRAMMATION

2.8.1 PROGRAMMES SEQUENTIELS

2.8.1.1 Syntaxe

2.8.1.2 Sémantique

2.8.1.2.1 Etats

2.8.1.2.2 Actions

2.8.1.2.3 Etats initiaux

2.8.1.2.4 Relation de transition

2.8.1.2.5 Traces

2.8.1.3 Exemple

2.8.2 PROGRAMMES PARALLELES ASYNCHRONES

2.8.2.1 Syntaxe

2.8.2.2 Sémantique

2.8.2.2.1 Etats

2.8.2.2.2 Actions

2.8.2.2.3 Etats initiaux

2.8.2.2.4 Relation de transition

2.8.2.2.5 Traces

2.8.2.3 Exemples

2.8.3 PROGRAMMES PARALLELES COMMUNICANTS

2.8.3.1 Syntaxe

2.8.3.2 Sémantique

2.8.3.2.1 Etats

2.8.3.2.2 Actions

2.8.3.2.3 Etats initiaux

2.8.3.2.4 Relation de transition

2.8.3.2.5 Traces

2.8.3.3 Exemple

2.8.4 PROGRAMMES PARALLELES FAIBLEMENT EQUITABLES

2.8.4.1 Syntaxe

2.8.4.2 Sémantique

2.8.4.3 Exemple

2.8.5 PROGRAMMES PARALLELES SYNCHRONES

2.8.5.1 Syntaxe

2.8.5.2 Sémantique

2.8.5.2.1 Etats

2.8.5.2.2 Actions

2.8.5.2.3 Etats initiaux

2.8.5.2.4 Relation de transition

2.8.5.2.5 Traces

2.8.5.2.6 Propriétés de la sémantique des sémaphores

2.8.5.3 Sémantique libérale

2.8.5.4 Exemple

2.9 REFERENCES

2. SEMANTIQUE OPERATIONNELLE

Une preuve de correction d'un programme consiste à démontrer une relation entre une sémantique et une spécification de ce programme. Pour faire l'étude des méthodes de preuve de programmes, il est donc nécessaire de disposer d'une méthode de définition de la sémantique des programmes. Dans ce chapitre nous expliquerons la méthode que nous avons choisie pour définir la sémantique des programmes.

Pour définir la sémantique d'un langage de programmation il faut simplement définir la sémantique de tous les programmes de ce langage. Pour ce faire on procède généralement par induction sur la syntaxe abstraite des programmes.

Pour définir la sémantique d'un programme, il faut définir un modèle de l'ensemble des exécutions possibles de ce programme sur ordinateur. Il s'agit d'un modèle parce qu'une simplification de la réalité est nécessaire pour éviter d'avoir à tenir compte d'une multitude de détails, en général liés à une implémentation (nombre et capacité des mémoires, conventions de codage des informations, temps de calcul des opérations, ...). Il se pose alors le problème de savoir à quel niveau d'abstraction doit être définie la sémantique. Par exemple de manière classique la sémantique dénotationnelle (Milne-Strachey [76]) a pour but d'associer à un texte de programme syntaxiquement correct une fonction mathématique qui, à une donnée, associe le résultat du programme pour cette donnée. Ce type de définition rend difficilement compte de certaines notions liées à l'exécution (par exemple que deux processus d'un programme parallèle sont en exclusion mutuelle en cours d'exécution).

Plutôt que de retenir les états d'entrée et de sortie, nous considérons un modèle opérationnel dont le niveau d'abstraction est celui qui permet d'observer la suite d'états intermédiaires par lesquels passe l'ordinateur pendant l'exécution d'un programme (ce qui permet par exemple d'étudier l'ensemble des valeurs prises par une variable au cours du calcul, de s'assurer qu'un processus d'un programme parallèle ne meurt pas de faim ou d'étudier des propriétés temps-réel en incluant une mesure de distance entre deux états successifs).

Un premier modèle de la sémantique opérationnelle des programmes consiste à utiliser les systèmes de transition (Keller [79]). Un système de transition définit l'ensemble des états du programme et une relation de transition entre un état et ses successeurs possibles. Les traces (ou exécutions) du programme sont alors les séquences d'états qui commencent par un état initial et telles que deux états successifs sont liés par la relation de transition. C'est le modèle défini dans Cousot-P [79,81] que nous avons utilisé dans nos premiers travaux. Cette définition de la sémantique des programmes est bien adaptée à l'étude des méthodes de preuve car la relation de transition formalise bien la notion de pas du programme et les preuves procèdent généralement par induction sur le nombre de pas d'exécution du programme. Malheureusement, cette approche ne permet pas de définir de relation entre plus de deux états successifs du programme. Ceci est gênant par exemple pour définir la sémantique de programmes parallèles équitables dont l'exécution est contrôlée par un agent extérieur dont l'état ne fait pas partie de celui du programme.

Un modèle plus général est celui des traces d'exécution (Pratt [79], Lamport [80]). Avec ce modèle la sémantique d'un programme est définie comme un ensemble de traces, chaque

trace étant la séquence des états intermédiaires observés au cours d'un calcul terminé ou infini. Contrairement à ce qui se fait généralement dans la littérature (cf. par exemple Pratt [79]), nous ne considérons pas de traces incomplètes qui correspondraient à un calcul en cours. Ceci nous permet de n'avoir à faire aucune hypothèse sur l'ensemble des traces que nous considérons (alors que Pratt [79] doit supposer que tout préfixe d'une trace en cours est une trace en cours, etc.). De plus, il n'y a aucune perte de généralité en omettant les traces inachevées. Ce modèle est plus général que les systèmes de transition car il permet de définir des relations entre un nombre quelconque d'états successifs du programme. Cependant il est mal adapté pour rendre compte des méthodes de preuve de programmes qui reprennent toutes l'idée due à Floyd [67]-Naur [66] qu'il est plus simple de raisonner sur des ensembles d'états par induction sur le nombre de pas du programme que sur l'ensemble des traces d'exécution.

Dans la suite nous associerons donc à un programme une sémantique (définie comme un ensemble de traces, chaque trace étant une suite finie ou infinie d'états) et un système de transition (défini comme une relation de transition sur un ensemble d'états). Dans ce chapitre nous nous posons la question suivante : étant donnée une propriété P relative à une sémantique S caractériser les propriétés P' et les sémantiques S' en fonction de P et S telles que P est vrai de S si, seulement si ou si et seulement si P' est vrai de S' . Ceci nous amènera en particulier à la question suivante : étant donnée une sémantique est-elle (ou jusqu'à quel point est-elle) engendrée par un système de transition ? Les résultats obtenus nous permettront plus tard de répondre à la question plus générale : étant donnée une preuve d'une propriété relative à une sémantique est-il possible (ou jusqu'à quel point est-il possible) de la remplacer par une preuve d'une autre propriété relative à une autre sémantique ou à un système de transition.

2.1 DEFINITION DE LA SEMANTIQUE PAR UN ENSEMBLE DE TRACES COMPLETES

Dans ce paragraphe nous définissons formellement la notion de trace. Une trace représente la séquence finie (ou infinie) des états intermédiaires au cours d'une exécution achevée (respectivement, ayant bouclé) du programme. Chaque état se décompose généralement en un état contrôle (spécifiant le ou les points du programme où se trouve l'exécution) et un état mémoire (indiquant la valeur actuelle des identificateurs). Deux états successifs dans une trace sont séparés par une action qui est généralement le nom de l'opération atomique qui a permis de passer d'un état à son successeur.

Nous définissons ensuite une sémantique comme étant un ensemble de traces d'exécution.

2.1.1 TRACES

Une trace d'exécution p est une séquence non vide finie ou infinie d'états séparés par des actions.

$$p = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \dots s_i \xrightarrow{a_i} s_{i+1} \dots$$

Intuitivement, cette trace est un modèle d'une exécution du programme qui commence dans l'état s_0 , exécute une action nommée a_0 pour atteindre l'état s_1 , puis l'action a_1 pour atteindre l'état s_2 , etc. Si l'exécution se termine alors la trace est finie sinon elle est infinie. Les traces sont complètes en ce sens qu'elles représentent un calcul achevé ou qui ne se termine pas (et jamais un calcul "en cours"). Nous n'utiliserons pas la notion de trace vide qui ne correspond à aucune réalité physique (puisque une exécution doit toujours commencer par un état initial).

Exemple

L'exécution d'un programme qui exécute deux fois l'action a puis l'action b et se termine, peut se décrire par la trace finie :

$$0 \xrightarrow{a} 0 \xrightarrow{a} 0 \xrightarrow{b} 1$$

L'exécution d'un programme qui boucle en exécutant l'action a sans arrêt peut se décrire par la trace infinie :

$$0 \xrightarrow{a} 0 \xrightarrow{a} 0 \dots 0 \xrightarrow{a} 0 \dots$$

□

Une trace p sur un ensemble S d'états et un ensemble A d'actions est un triplet $\langle m, s, a \rangle$. La longueur m de la trace est un entier non nul ($m = \{0, \dots, m-1\} \in (\omega \vee 0)$) si la trace est finie. C'est $\omega = \{0, 1, \dots\}$ si la trace est infinie. Par conséquent $m \in (\omega+1 \vee 0)$. $s \in (m \rightarrow S)$ est la séquence des états (le i ème état $s(i)$ en comptant à partir de zéro étant noté s_i).

La séquence des actions est $a \in ((m-1) \rightarrow A)$, (la i ème action $a(i)$ en comptant à partir de zéro étant notée a_i).

Définition 2.1.1.1

L'ensemble des traces sur un ensemble S d'états et A d'actions est défini par :

$$\Sigma^m \langle S, A \rangle = \{ \langle m, s, a \rangle : s \in (m \rightarrow S) \wedge a \in ((m-1) \rightarrow A) \}$$

(traces de longueur $m \in (\omega+1 \vee 0)$)

$$\Sigma^{<l} \langle S, A \rangle = \bigcup_{m \in (\omega \vee 0)} \Sigma^m \langle S, A \rangle$$

(traces de longueur strictement inférieure à $l \in (\omega+2)$)

$$\Sigma^{\leq l} \langle S, A \rangle = \Sigma^{<l} \langle S, A \rangle \cup \Sigma^l \langle S, A \rangle$$

(traces de longueur inférieure ou égale à $l \in (\omega+1)$)

Nous appellerons $\Sigma^{\omega} \langle S, A \rangle$ l'ensemble des traces finies, $\Sigma^{\omega} \langle S, A \rangle$ l'ensemble des traces infinies et $\Sigma^{\leq \omega} \langle S, A \rangle$ l'ensemble des traces sur S et A . L'ensemble des traces étant d'usage fréquent, nous poserons $\Sigma \langle S, A \rangle = \Sigma^{\leq \omega} \langle S, A \rangle$ et nous omettons le couple $\langle S, A \rangle$ si nous pouvons le déterminer sans ambiguïté d'après le contexte.

La longueur d'une trace $p = \langle n, A, a \rangle$ comptée en nombre d'états est $|p| = n$. La longueur de p comptée en nombre d'actions est $|p|_A = n-1$. Le $i^{\text{ème}}$ état de p (compté à partir de zéro) est $p_i = s_i$ pour $i \in |p|$. La $i^{\text{ème}}$ action de p (compté à partir de zéro) est $p_i = a_i$ pour $i \in |p|_A$.

Le préfixe $p^{<m}$, $m \in (\omega+2) \cup 0$ (respectivement $p^{\leq m}$, $m \in (\omega+1)$) d'une trace $p = \langle n, A, a \rangle$ est p si $m \geq n$ (respectivement $m+1 \geq n$) sinon $\langle m, s^{<m}, a^{<m-1} \rangle$ (respectivement $\langle m+1, s^{\leq m}, a^{\leq m} \rangle$), où $f^{<m}$ (respectivement $f^{\leq m}$) est le préfixe (non vide) $\langle f_0, \dots, f_{m-1} \rangle$ (respectivement $\langle f_0, \dots, f_m \rangle$) d'une séquence $\langle f_0, \dots, f_{m-1}, f_m, \dots \rangle$.

Le suffixe $p^{>m}$, $m+1 \leq n$ (respectivement $p^{\geq m}$, $m \leq n$) d'une trace $p = \langle n, A, a \rangle$ est $\langle n-(m+1), s^{>m}, a^{>m} \rangle$ (respectivement $\langle n-m, s^{\geq m}, a^{\geq m} \rangle$) où $f^{>m}$ (respectivement $f^{\geq m}$) est le suffixe (non vide) $\langle f_{m+1}, \dots \rangle$ (respectivement $\langle f_m, \dots \rangle$) d'une séquence $\langle f_0, \dots, f_m, f_{m+1}, \dots \rangle$.

La tranche $p^{<m, m' \rangle}$ (respectivement $p^{<m, m' \rangle}$, $p^{<m, m' \rangle}$, $p^{<m, m' \rangle}$) est $(p^{<m' \rangle})^{>m}$ (respectivement $(p^{<m' \rangle})^{\geq m}$, $(p^{<m' \rangle})^{\geq m}$, $(p^{<m' \rangle})^{>m}$).

La concaténation de $p \in \Sigma \langle S, A \rangle$ et $q \in \Sigma \langle S, A \rangle$ est $p \wedge q = p$ si $|p| = \omega$ sinon $|p| < \omega$ et si $p|_A = q_0$ alors $p \wedge q = r$ tel que $r|_A = p$ et $r^>|p|_A = q$ sinon $p|_A \neq q_0$ et $p \wedge q$ est indéfinie.

La concaténation de $p \in \Sigma \langle S, A \rangle$ et $q \in \Sigma \langle S, A \rangle$ via une action $a \in A$ est $p \xrightarrow{a} q = p$ si $|p| = \omega$ sinon $p \xrightarrow{a} q = r \in \Sigma \langle S, A \rangle$ telle que $r|_A = p$, $r^>|p|_A = q$ et $r_{|p|_A} = a$.

Par abus de notation, nous notons s la trace de longueur 1 constituée du seul état s , c'est-à-dire $\langle 1, \{ \langle 0, s \rangle \}, 0 \rangle$. De ce fait une trace p finie peut se noter $p_0 \xrightarrow{f_0} p_1 \dots p_{|p|-1} \xrightarrow{f_{|p|-1}} p_{|p|}$ et une trace infinie par $p_0 \xrightarrow{f_0} p_1 \dots p_i \xrightarrow{f_i} p_{i+1} \dots$ et donc par abus de notation, nous écrivons $\langle p_i \xrightarrow{f_i} p_{i+1} : i \in |p| \rangle$ (par analogie avec l'écriture $\langle f_i : i \in I \rangle$ de la fonction F telle que $\forall i \in I. F(i) = f_i$).

Exemple

$$p = p_0 \xrightarrow{f_0} p_1 \xrightarrow{f_1} p_2$$

$$q = q_0 \xrightarrow{g_0} q_1 \xrightarrow{g_1} q_2 \xrightarrow{g_2} q_3 \xrightarrow{g_3} q_4 \dots q_i \xrightarrow{g_i} q_{i+1} \dots$$

$$|p| = 3, |p| = 2, |q| = |q| = \omega$$

$$p^{<0} \text{ est indéfini, } p^{<1} = p_0, p^{<2} = p_0 \xrightarrow{f_0} p_1, p^{<m} = p \text{ pour } m \geq 3$$

$$q^{<0} \text{ est indéfini, } q^{<1} = q_0, q^{<m} = q_0 \xrightarrow{g_0} q_1 \dots \xrightarrow{g_{m-2}} q_{m-1} \text{ pour } m \geq 2, q^{<\omega} = q$$

$$p^{\geq 0} = p, p^{\geq 1} = p_1 \xrightarrow{f_1} p_2, p^{\geq 2} = p_2, p^{\geq m} \text{ est indéfini pour } m \geq 3$$

$$q^{\geq m} = q_m \xrightarrow{g_m} q_{m+1} \dots q_i \xrightarrow{g_i} q_{i+1} \dots \text{ pour } m \geq 0, q^{\geq \omega} \text{ est indéfini}$$

$$q^{\langle 2, 4 \rangle} = q_2 \xrightarrow{g_2} q_3$$

$$q \xrightarrow{a} p = q$$

$$p \xrightarrow{a} q = p_0 \xrightarrow{f_0} p_1 \xrightarrow{f_1} p_2 \xrightarrow{a} q_0 \xrightarrow{g_0} q_1 \dots q_i \xrightarrow{g_i} q_{i+1} \dots$$

□

2.1.2 SEMANTIQUE

La sémantique d'un langage associe à tout programme un ensemble de traces car l'exécution d'un programme (par exemple parallèle) est en général non déterministe. Un ensemble de traces peut également être nécessaire pour décrire les exécutions d'un programme déterministe si par exemple les états initiaux correspondant à des données différentes sont différents.

La sémantique d'un langage $\mathcal{LP}$ associe à tout programme P_r de $\mathcal{LP}$ un ensemble non vide $S[P_r]$ d'états, un ensemble non vide $A[P_r]$ d'actions et un ensemble $\Sigma[P_r] \subseteq \Sigma \langle S[P_r], A[P_r] \rangle$ de traces.

Exemple 2.1.2-1

La sémantique d'un programme qui exécute de zéro à deux fois l'action a puis l'action b peut se décrire par la sémantique $\langle S, A, \Sigma \rangle$ où $S = \{0, 1\}$, $A = \{a, b\}$ et $\Sigma = \{0 \xrightarrow{b} 1, 0 \xrightarrow{a} 0 \xrightarrow{b} 1, 0 \xrightarrow{a} 0 \xrightarrow{a} 0 \xrightarrow{b} 1\}$.

□

Exemple 2.1.2-2

La sémantique d'un programme qui exécute un nombre quelconque mais fini de fois l'action a puis l'action b peut se décrire par la sémantique $\langle S, A, \Sigma \rangle$ où $S = \{0, 1\}$, $A = \{a, b\}$ et

$$\Sigma = \left\{ \begin{array}{l} 0 \xrightarrow{b} 1 \\ 0 \xrightarrow{a} 0 \xrightarrow{b} 1 \\ \dots \\ 0 \xrightarrow{a} 0 \xrightarrow{a} 0 \dots 0 \xrightarrow{a} 0 \xrightarrow{b} 1 \end{array} \right\}$$

□

Exemple 2.1.2-3

La sémantique d'un programme qui exécute un nombre fini de fois l'action a puis l'action b ou exécute un nombre infini de fois l'action a peut se décrire par la sémantique $\langle S, A, \Sigma \rangle$ où $S = \{0, 1\}$, $A = \{a, b\}$ et

$$\Sigma = \left\{ \begin{array}{l} \dots \\ 0 \xrightarrow{a} 0 \dots 0 \xrightarrow{a} 0 \xrightarrow{b} 1 \\ \dots \\ 0 \xrightarrow{a} 0 \dots 0 \xrightarrow{a} 0 \dots \end{array} \right\}$$

□

Définition 2.1.2:1

Sémantiques

L'ensemble des sémantiques sur des ensembles $\mathcal{S}$ (d'états) et $\mathcal{A}$ (d'actions) est $\text{Sem} \langle \mathcal{S}, \mathcal{A} \rangle = \{ \langle S, A, \Sigma \rangle : S \in \mathcal{S} \wedge A \in \mathcal{A} \wedge \Sigma \in \Sigma \langle S, A \rangle \}$

2.2 DEFINITION DES SYSTEMES DE TRANSITION

Etant donnés des ensembles S d'états et A d'actions, un système de transition formalise la notion d'états initiaux d'un programme (comme un sous-ensemble de S caractérisé par une fonction ε à valeurs de vérité ($\#$ vrai, $\#$ faux)) et de pas du programme correspondant à l'exécution d'une action $a \in A$ (comme une relation de transition t_a entre un état et ses successeurs possibles).

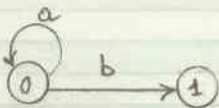
Définition 2.2:1

Systèmes de transition

L'ensemble des systèmes de transition sur des ensembles $\mathcal{S}$ (d'états) et $\mathcal{A}$ (d'actions) est $\text{Trans}(\mathcal{S}, \mathcal{A}) = \{ \langle S, A, t, \varepsilon \rangle : S \subseteq \mathcal{S} \wedge A \subseteq \mathcal{A} \wedge t \in (A \rightarrow (S \times S \rightarrow \{\#, \#\})) \wedge \varepsilon \in (S \rightarrow \{\#, \#\}) \}$.

Exemple 2.2-1

Un programme qui exécute l'action b ou bien exécute un nombre fini de fois l'action a puis l'action b ou bien un nombre infini de fois l'action a , peut-être informellement représenté par l'automate fini suivant:



que nous formalisons par le système de transition $\langle S, A, t, \varepsilon \rangle$ où $S = \{0, 1\}$, $A = \{a, b\}$, $t_a(s, s') = [s = s' = 0]$, $t_b(s, s') = [s = 0 \wedge s' = 1]$ et $\varepsilon(s) = [s = 0]$.

□

2.3 SYSTEME DE TRANSITION ENGENDRE PAR UNE SEMANTIQUE

Pour formaliser les notions d'"état initial" et de "pas de calcul" pour une sémantique $\langle S, A, \Sigma \rangle$, nous définissons le système de transition $\langle S, A, t_{\langle S, A, \Sigma \rangle}, e_{\langle S, A, \Sigma \rangle} \rangle$ qu'elle engendre. (Pour alléger les notations, nous laisserons implicites certains paramètres quand ils peuvent être aisément déterminés d'après le contexte. Par exemple nous écrivons $\langle S, A, t, e \rangle$ au lieu de $\langle S, A, t_{\langle S, A, \Sigma \rangle}, e_{\langle S, A, \Sigma \rangle} \rangle$).

Définition 2.3.1 Système de transition engendré par une sémantique

Les états initiaux engendrés par une sémantique $\langle S, A, \Sigma \rangle$ sont caractérisés par

$$e \in (S \rightarrow \{\text{tt}, \text{ff}\})$$

$$e(s) = [\exists p \in \Sigma. p_0 = s]$$

La relation de transition engendrée par une sémantique $\langle S, A, \Sigma \rangle$ est définie par :

$$t \in (A \rightarrow (S \times S \rightarrow \{\text{tt}, \text{ff}\}))$$

$$t_a(s, s') = [\exists p \in \Sigma, i \in |p|. (p_i = s \wedge p_i = a \wedge p_{i+1} = s')]$$

Le système de transition engendré par une sémantique $\langle S, A, \Sigma \rangle$ est $\langle S, A, t_{\langle S, A, \Sigma \rangle}, e_{\langle S, A, \Sigma \rangle} \rangle$ où $t_{\langle S, A, \Sigma \rangle}$ et $e_{\langle S, A, \Sigma \rangle}$ sont respectivement la relation de transition et les états initiaux engendrés par $\langle S, A, \Sigma \rangle$.

Exemple

Les sémantiques données en exemples 2.1.2-1, 2.1.2-2 et 2.1.2-3 engendrent exactement le même système de transition 2.2-1.

□

2.4 SEMANTIQUE ENGENDREE PAR UN SYSTEME DE TRANSITION

La description des comportements possibles d'un programme par un système de transition est souvent plus simple à utiliser voire à comprendre que la description par un ensemble de traces. Pour répondre à la question de savoir dans quelles conditions la donnée d'une sémantique $\langle S, A, \Sigma \rangle$ d'un programme est équivalente à la donnée du système de transition $\langle S, A, t, \varepsilon \rangle$ engendré par cette sémantique, nous définirons l'ensemble des traces complètes engendrées par un système de transition quelconque.

Définition 2.4:1

La sémantique engendrée par un système de transition $\langle S, A, t, \varepsilon \rangle$ est $\langle S, A, \Sigma \langle S, A, t, \varepsilon \rangle \rangle$ avec :

- $\Sigma^n \langle S, A, t, \varepsilon \rangle = \{ p \in \Sigma^n \langle S, A \rangle : \varepsilon(p_0) \wedge \forall i \in [n]. t_{\#i}(p_i, p_{i+1}) \wedge \forall a \in A, a \in S. \neg t_a(p_{n-1}, a) \}$
(traces complètes finies de longueur $n \in (\omega \cup 0)$)
- $\Sigma^\omega \langle S, A, t, \varepsilon \rangle = \{ p \in \Sigma^\omega \langle S, A \rangle : \varepsilon(p_0) \wedge \forall i \in \omega. t_{\#i}(p_i, p_{i+1}) \}$
(traces infinies)
- $\Sigma \langle S, A, t, \varepsilon \rangle = \bigcup_{n \in (\omega+1) \cup 0} \Sigma^n \langle S, A, t, \varepsilon \rangle$
(traces complètes)

Exemple 2.4-1

La sémantique engendrée par le système de transition donné en exemple 2.2-1 est définie en 2.1.2-3.

□

Dans la suite nous utiliserons les notations suivantes :

$$\Sigma^{< \ell} \langle S, A, t, \varepsilon \rangle = \bigcup_{n \in (\ell \cup 0)} \Sigma^n \langle S, A, t, \varepsilon \rangle \quad (\text{traces de longueur strictement inférieure à } \ell \in (\omega+2)),$$

(en particulier $\Sigma^{\omega}\langle S, A, t, \epsilon \rangle$ est l'ensemble des traces finies engendrées par le système de transition $\langle S, A, t, \epsilon \rangle$).

$\Sigma^{\leq l}\langle S, A, t, \epsilon \rangle = (\Sigma^l\langle S, A, t, \epsilon \rangle \cup \Sigma^l\langle S, A, t, \epsilon \rangle)$, (traces de longueur inférieure ou égale à $l \in (\omega+1)$).

2.5 RELATIONS ENTRE SEMANTIQUES ET ENTRE SYSTEMES DE TRANSITION

Notons que le système de transition $\langle S, A, T \langle S, A, \Sigma \langle S, A, T, E \rangle, E \langle S, A, \Sigma \langle S, A, T, E \rangle \rangle \rangle$ engendré par la sémantique $\langle S, A, \Sigma \langle S, A, T, E \rangle \rangle$ engendrée par le système de transition $\langle S, A, T, E \rangle$ est $\langle S, A, T, E \rangle$ lui-même. Par contre les exemples 2.3-1 et 2.4-1 montrent que la sémantique $\langle S, A, \Sigma \langle S, A, T \langle S, A, \Sigma \rangle, E \langle S, A, \Sigma \rangle \rangle$ engendrée par le système de transition $\langle S, A, T \langle S, A, \Sigma \rangle, E \langle S, A, \Sigma \rangle \rangle$ engendré par la sémantique $\langle S, A, \Sigma \rangle$ n'est en général pas cette sémantique $\langle S, A, \Sigma \rangle$ elle-même. Ceci nous amène donc à étudier dans ce paragraphe les relations qui peuvent exister entre sémantiques. Ensuite, au paragraphe 2.6, il nous sera possible d'énoncer une condition nécessaire et suffisante pour qu'une sémantique soit close (c'est-à-dire égale à la sémantique engendrée par le système de transition qu'elle engendre).

Nous incluons également dans ce paragraphe 2.5, la définition de relations entre sémantiques qu'il est nécessaire d'étudier pour justifier certaines méthodes de preuve de propriétés de programmes. En effet, pour certaines classes de propriétés, les démonstrations de correction se font plus facilement en faisant abstraction de certains aspects de la sémantique des programmes. On raisonne donc non pas sur la sémantique exacte du programme mais sur une sémantique approchée qui lui correspond selon une relation qui conserve la propriété à démontrer. Par exemple les preuves de propriétés d'invariance (correction partielle, exclusion mutuelle, ...) de programmes parallèles avec exécution équitable des processus concurrents se font beaucoup plus aisément en raisonnant sur le programme parallèle non équitable correspondant.

Pour formaliser cette méthode de preuve considérons (pour l'instant en simplifiant) qu'une propriété d'un programme est une relation entre une spécification $Sp \in \underline{\text{Spec}}$ et la sémantique $\langle s, A, \Sigma \rangle \in \underline{\text{Sem}} \langle \mathcal{L}, \mathcal{A} \rangle$ du programme. Nous avons $P \in ((\underline{\text{Spec}} \times \underline{\text{Sem}} \langle \mathcal{L}, \mathcal{A} \rangle) \rightarrow \{\text{tt}, \text{ff}\})$.

La preuve se fait en raisonnant sur une propriété $P' \in ((\underline{\text{Spec}} \times \underline{\text{Sem}} \langle \mathcal{L}, \mathcal{A} \rangle) \rightarrow \{\text{tt}, \text{ff}\})$ reliant une spécification sp' et une sémantique approchée $\langle s', A', \Sigma' \rangle$ du programme.

Pour justifier cette méthode de preuve, il faut montrer que la sémantique approchée $\langle s', A', \Sigma' \rangle$ du programme est liée à la sémantique exacte du programme $\langle s, A, \Sigma \rangle$ par une relation entre sémantiques $R \in ((\underline{\text{Sem}} \langle \mathcal{L}, \mathcal{A} \rangle \times \underline{\text{Sem}} \langle \mathcal{L}, \mathcal{A} \rangle) \rightarrow \{\text{tt}, \text{ff}\})$ qui conserve la propriété originale P à démontrer:

$$[P'(sp', \langle s', A', \Sigma' \rangle) \wedge R(\langle s', A', \Sigma' \rangle, \langle s, A, \Sigma \rangle)] \Rightarrow P(sp', \langle s, A, \Sigma \rangle)$$

(En général, la preuve est faite pour toutes spécifications sp' et sp liées par une certaine relation entre spécifications).

Enfin, une relation R entre sémantiques induit une relation entre systèmes de transition, également notée R et définie par:

$$R \in ((\underline{\text{Trans}} \langle \mathcal{L}, \mathcal{A} \rangle \times \underline{\text{Trans}} \langle \mathcal{L}, \mathcal{A} \rangle) \rightarrow \{\text{tt}, \text{ff}\})$$

$$R(\langle s', A', t', E' \rangle, \langle s, A, t, E \rangle) = R(\langle s', A', \Sigma \langle s', A', t', E' \rangle \rangle, \langle s, A, \Sigma \langle s, A, t, E \rangle \rangle)$$

De la même façon une relation R entre systèmes de transition induit une relation entre sémantiques, également notée R et définie par:

$$R \in ((\underline{\text{Sem}} \langle \mathcal{L}, \mathcal{A} \rangle \times \underline{\text{Sem}} \langle \mathcal{L}, \mathcal{A} \rangle) \rightarrow \{\text{tt}, \text{ff}\})$$

$$R(\langle s', A', \Sigma' \rangle, \langle s, A, \Sigma \rangle) = R(\langle s', A', t \langle s', A', \Sigma' \rangle \rangle, \langle s, A, t \langle s, A, \Sigma \rangle \rangle)$$

Il est donc intéressant d'étudier pour toute relation R entre sémantiques ou systèmes de transition la relation induite respectivement entre systèmes de transition et sémantiques. Donnons maintenant des exemples de relations, entre sémantiques et entre systèmes de transition, qui seront utilisées ultérieurement.

2.5.1 INCLUSION DE SEMANTIQUES ET DE SYSTEMES DE TRANSITION

on utilise la relation d'inclusion entre sémantiques quand on fait des preuves de programmes relatives à un sur- ou sous-ensemble des traces d'exécution du programme. Par exemple, on peut quelquefois démontrer une propriété d'invariance (correction partielle, ...) ou de fatalité (termination, ...) d'un programme parallèle en ignorant les hypothèses d'exécution équitables des processus concurrents (tout processus indéfiniment activable est fatalement activé, ...). Ceci vient du fait que ces propriétés étant vraies pour une sémantique $\langle S', A', \Sigma' \rangle$ le sont également pour toute sémantique $\langle S, A, \Sigma \rangle$ telle que $\Sigma \subseteq \Sigma'$.

Plus généralement, la relation d'inclusion entre sémantiques est définie par $\langle S, A, \Sigma \rangle \subseteq \langle S', A', \Sigma' \rangle$ si et seulement si $[S \subseteq S' \wedge A \subseteq A' \wedge \Sigma \subseteq \Sigma']$.

Muni de cet ordre partiel réflexif, $\text{sem} \langle \mathcal{P}, \mathcal{A} \rangle$ est un treillis complet dont l'infimum est $\langle 0, 0, 0 \rangle$ et le supremum $\langle \mathcal{P}, \mathcal{A}, \Sigma \langle \mathcal{P}, \mathcal{A} \rangle \rangle$, (où 0 désigne l'ensemble vide, cf. annexe I-2).

Le lemme qui suit caractérise la relation d'inclusion $\subseteq$ entre systèmes de transition induite par l'inclusion de sémantiques et qui est donc définie par :

$$[\langle S, A, t, E \rangle \subseteq \langle S', A', t', E' \rangle] \Leftrightarrow [S \subseteq S' \wedge A \subseteq A' \wedge \Sigma \langle S, A, t, E \rangle \subseteq \Sigma \langle S', A', t', E' \rangle]$$

Pour exprimer ce lemme nous définissons :

$$\text{Acc} \langle S, A, t, E \rangle (\Delta) = [\exists p \in \Sigma \langle S, A, t, E \rangle, i \in |p|. \Delta = p_i]$$

(qui caractérise les états accessibles d'un système de transition)

$$\text{Blo} \langle S, A, t, E \rangle (\Delta) = [\forall \Delta' \in S, a \in A. \neg t_a(\Delta, \Delta')]$$

(qui caractérise les états de blocage d'un système de transition)

et rappelons que $t_a \text{Acc}$ est la instruction gauche de t_a à Acc (cf. annexe I-2).

Lemme 2.5.1 v1

$$[\langle S, A, t, \epsilon \rangle \in \langle S', A', t', \epsilon' \rangle]$$

$$\iff [S \in S' \wedge A \in A' \wedge (\forall a \in A. t_a \upharpoonright \text{Acc} \Rightarrow t'_a) \wedge (B \downarrow \wedge \text{Acc} \Rightarrow B \downarrow') \wedge (\epsilon \Rightarrow \epsilon')]$$

avec $\text{Acc} = \text{Acc}\langle S, A, t, \epsilon \rangle$, $B \downarrow = B \downarrow \langle S, A, t, \epsilon \rangle$ et $B \downarrow' = B \downarrow \langle S', A', t', \epsilon' \rangle$.

Démonstration

($\Rightarrow$) Si $\langle S, A, t, \epsilon \rangle \in \langle S', A', t', \epsilon' \rangle$ alors par définition on a $\Sigma\langle S, A, t, \epsilon \rangle \in \Sigma\langle S', A', t', \epsilon' \rangle$. Si $t_a \upharpoonright \text{Acc}(A, A')$ est vrai, il existe une trace p de $\Sigma\langle S, A, t, \epsilon \rangle$ et $i \in |p|$ tel que $A = p_i$ et $t_a(A, A')$ et donc une trace p' de $\Sigma\langle S, A, t, \epsilon \rangle$ telle que $(i+1) \in |p'| \wedge A = p'_i \wedge A' = p'_{i+1}$. Comme $p' \in \Sigma\langle S', A', t', \epsilon' \rangle$, la définition 2.4:1 entraîne que $t'_a(A, A')$ est vrai. De même si $(B \downarrow \wedge \text{Acc})(A)$ est vrai, il existe une trace finie p de $\Sigma\langle S, A, t, \epsilon \rangle$ de longueur n telle que $p_{n-1} = A \wedge \forall a \in A, A' \in S. \neg t_a(A, A')$. Comme $p \in \Sigma\langle S', A', t', \epsilon' \rangle$, la définition 2.4:1 entraîne que $\forall a \in A, A' \in S. \neg t'_a(p_{n-1}, A')$ et donc $B \downarrow'(A)$. Enfin si $\epsilon(A)$ alors d'après 2.4:1 il existe $p \in \Sigma\langle S, A, t, \epsilon \rangle$ telle que $p_0 = A$. Comme $p \in \Sigma\langle S', A', t', \epsilon' \rangle$ nous avons $\epsilon'(p_0)$ et donc $\epsilon'(A)$.

($\Leftarrow$) Il faut démontrer que $\Sigma\langle S, A, t, \epsilon \rangle \in \Sigma\langle S', A', t', \epsilon' \rangle$. Si $p \in \Sigma\langle S, A, t, \epsilon \rangle$ nous avons $\epsilon(p_0)$ et donc $\epsilon'(p_0)$. Si $i \in |p|$ alors nous avons $\text{Acc}(p_i) \wedge t_{\#i}(p_i, p_{i+1})$ et par conséquent $t'_{\#i}(p_i, p_{i+1})$. Si p est une trace infinie, nous avons démontré que $p \in \Sigma\langle S', A', t', \epsilon' \rangle$. Si p est une trace finie de longueur n , nous avons $(\text{Acc} \wedge B \downarrow)(p_{n-1})$, donc $B \downarrow'(p_{n-1})$, qui d'après 2.4:1, implique $p \in \Sigma\langle S', A', t', \epsilon' \rangle$.

□

2.5.2 EQUIVALENCE DE SYSTEMES DE TRANSITION

La relation d'égalité entre sémantiques induit une relation d'équivalence entre systèmes de transition définie par :

$$[\langle S, A, t, \epsilon \rangle \equiv \langle S', A', t', \epsilon' \rangle] \Leftrightarrow [S = S' \wedge A = A' \wedge \Sigma \langle S, A, t, \epsilon \rangle = \Sigma \langle S', A', t', \epsilon' \rangle]$$

(Par dérogation à la convention de 2.5, nous notons $\equiv$ la relation induite, réservant $=$ à l'égalité de systèmes de transition).

Lemme 2.5.2~1

$$\begin{aligned} & [\langle S, A, t, \epsilon \rangle \equiv \langle S', A', t', \epsilon' \rangle] \\ \Leftrightarrow & [S = S' \wedge A = A' \wedge (\forall a \in A. t_a \upharpoonright \text{Acc} = t'_a \upharpoonright \text{Acc}') \wedge (B\epsilon \wedge \text{Acc} = B\epsilon' \wedge \text{Acc}') \wedge \epsilon = \epsilon'] \\ \Rightarrow & [\text{Acc} = \text{Acc}'] \end{aligned}$$

avec $\text{Acc} = \text{Acc} \langle S, A, t, \epsilon \rangle$, $\text{Acc}' = \text{Acc} \langle S', A', t', \epsilon' \rangle$, $B\epsilon = B\epsilon \langle S, A, t, \epsilon \rangle$ et $B\epsilon' = B\epsilon \langle S', A', t', \epsilon' \rangle$

Démonstration

Si $\langle S, A, t, \epsilon \rangle \equiv \langle S', A', t', \epsilon' \rangle$ alors $\Sigma \langle S, A, t, \epsilon \rangle = \Sigma \langle S', A', t', \epsilon' \rangle$ et donc par définition de Acc , nous avons $\text{Acc} \langle S, A, t, \epsilon \rangle = \text{Acc} \langle S', A', t', \epsilon' \rangle$.

$\langle S, A, t, \epsilon \rangle \equiv \langle S', A', t', \epsilon' \rangle$ si et seulement si $\langle S, A, t, \epsilon \rangle \subseteq \langle S', A', t', \epsilon' \rangle$ et $\langle S', A', t', \epsilon' \rangle \subseteq \langle S, A, t, \epsilon \rangle$ et donc d'après le lemme 2.5.1~1, si et seulement si $S = S' \wedge A = A' \wedge \forall a \in A. (t_a \upharpoonright \text{Acc} \Rightarrow t'_a \wedge t'_a \upharpoonright \text{Acc}' \Rightarrow t_a) \wedge B\epsilon \wedge \text{Acc} \Rightarrow B\epsilon' \wedge B\epsilon' \wedge \text{Acc}' \Rightarrow B\epsilon \wedge \epsilon = \epsilon'$.
Pour tout $a \in A = A'$, nous avons $[t_a \upharpoonright \text{Acc} \Rightarrow t'_a \wedge t'_a \upharpoonright \text{Acc}' \Rightarrow t_a] \Rightarrow [t_a \upharpoonright \text{Acc} \Rightarrow t'_a \upharpoonright \text{Acc} = t'_a \upharpoonright \text{Acc}' \Rightarrow t_a \upharpoonright \text{Acc}' = t_a \upharpoonright \text{Acc}] = [t_a \upharpoonright \text{Acc} = t'_a \upharpoonright \text{Acc}'] \Rightarrow [t_a \upharpoonright \text{Acc} \Rightarrow t'_a \wedge t'_a \upharpoonright \text{Acc}' \Rightarrow t_a]$. De plus $[B\epsilon \wedge \text{Acc} \Rightarrow B\epsilon' \wedge B\epsilon' \wedge \text{Acc}' \Rightarrow B\epsilon] = [B\epsilon \wedge \text{Acc} = B\epsilon' \wedge \text{Acc}']$ car $\text{Acc} = \text{Acc}'$.

□

Par abus de langage nous parlerons du système de transition qui engendre une sémantique en sous-entendant un représentant quelconque de sa classe d'équivalence.

2.5.3 CONCORDANCE ENTRE SEMANTIQUES ET ENTRE SYSTEMES DE TRANSITION A DES RELATIONS ENTRE ETATS ET/OU ACTIONS PRES

La relation de concordance entre sémantiques sera utilisée dans les justifications de méthodes de preuves pour éliminer des informations contenues dans les états ou actions. Il s'agit par exemple d'éliminer les variables auxiliaires introduites pour faciliter une démonstration ou d'identifier toutes les actions d'un même processus d'un programme parallèle.

Soient $rs \in (\mathcal{S} \times \mathcal{S} \rightarrow \{\text{tt}, \text{ff}\})$ une relation entre états et $ra \in (\mathcal{A} \times \mathcal{A} \rightarrow \{\text{tt}, \text{ff}\})$ une relation entre actions.

La relation de concordance entre traces aux relations rs entre états et ra entre actions près est définie par :

$$\simeq \langle rs, ra \rangle \in (\Sigma \langle \mathcal{S}, \mathcal{A} \rangle \times \Sigma \langle \mathcal{S}, \mathcal{A} \rangle \rightarrow \{\text{tt}, \text{ff}\})$$

$$\simeq \langle rs, ra \rangle(p, q) = [|p| = |q| \wedge \forall i \in |p|. rs(p_i, q_i) \wedge \forall i \in |p|. ra(p_i, q_i)]$$

La relation de concordance entre sémantiques aux relations rs entre états et ra entre actions près est définie par :

$$\simeq \langle rs, ra \rangle \in (\text{Sem} \langle \mathcal{S}, \mathcal{A} \rangle \times \text{Sem} \langle \mathcal{S}, \mathcal{A} \rangle \rightarrow \{\text{tt}, \text{ff}\})$$

$$\simeq \langle rs, ra \rangle \langle \langle S, A, \Sigma \rangle, \langle S', A', \Sigma' \rangle \rangle = [S' = rs[S] \wedge A' = ra[A] \wedge \Sigma' = \simeq \langle rs, ra \rangle[\Sigma]]$$

La relation induite entre systèmes de transition est la relation de concordance entre systèmes de transition aux relations rs entre états et ra entre actions près qui est définie par :

$$\simeq \langle rs, ra \rangle \in (\text{Tran} \langle \mathcal{S}, \mathcal{A} \rangle \times \text{Tran} \langle \mathcal{S}, \mathcal{A} \rangle \rightarrow \{\text{tt}, \text{ff}\})$$

$$\simeq \langle rs, ra \rangle \langle \langle S, A, E, E \rangle, \langle S', A', E', E' \rangle \rangle = \simeq \langle rs, ra \rangle \langle \langle S, A, \Sigma \langle S, A, E, E \rangle \rangle, \langle S', A', \Sigma \langle S', A', E', E' \rangle \rangle \rangle$$

Lemme 2.5.3~1

Si

$$(\exists \Delta \in S. \varepsilon(\Delta) \wedge \tau_\Delta(\Delta, \Delta')) = \varepsilon^\#(\Delta') \quad (a)$$

$$\wedge (\tau_\alpha(a, a') \wedge \tau_\alpha^{-1}(\Delta'_0, \Delta_0) \wedge t_\alpha(\Delta_0, \Delta_1) \wedge \tau_\alpha(\Delta_1, \Delta'_1)) \Rightarrow t_{\alpha'}^\#(\Delta'_0, \Delta'_1) \quad (b)$$

$$\wedge (\tau_\alpha(\Delta_0, \Delta'_0) \wedge t_{\alpha'}^\#(\Delta'_0, \Delta'_1)) \Rightarrow (\exists \Delta_1 \in S, a \in A. \tau_\alpha(\Delta_1, \Delta'_1) \wedge \tau_\alpha(a, a') \wedge t_\alpha(\Delta_0, \Delta_1)) \quad (c)$$

$$\wedge (\tau_\alpha(\Delta_0, \Delta'_0) \wedge t_\alpha(\Delta_0, \Delta_1)) \Rightarrow (\exists \Delta'_1 \in S', a' \in A'. t_{\alpha'}^\#(\Delta'_0, \Delta'_1)) \quad (d)$$

alors

$$\Sigma \langle \tau_\alpha[S], \tau_\alpha[A], t^\#, \varepsilon^\# \rangle = \{ q \in \Sigma \langle \tau_\alpha[S], \tau_\alpha[A] \rangle. \exists p \in \Sigma \langle S, A, t, \varepsilon \rangle. \preceq \langle \tau_\alpha, \tau_\alpha \rangle(p, q) \}$$

et donc

$$\preceq \langle \tau_\alpha, \tau_\alpha \rangle(\langle S, A, t, \varepsilon \rangle, \langle S', A', t', \varepsilon' \rangle) \Leftrightarrow (\langle S', A', t', \varepsilon' \rangle \equiv \langle \tau_\alpha[S], \tau_\alpha[A], t^\#, \varepsilon^\# \rangle)$$

Démonstration

Commençons par montrer que $\Sigma \langle \tau_\alpha[S], \tau_\alpha[A], t^\#, \varepsilon^\# \rangle = \{ q \in \Sigma \langle \tau_\alpha[S], \tau_\alpha[A] \rangle. \exists p \in \Sigma \langle S, A, t, \varepsilon \rangle. \preceq \langle \tau_\alpha, \tau_\alpha \rangle(p, q) \}$.

$\exists p \in \Sigma \langle S, A, t, \varepsilon \rangle. \preceq \langle \tau_\alpha, \tau_\alpha \rangle(p, q)$.

Montrons que si $\exists p \in \Sigma \langle S, A, t, \varepsilon \rangle. \preceq \langle \tau_\alpha, \tau_\alpha \rangle(p, q)$ alors $q \in \Sigma \langle \tau_\alpha[S], \tau_\alpha[A], t^\#, \varepsilon^\# \rangle$.

D'après la définition de $\preceq \langle \tau_\alpha, \tau_\alpha \rangle(p, q)$, 2.4:1, (a) et (b), nous avons $|p| = |q|$,

$\varepsilon(p_0) \wedge \tau_\alpha(p_0, q_0) \Rightarrow \varepsilon^\#(q_0)$, $\forall i \in |p|$. $(\tau_\alpha(p_i, q_i) \wedge \tau_\alpha(p_i, q_i) \wedge t_{\beta_i}(p_i, p_{i+1}) \wedge \tau_\alpha(p_{i+1}, q_{i+1})) \Rightarrow t_{\beta_i}^\#(q_i, q_{i+1})$ et donc $q \in \Sigma \langle \tau_\alpha[S], \tau_\alpha[A], t^\#, \varepsilon^\# \rangle$ si p est infinie. Si p est une trace finie de longueur n , nous avons $\forall a' \in A', \Delta \in S'. \neg t_{\alpha'}^\#(q_{n-1}, \Delta')$ car sinon $\tau_\alpha(p_{n-1}, q_{n-1})$ et (c) impliqueraient $\forall \Delta \in S, a \in A. t_\alpha(p_{n-1}, \Delta)$ en contradiction avec l'hypothèse que p est de longueur n .

Si $q \in \Sigma \langle \tau_\alpha[S], \tau_\alpha[A], t^\#, \varepsilon^\# \rangle$ alors $\exists p \in \Sigma \langle S, A, t, \varepsilon \rangle. \preceq \langle \tau_\alpha, \tau_\alpha \rangle(p, q)$. Nous construisons p tel que $|p| = |q|$ comme suit: d'après (a), $\varepsilon^\#(q_0) \Rightarrow (\exists p_0 \in S. \varepsilon(p_0) \wedge \tau_\alpha(p_0, q_0))$. Disposant de p_i tel que $\tau_\alpha(p_i, q_i)$ et $i \in |q|$, nous avons $t_{\beta_i}^\#(q_i, q_{i+1})$ et d'après (c), il existe p_{i+1} et β_i tels que $\tau_\alpha(p_{i+1}, q_{i+1})$, $\tau_\alpha(p_i, q_i)$ et $t_{\beta_i}(p_i, p_{i+1})$. Si q est infinie, ceci montre que $p \in \Sigma \langle S, A, t, \varepsilon \rangle$. Si q est finie de longueur n , il reste à montrer que $\forall a \in A, \Delta \in S. \neg t_\alpha(p_{n-1}, \Delta)$. Dans le cas contraire $\tau_\alpha(p_{n-1}, q_{n-1})$ et (d) impliqueraient l'existence de $\Delta' \in S', a' \in A'$ tels que $t_{\alpha'}^\#(q_{n-1}, \Delta')$, en contradiction avec la définition 2.4:1 d'une trace finie de longueur n .

Comme conséquence immédiate, nous obtenons $\approx \langle r_s, r_a \rangle (\langle S, A, E \rangle, \langle S', A', E' \rangle) \Leftrightarrow (\langle S', A', E' \rangle \equiv \langle r_s[S], r_a[A], t^*, e^* \rangle)$ en observant que $S' = r_s[S]$ et $A' = r_a[A]$.

□

Nous utiliserons principalement trois cas particuliers :

2.5.3.1 Concordance à une fonction des états près

Il s'agit du cas où r_s est une fonction $f_s \in (S \rightarrow S')$ et r_a la relation d'identité 1_A .

La sémantique concordante à $\langle S, A, \Sigma \rangle$ à la fonction $f_s \in (S \rightarrow S')$ des états près est donc $\approx \langle f_s \rangle (\langle S, A, \Sigma \rangle) = \langle f_s[S], A, \{ \langle m, f_s(s), a \rangle : \langle m, s, a \rangle \in \Sigma \} \rangle$ et f_s est étendue à $(\Sigma \langle S, A \rangle \rightarrow \Sigma \langle S', A' \rangle)$ par $\forall i \in |A|. f_s(A)_i = f_s(A_i)$.

2.5.3.2 Concordance à l'annulation des états près

- Quand les preuves de correction d'un programme portent uniquement sur les actions des traces, il est parfois possible d'éliminer les états en les identifiant à un état unique noté par convention s_w .

Nous définissons la sémantique concordante à $\langle S, A, \Sigma \rangle$ à l'annulation des états près comme la sémantique concordante à la fonction $f_s \in (S \rightarrow \{s_w\})$ près définie par $\forall s \in S. f_s(s) = s_w$.

- Si une sémantique $\langle S, A, \Sigma \rangle$ ne comporte qu'un seul état, nous pouvons identifier les traces $\langle m, s, a \rangle$ à des séquences d'actions a et l'ensemble Σ des traces à l'ensemble des séquences d'actions $\{a : \exists m, s. \langle m, s, a \rangle \in \Sigma\}$ quand elles ne sont pas vides sinon $\{s_w\}$ où s_w désigne par convention une action unique. Dans ce cas, cette sémantique peut donc être directement définie par un couple $\langle A, \Sigma \rangle$ où A est un ensemble d'actions et $\Sigma \subseteq \{A^{*w} \cup \{s_w\}\}$.

- Remarquons que nous pouvons toujours, sans perte d'informations, ramener une sémantique $\langle S, A, \Sigma \rangle$ à une sémantique $\langle S', A', \Sigma' \rangle$ ne comportant qu'un seul état en choisissant $S' = \{\underline{a}\}$, $A' = S \times (A \cup \{\underline{a}\})$ où $\underline{a} \notin A$ (c'est-à-dire que les actions incluent l'état dans lequel elles sont effectuées et que nous rajoutons une action unique après le dernier état d'une trace finie) et $\Sigma' = \{ \underline{a} \xrightarrow{\langle p_0, \beta_0 \rangle} \underline{a} \dots \underline{a} \xrightarrow{\langle p_{|\beta|-1}, \beta_{|\beta|-1} \rangle} \underline{a} \xrightarrow{\langle p_{|\beta|}, \underline{a} \rangle} \underline{a} : p \in \Sigma \wedge |p| < \omega \} \cup \{ \underline{a} \xrightarrow{\langle p_i, \beta_i \rangle} \underline{a} : i \in \omega : p \in \Sigma \wedge |p| = \omega \}$. Cet argument montre que nous pouvons toujours raisonner sur les actions, ce que nous faisons rarement.

2.5.3.3 Concordance à l'annulation des actions près

- Quand les preuves de correction d'un programme (comme la correction partielle) portent uniquement sur les états, il est parfois possible d'éliminer les actions. Il se peut également que les actions n'aient pas besoin d'être notées dans les traces parce que les états comportent des informations de contrôle suffisamment riches pour que la transition entre deux états ne puisse correspondre qu'à une seule action qu'il est possible de déterminer sans ambiguïté à partir de ces informations de contrôle. Éliminer les actions revient à les identifier toutes à une action unique $\underline{a}$ en choisissant $S' = S$, $A' = \{\underline{a}\}$, $\tau_s = \underline{1}$ et $\tau_a(a, a') = [a' = \underline{a}]$.

- Si une sémantique $\langle S, A, \Sigma \rangle$ ne comporte qu'une seule action, nous pouvons identifier les traces $\langle m, s, a \rangle$ à des séquences non vides d'états s et l'ensemble des traces Σ à $\{s : \exists m, a. \langle m, s, a \rangle \in \Sigma\}$. Cette sémantique peut donc être définie par un couple $\langle S, \Sigma \rangle$ où S est un ensemble d'états et $\Sigma \subseteq S^{*\omega}$.

- Remarquons que nous pouvons toujours ramener une sémantique $\langle S, A, \Sigma \rangle$ à une sémantique $\langle S', A', \Sigma' \rangle$ ne comportant qu'une seule action, en choisissant $S' = S \times (A \cup \{\underline{a}\})$ où $\underline{a} \notin A$ (c'est-à-dire que les états incluent une partie contrôle indiquant la prochaine action à effectuer ou $\underline{a}$ si

c'est la dernière), $A' = \{a\}$ et $\Sigma' = \{ \langle p_0, p_0 \rangle \xrightarrow{a} \dots \xrightarrow{a} \langle p_{|p|}, a \rangle : p \in \Sigma \wedge |p| < w \} \cup \{ \langle \langle p_i, p_i \rangle \xrightarrow{a} \langle p_{i+1}, p_{i+1} \rangle : i \in w \} : p \in \Sigma \wedge |p| = w \}$. Cet argument montre que nous pouvons toujours raisonner uniquement sur des états, ce que nous faisons souvent.

2.5.4 REDUCTION DE SEMANTIQUES

Souvent, une preuve de correction d'un programme est simplifiée en ne tenant pas compte de certains états intermédiaires du programme. Par exemple, la compilation de langages de haut niveau traduit l'évaluation d'une expression arithmétique en une suite d'instructions machine comportant des points de contrôle et des registres intermédiaires. Sous certaines conditions (absence d'effets de bord, ...) il peut être plus simple de ne pas tenir compte de cette décomposition des calculs dans les preuves. Ceci revient à considérer que l'évaluation de l'expression est indivisible, autrement dit que les états intermédiaires du calcul sont inobservables.

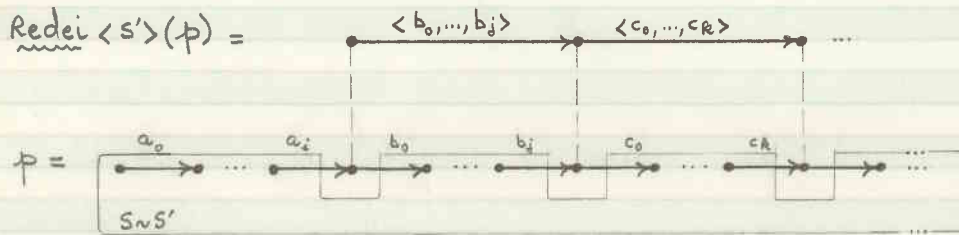
Exemple

Soit la sémantique $S = \{0, 1, 2, 3\}$, $A = \{R := X; , R := R+1; , X := R;\}$, $\Sigma = \{0 \xrightarrow{R := X;} 1 \xrightarrow{R := R+1;} 2 \xrightarrow{X := R;} 3\}$. Par élimination de l'ensemble $\{1, 2\}$ des états inobservables, nous obtenons la sémantique $S' = \{0, 3\}$, $A' = \{R := X; R := R+1; X := R;\}$, $\Sigma = \{0 \xrightarrow{R := X; R := R+1; X := R;} 3\}$.

□

2.5.4.1 Réduction des états inobservables

Etant données S, A, S' ($S \neq \emptyset \wedge S' \subseteq S$) et $A' = A^{*\omega}$, nous notons $\text{Redei} \langle S' \rangle (p)$ lorsque $(S' \cap \{p_i : i \in |p|\}) \neq \emptyset$, la trace de $\Sigma \langle S', A' \rangle$ dérivée de $p \in \Sigma \langle S, A \rangle$ par réduction des états inobservables de $S \setminus S'$. Le schéma suivant donne l'intuition de la définition.



Si $p = \langle m, A, a \rangle \in \Sigma \langle S, A \rangle$ alors $\text{Redei} \langle S' \rangle (p)$ lorsque $(S' \cap A) \neq \emptyset$ est la trace $p' = \langle m', A', a' \rangle \in \Sigma \langle S', A^{*\omega} \rangle$ définie comme suit :

Si $i \leq m$ alors $\text{card}(\{A_j \in S : j \leq i \wedge A_j \in S'\})$ est le nombre d'états observables dans p de rang strictement inférieur à i (qui peut être ω quand $i = m = \omega$).

En particulier $m' = \text{card}(\{A_j \in S : j \in m \wedge A_j \in S'\})$ est le nombre d'états observables dans p . Nous avons $m' \neq 0$. Si $k \in m'$ alors $r(k) = \sup \{i : i \in m \wedge (\text{card}(\{A_j \in S : j \leq i \wedge A_j \in S'\}) = k)\}$ est le rang (compté à partir de zéro) du $k^{\text{ème}}$ état observable de p . Nous avons $A'(k) \in (m' \rightarrow S')$ telle que $\forall k \in m' : A'(k) = A(r(k))$. Posons $a' \in ((m'-1) \rightarrow A')$ telle que $\forall k \in (m'-1) : a'(k) = a^{\langle r(k), r(k+1) \rangle}$.

Etant donnée une sémantique $\langle S, A, \Sigma \rangle \in \text{Sem} \langle \mathcal{V}, \mathcal{A} \rangle$ et un ensemble non vide d'états (dits observables) $S' \subseteq S$, la sémantique dérivée de $\langle S, A, \Sigma \rangle$ par réduction des états inobservables $S \setminus S'$ est $\text{Redei} \langle S' \rangle (\langle S, A, \Sigma \rangle) = \langle S', A^{*\omega}, \text{Redei} \langle S' \rangle [\Sigma] \rangle$.

La relation de réduction dérivée entre systèmes de transition est

$$\text{Redei} \langle S' \rangle (\langle S, A, t, E \rangle, \langle S', A', t', E' \rangle) = [\text{Redei} \langle S' \rangle (\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle) = \langle S', A', \Sigma \langle S', A', t', E' \rangle \rangle]$$

Pour étudier les propriétés de cette relation, nous utiliserons la notation suivante :

Si $t \in (A \rightarrow (S \times S \rightarrow \{\#, \# \}))$ alors nous étendons t à $((2^S)^3 \rightarrow (A^{<\omega} \rightarrow (S \times S \rightarrow \{\#, \# \})))$

par :

$$t \upharpoonright_{S_d, S_i, S_f \upharpoonright_{<}} (\Delta, \Delta') = [\Delta' = \Delta \wedge \Delta' \in S_f]$$

$$t \upharpoonright_{S_d, S_i, S_f \upharpoonright_{<a_0, \dots, a_m}} (\Delta, \Delta') = [\exists \lambda \in (m+2 \rightarrow S). (\lambda_0 = \Delta \in S_d \wedge \forall j \in (m+1 \cup 0). \lambda_j \in S_i \wedge \lambda_{m+1} = \Delta' \in S_f \wedge \forall j \in (m+1). t_{a_j}(\lambda_j, \lambda_{j+1}))]$$

c'est-à-dire que nous passons de $\Delta \in S_d$ à $\Delta' \in S_f$ par les actions $a_0, \dots, a_m$ sur des états intermédiaires dans S_i .

Lemme 2.5.4 v1

(1) Si $\langle S, A, t, \epsilon \rangle \in \text{Tran} \langle \mathcal{P}, \mathcal{A} \rangle$ est un système de transition, $S' \subseteq S$ un ensemble non vide d'états et $\epsilon^\# \in (S' \rightarrow \{\#, \# \})$, $t^\# \in (A^{<\omega} \rightarrow (S' \times S' \rightarrow \{\#, \# \}))$ sont définis par :

$$\epsilon^\#(\Delta') = [\exists \Delta \in S, \alpha \in A^{<\omega}. \epsilon(\Delta) \wedge t \upharpoonright_{S \times S', S \times S', S' \upharpoonright_\alpha} (\Delta, \Delta')] \quad (a)$$

$$t^\#_\alpha(\Delta, \Delta') = t \upharpoonright_{S', S \times S', S' \upharpoonright_\alpha} (\Delta, \Delta') \quad (b)$$

alors

$$\Sigma \langle S', A^{<\omega}, t^\#, \epsilon^\# \rangle \subseteq \text{Redei} \langle S' \rangle [\Sigma \langle S, A, t, \epsilon \rangle]$$

et donc

$$\text{Redei}(\langle S, A, t, \epsilon \rangle, \langle S', A', t', \epsilon' \rangle) \Rightarrow [\langle S', A^{<\omega}, t^\#, \epsilon^\# \rangle \subseteq \langle S', A', t', \epsilon' \rangle]$$

(2) Si de plus

$$\forall p \in \Sigma \langle S, A, t, \epsilon \rangle, i \in |p|, \alpha \in A^{<\omega}. t^\#_\alpha(p_i, \Delta') \Rightarrow [\exists j > i. j \in |p| \wedge p_j \in S'] \quad (c)$$

alors

$$\Sigma \langle S', A^{<\omega}, t^\#, \epsilon^\# \rangle = \text{Redei} \langle S' \rangle [\Sigma \langle S, A, t, \epsilon \rangle]$$

et donc

$$\text{Redei} \langle S' \rangle (\langle S, A, t, \epsilon \rangle, \langle S', A', t', \epsilon' \rangle) \Leftrightarrow [\langle S', A', t', \epsilon' \rangle \subseteq \langle S', A^{<\omega}, t^\#, \epsilon^\# \rangle]$$

Démonstration

(1) Si $q \in \Sigma \langle S', A^{K\omega}, t^\#, \varepsilon^\# \rangle$ alors nous pouvons construire $p \in \Sigma \langle S, A, t, \varepsilon \rangle$ tel que $q = \text{Redei} \langle S' \rangle (p)$. D'après 2.4:1, nous avons $\varepsilon^\#(q_0)$ et donc d'après (a), il existe meun, $p_0, \dots, p_m \in S$, $p_0, \dots, p_{m-1} \in A$ tels que $\forall i \in m$. $p_i \in (S' \cup S)$, $p_m = q_0 \in S'$, $\varepsilon(p_0)$ et $\forall i \in m$. $t_{p_i}^\#(p_i, p_{i+1})$. Posons $r(0) = m$ de sorte que $q_0 = p_{r(0)}$ et $\forall i \in r(0)$. $p_i \notin S'$. Si p a été construit jusqu'au rang $r(k)$ tel que $q_k = p_{r(k)}$ et $k \in |q|$ alors d'après 2.4:1 nous avons $t_{q_k}^\#(q_k, q_{k+1})$ et donc d'après (b), il existe m avec $m+1 = |q_{k+1}|$, $p_{r(k)+1}, \dots, p_{r(k)+m+1} \in S$, $p_{r(k)} = q_k$, $p_{r(k)+m} = q_{k+1}$ tels que $\forall i \in ((m+1) \cup 0)$. $p_{r(k)+i} \in (S \cup S')$, $p_{r(k)+m+1} = q_{k+1} \in S'$ et pour $i = r(k), \dots, r(k)+m$, nous avons $t_{p_i}^\#(p_i, p_{i+1})$. Posons $r(k+1) = r(k) + m + 1$ de sorte que $q_{k+1} = p_{r(k+1)}$ et $q_k = p_{r(k)}$. Si q est infinie alors par cette construction p l'est également et donc $p \in \Sigma \langle S, A, t, \varepsilon \rangle$. Si q est finie de longueur l , poursuivons la construction comme suit: soit $r \in \langle S, A, t, \varepsilon \rangle$ une trace finie ou infinie, avec $\varepsilon'(s) = [s = p_{r(l-1)}]$. Posons $p_{r(l-1)+i} = r_i$ pour $i \in |r|$ et $p_{r(l-1)+i} = r_i$ pour $i \in |r_s|$. Observons que $\forall i \in (|r| \cup 0)$. $r_i \notin S'$ car sinon pour le plus petit $i \in (|r| \cup 0)$ tel que $r_i \in S'$ nous aurions $t_{\langle r_0, \dots, r_{i-1} \rangle}^\#(r_0, r_i)$ en contradiction avec le fait que $q_{l-1} = p_{r(l-1)} = r_0$ n'a pas de successeur puisque q est de longueur l . Il reste à démontrer par récurrence sur k que $r(k) = \sup \{i: i \in |p| \wedge \text{card}(\{p_j \in S: j \in i \wedge p_j \in S'\}) = k\}$. Comme $\forall j \in r(0)$. $p_j \notin S'$ et $p_{r(0)} \in S'$, c'est vrai pour $k=0$. Si c'est vrai pour k alors $\text{card}(\{p_j \in S: j \in r(k) \wedge p_j \in S'\}) = k$, $p_{r(k)} \in S'$, $p_{r(k)+1}, \dots, p_{r(k+1)-1} \notin S'$ et $p_{r(k+1)} \in S'$ impliquent $\text{card}(\{p_j \in S: j \in i \wedge p_j \in S'\}) = k+1$ pour $i = r(k)+1, \dots, r(k+1)$.

Observons que $\text{Redei} \langle S' \rangle \langle \langle S, A, t, \varepsilon \rangle, \langle S', A', t', \varepsilon' \rangle \rangle$ est équivalent à $A' = A^{K\omega} \wedge \text{Redei} \langle S' \rangle [\Sigma \langle S, A, t, \varepsilon \rangle] = \Sigma \langle S', A', t', \varepsilon' \rangle$, ce qui implique $\Sigma \langle S', A^{K\omega}, t^\#, \varepsilon^\# \rangle \subseteq \Sigma \langle S', A', t', \varepsilon' \rangle$.

(2) A la suite de (1), montrons que si $\exists p \in \Sigma \langle S, A, t, \varepsilon \rangle$ tel que $q = \text{Redei} \langle S' \rangle (p)$ alors $q \in \Sigma \langle S', A^{K\omega}, t^\#, \varepsilon^\# \rangle$. Comme $(S' \cap \{p_i: i \in |p|\}) \neq \emptyset$, il existe $j \in |p|$ tel que $p_j \in S'$. $r(0)$ est le plus petit $j \in |p|$ tel que $p_j \in S'$ et nous avons $p_{r(0)} = q_0$ et donc $\varepsilon^\#(q_0)$. Si $k \in |q|$, nous avons $t_{\langle p_0, \dots, p_{r(k)-1} \rangle}^\#(p_{r(k)}, p_{r(k+1)})$, $q_k = p_{r(k)}$, $q_{k+1} = p_{r(k+1)}$ et $q_k = p_{r(k)}$ ce qui implique $t_{q_k}^\#(q_k, q_{k+1})$. D'après 2.4:1, nous avons donc $q \in \Sigma \langle S', A^{K\omega}, t^\#, \varepsilon^\# \rangle$ si q est infinie. Si q est finie de longueur l , il faut montrer que q_{l-1} n'a pas de successeur pour $t^\#$. Par l'absurde, supposons qu'il y en ait un, soit s . Il existe donc $\alpha \in A^{K\omega}$ tel que $t_\alpha^\#(q_{l-1}, s)$. Comme $q_{l-1} = p_{r(l-1)}$ il existe d'après (c) un k tel que $r(l-1) < k \leq |p| \wedge p_k \in S'$. D'autre part,

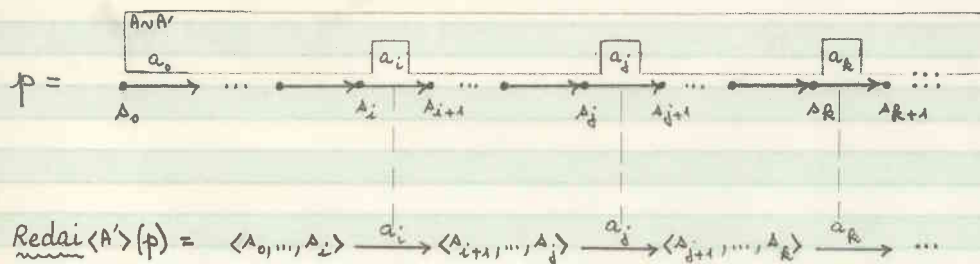
$\kappa(l-1) = \sup(\{i: i \in |p| \wedge (\text{card}(\{p_j \in S: j \in i \wedge p_j \in S'\}) = l-1)\})$ et $p_{\kappa(l-1)} \in S'$ d'où
 $\text{card}(\{p_j \in S: j \in (\kappa(l-1)+1) \wedge p_j \in S'\}) = l$. Par définition de $q = \text{Redei}\langle S' \rangle(p)$ et $|q| = l$
 nous avons $l = \text{card}(\{p_j \in S: j \in |p| \wedge p_j \in S'\})$. Nous en déduisons la contradiction
 $\forall R. (\kappa(l-1) < R < |p| \Rightarrow p_R \notin S')$.

Comme $\text{Redei}\langle S' \rangle(\langle S, A, t, \epsilon \rangle, \langle S', A', t', \epsilon' \rangle)$ est équivalent à $A' = A^{*w} \wedge$
 $\text{Redei}\langle S' \rangle[\Sigma \langle S, A, t, \epsilon \rangle] = \Sigma \langle S', A', t', \epsilon' \rangle$ soit $A' = A^{*w} \wedge \Sigma \langle S', A^{*w}, t^\#, \epsilon^\# \rangle = \Sigma \langle S', A', t', \epsilon' \rangle$
 c'est-à-dire $\langle S', A', t', \epsilon' \rangle \equiv \langle S', A^{*w}, t^\#, \epsilon^\# \rangle$.

□

2.5.4.2 Réduction des actions inobservables

De manière analogue, nous pouvons définir la réduction des actions inobservables d'une trace. Etant donnés $S, A, S' = S^{*w}$ et A' ($A' \in A \wedge A' \neq \emptyset$), nous notons $\text{Redai}\langle A' \rangle(p)$ lorsque $(A' \cap \{a_i: i \in |p|\}) \neq \emptyset$ la trace de $\Sigma \langle S', A' \rangle$ dérivée de $p \in \Sigma \langle S, A \rangle$ par réduction des actions inobservables de $A \setminus A'$. L'idée se représente informellement comme suit :



et la formalisation est tout à fait similaire à ce qui précède.

2.6 FERMETURES DE SEMANTIQUES

Nous définissons maintenant des relations entre sémantiques au moyen d'opérateurs de fermeture sur le treillis complet $\text{Sem} \langle \mathcal{L}, \mathcal{A} \rangle$ muni de l'inclusion $\subseteq$. Nous étudions, quand elle est intéressante, la relation induite entre systèmes de transition.

Dans une première partie (2.6.1 à 2.6.4) nous nous intéressons à des relations entre sémantiques qui seront ultérieurement utilisées pour justifier certaines méthodes de preuve. Il s'agit de raisonner sur les préfixes des traces (2.6.1), les suffixes des traces (2.6.2), les états et actions accessibles (2.6.3) ou les traces équitables.

Dans une deuxième partie (2.6.5 à 2.6.8) nous introduisons successivement (en 2.6.5) la notion de sémantique fermée par fusion (Pratt [79]), (en 2.6.6) la notion de sémantique réduite par élimination des traces préfixes stricts, (en 2.6.7) la notion de fermeture d'une sémantique par limites (Abrahamson [80]) ce qui permet (en 2.6.8) de donner des conditions nécessaires et suffisantes pour qu'une sémantique $\langle S, A, \Sigma \rangle$ d'une part et la sémantique $\text{Rtran}(\langle S, A, \Sigma \rangle)$ engendrée par le système de transition $\langle S, A, T \langle S, A, \Sigma \rangle, E \langle S, A, \Sigma \rangle \rangle$ engendré par $\langle S, A, \Sigma \rangle$ d'autre part soient $\subseteq$ -comparables (cf. théorèmes 2.6.8v2, 2.6.8v3) ou égales (cf. théorème 2.6.8v4). Ces résultats seront utilisés dans le paragraphe suivant pour définir des sémantiques à l'aide de systèmes de transition.

2.6.1 FERMETURE D'UNE SEMANTIQUE PAR PREFIXES

Certaines propriétés des sémantiques comme l'invariance se conservent par fermeture par préfixes.

La relation de préfixe ou facteur gauche sur $\Sigma^{\omega}\langle S, A \rangle$, définie par $(p \rightarrow q \Leftrightarrow \exists i \in |q|. p = q^{<i})$ est une relation d'ordre réflexive.

La fermeture par préfixes d'une sémantique $\langle S, A, \Sigma \rangle$ est la sémantique $\text{Pref}(\langle S, A, \Sigma \rangle) = \langle S, A, \{p \in \Sigma^{\omega}\langle S, A \rangle : \exists q \in \Sigma. p \rightarrow q\} \rangle$

Pref est un opérateur de fermeture supérieure sur $\text{Sem}\langle \mathcal{S}, \mathcal{A} \rangle$ muni de l'inclusion $\subseteq$.

La relation induite sur les systèmes de transition n'est pas intéressante car une sémantique fermée par préfixes ne peut pas être engendrée par un système de transition, sauf si toutes les traces sont réduites à un seul état.

Il est quelquefois beaucoup plus facile de faire des preuves en raisonnant sur les préfixes finis ou traces incomplètes (c'est-à-dire sur des calculs "en cours") plutôt que sur des traces complètes (c'est-à-dire sur des calculs terminés ou infinis). Dans ce cas, nous pouvons raisonner sur la fermeture par préfixes finis de la sémantique:

La fermeture par préfixes finis d'une sémantique $\langle S, A, \Sigma \rangle$ est $\text{Pref}^{<\omega}(\langle S, A, \Sigma \rangle) = \langle S, A, \{p \in \Sigma^{<\omega}\langle S, A \rangle : \exists q \in \Sigma. p \rightarrow q\} \rangle$

$\text{Pref}^{<\omega}$ n'est pas extensif (nous n'avons pas $\langle S, A, \Sigma \rangle \subseteq \text{Pref}^{<\omega}(\langle S, A, \Sigma \rangle)$ quand Σ contient une trace infinie), toutefois nous avons :

Lemme 2.6.1~1

$\text{Pref}_{\text{inf}}^{\omega}$ est un opérateur de préfermeture supérieure sur $\text{Sem} \langle \mathcal{S}, \mathcal{A} \rangle$.

De nouveau la relation induite sur les systèmes de transition est sans intérêt.

Nous utiliserons la remarque triviale que l'ensemble des préfixes d'un ensemble de traces est constitué de préfixes finis et des traces infinies :

Lemme 2.6.1~2

$$\text{Pref}_{\text{inf}}(\langle S, A, \Sigma \rangle) = \text{Pref}_{\text{inf}}^{\omega}(\langle S, A, \Sigma \rangle) \cup \langle S, A, \Sigma \cap \Sigma^{\omega} \langle S, A \rangle \rangle$$

2.6.2 FERMETURE D'UNE SEMANTIQUE OU D'UN SYSTEME DE TRANSITION PAR SUFFIXES

Certaines preuves de programmes sont relatives à un état initial qui ne correspond pas forcément au point de départ de l'exécution du programme. Nous raisonnons alors sur une fermeture de la sémantique du programme par suffixes.

La relation de suffixe ou facteur droit sur $\Sigma^{\omega} \langle S, A \rangle$ est définie par $p \rightarrow^* q \Leftrightarrow \exists i \in \mathbb{N}. p = q \gg^i$.

La fermeture par suffixes d'une sémantique $\langle S, A, \Sigma \rangle$ est la sémantique $\text{Suff}(\langle S, A, \Sigma \rangle) = \langle S, A, \{p \in \Sigma^{\omega} \langle S, A \rangle : \exists q \in \Sigma. p \rightarrow^* q\} \rangle$.

Suff est un opérateur de fermeture supérieure sur $\text{Sem} \langle \mathcal{P}, \mathcal{A} \rangle$ muni de l'inclusion $\subseteq$.

La relation induite sur les systèmes de transition est définie par $\text{Suff}(\langle S, A, t, \varepsilon \rangle, \langle S', A', t', \varepsilon' \rangle) = [\text{Suff}(\langle S, A, \Sigma \langle S, A, t, \varepsilon \rangle \rangle) = \langle S', A', \Sigma \langle S', A', t', \varepsilon' \rangle \rangle]$

Lemme 2.6.2~1

$$\text{Suff}(\langle S, A, t, \varepsilon \rangle, \langle S', A', t', \varepsilon' \rangle) \Leftrightarrow [\langle S', A', t', \varepsilon' \rangle \equiv \langle S, A, t, \varepsilon^* \rangle]$$

avec $\varepsilon^*(\lambda) = [\exists \lambda' \in S. \varepsilon(\lambda') \wedge t^*(\lambda', \lambda)]$

en notant t^* la fermeture transitive réflexive de t définie comme suit :

$$t^*(\lambda, \lambda') = \bigcup_{n \geq 0} t^n(\lambda, \lambda') \quad \text{avec} \quad t^0(\lambda, \lambda') = [\lambda' = \lambda]$$

$$t^{n+1}(\lambda, \lambda') = [\exists q \in A, \lambda'' \in S. (t_n(\lambda, \lambda'') \wedge t^n(\lambda'', \lambda'))]$$

Démonstration

Nous avons $\{p : \exists q \in \Sigma\langle S, A, t, E \rangle. p \rightarrow q\} = \Sigma\langle S, A, t, E^* \rangle$ et donc

$$\text{Suff}(\langle S, A, t, E \rangle, \langle S', A', t', E' \rangle) \Leftrightarrow [\langle S', A', \Sigma\langle S', A', t', E' \rangle \rangle = \langle S, A, \Sigma\langle S, A, t, E^* \rangle \rangle].$$

□

Suff est un opérateur de fermeture supérieure sur Trans $\langle \mathcal{V}, \mathcal{A} \rangle / \equiv$ muni de l'inclusion.

2.6.3 FERMETURE D'UNE SEMANTIQUE OU D'UN SYSTEME DE TRANSITION PAR REDUCTION AUX ETATS ET/OU ACTIONS ACCESSIBLES

Une preuve de programme souvent se simplifie en raisonnant non pas sur des états quelconques du programme mais sur l'ensemble de ceux qui sont accessibles au cours d'un calcul (ou sur un sous-ensemble de ceux-ci caractérisé par un invariant). Ceci revient à raisonner sur la réduction de la sémantique du programme aux états (et actions) accessibles.

Etant donnée une sémantique $\langle S, A, \Sigma \rangle$ la réduction aux états et actions accessibles de cette sémantique est $\text{Redeaa}(\langle S, A, \Sigma \rangle) = \langle \{s \in S : \exists p \in \Sigma, i \in |p| \cdot (p_i = s)\}, \{a \in A : \exists p \in \Sigma, j \in |p| \cdot (p_j = a)\}, \Sigma \rangle$.

Observons que Redeaa est un opérateur de fermeture inférieure sur $\text{Sem} \langle \mathcal{S}, \mathcal{A} \rangle$ muni de l'inclusion $\subseteq$.

Nous définissons de même la réduction aux états accessibles Redea et la réduction aux actions accessibles Redaa.

La relation induite sur les systèmes de transition est

$$\text{Redeaa}(\langle S, A, t, E \rangle, \langle S', A', t', E' \rangle) = [\text{Redeaa}(\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle) = \langle S', A', \Sigma \langle S', A', t', E' \rangle \rangle]$$

Lemme 2.6.3v1

$$\begin{aligned} & \text{Redeaa}(\langle S, A, t, E \rangle, \langle S', A', t', E' \rangle) \Leftrightarrow \\ & [\langle S', A', t', E' \rangle \equiv \langle \{s \in S : \exists s' \in S. (E(s') \wedge t^*(s', s))\}, \{a \in A : \exists s, s', s'' \in S. (E(s) \wedge t^*(s, s') \wedge t_a(s', s''))\}, t, E \rangle] \end{aligned}$$

Remarquons que Redeaa est un cas particulier de Redej. En effet, $\text{Redeaa}(\langle S, A, t, E \rangle, \langle S', A', t', E' \rangle) = \text{Redej}(\text{Acc}(\langle S, A, t, E \rangle), \langle S', A', t', E' \rangle)$ (où $\text{Acc} = \text{Acc} \langle S, A, t, E \rangle$).

2.6.4 FERMETURE D'UNE SEMANTIQUE PAR REDUCTION AUX TRACES EQUITABLES

Pour définir la sémantique de programmes parallèles avec hypothèse d'exécution équitable des processus, Lehman-Amueli-Stavi [81], nous pouvons spécifier une sémantique non équitable puis éliminer les traces non équitables. Cette réduction aux traces équitables évite d'avoir à spécifier un contrôleur d'exécution (scheduler) particulier de manière explicite dans la sémantique, et laisse ouvertes d'autres implémentations possibles.

Nous écrivons $\text{Enabled}(a, i, p, \Sigma)$ quand l'action a est activable au point $i \in |p|$ d'une trace p , c'est-à-dire qu'il existe une trace $q \in \Sigma$ ayant même préfixe que p jusqu'en i et a est activée en ce point :

$$\text{Enabled}(a, i, p, \Sigma) = [i \in |p| \wedge \exists q \in \Sigma. (i \in |q| \wedge q \leq^i p \wedge q_i = a)]$$

La réduction d'une sémantique aux traces faiblement équitables pour un ensemble α d'actions conserve les traces finies et les traces infinies pour lesquelles aucune action de α n'est, au delà d'un certain point continuellement activable et jamais activée :

$$\text{Wfair}(\alpha)(\langle S, A, \Sigma \rangle) =$$

$$\langle S, A, \{p \in \Sigma : (|p| = \omega) \Rightarrow \neg(\exists a \in \alpha, i \in \omega. \forall j \geq i. (\text{Enabled}(a, j, p, \Sigma) \wedge p_j \neq a))\} \rangle$$

$$\text{Wfair}(\langle S, A, \Sigma \rangle) = \text{Wfair}(\alpha)(\langle S, A, \Sigma \rangle)$$

Exemple

La réduction de la sémantique 2.1.2-3 aux traces faiblement équitables est la sémantique 2.1.2-2. La trace infinie $0 \xrightarrow{a} 0 \dots 0 \xrightarrow{a} 0 \dots$ n'est pas faiblement équitable pour $\{a, b\}$ car l'action b n'est jamais activée et toujours activable pour donner une trace $0 \xrightarrow{a} 0 \dots 0 \xrightarrow{b} 1$.

□

La réduction d'une sémantique aux traces fortement équitables pour un ensemble α d'actions conserve les traces finies et les traces infinies pour lesquelles aucune action de α n'est, au delà d'un certain point activable infiniment souvent et jamais activée :

$$\underline{Sfair} \langle \alpha \rangle (\langle S, A, \Sigma \rangle) =$$

$$\langle S, A, \{p \in \Sigma : (|p| = \omega) \Rightarrow \neg (\exists a \in \alpha, i \in \omega. (\forall j \geq i. \exists k \geq j. \text{Enabled}(a, k, p, \Sigma)) \wedge (\forall j \geq i. \#_j \neq a))\} \rangle$$

$$\underline{Sfair} (\langle S, A, \Sigma \rangle) = \underline{Sfair} \langle A \rangle (\langle S, A, \Sigma \rangle)$$

Remarquons que l'équité forte entraîne l'équité faible.

Lemme 2.6.4~1

$$(1) \quad \underline{Sfair} \langle \alpha \rangle (\langle S, A, \Sigma \rangle) \subseteq \underline{Wfair} \langle \alpha \rangle (\langle S, A, \Sigma \rangle) \subseteq \langle S, A, \Sigma \rangle$$

(2) $\underline{Wfair} \langle \alpha \rangle$ et $\underline{Sfair} \langle \alpha \rangle$ sont des opérateurs de fermeture inférieure sur $\underline{Sem} \langle \mathcal{O}, \mathcal{A} \rangle$ muni de l'inclusion $\subseteq$.

La relation induite sur les systèmes de transition n'a aucun intérêt car en général $\underline{Wfair} \langle \alpha \rangle (\Sigma \langle S, A, \mathcal{E}, \mathcal{E} \rangle)$ ne peut pas être engendré par un système de transition sur S et A .

Pour justifier les méthodes de preuve de propriétés d'invariance de programmes parallèles équitables, nous utiliserons le fait que tout préfixe fini d'une trace engendrée par un système de transition est préfixe d'une trace équitable relativement à tout ensemble fini d'actions et réciproquement :

Lemme 2.6.4~2

Si $\text{card}(\alpha) < \omega$ alors

- $\underline{Pref}^{\omega} \circ \underline{Wfair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, \mathcal{E}, \mathcal{E} \rangle \rangle) = \underline{Pref}^{\omega} (\langle S, A, \Sigma \langle S, A, \mathcal{E}, \mathcal{E} \rangle \rangle)$
- $\underline{Pref}^{\omega} \circ \underline{Sfair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, \mathcal{E}, \mathcal{E} \rangle \rangle) = \underline{Pref}^{\omega} (\langle S, A, \Sigma \langle S, A, \mathcal{E}, \mathcal{E} \rangle \rangle)$

Démonstration

Nous avons $\text{Pref}^{\omega}_{\text{fin}} \circ \text{Sfair}(\alpha)(\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) \subseteq \text{Pref}^{\omega}_{\text{fin}} \circ \text{Wfair}(\alpha)(\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) \subseteq \text{Pref}^{\omega}_{\text{fin}}(\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle)$ d'après le lemme 2.6.4v1.1 et le fait que $\text{Pref}^{\omega}_{\text{fin}}$ est monotone pour $\subseteq$.

Pour montrer que $\text{Pref}^{\omega}_{\text{fin}}(\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) \subseteq \text{Pref}^{\omega}_{\text{fin}} \circ \text{Sfair}(\alpha)(\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle)$ nous considérons un préfixe fini p de longueur n d'une trace de $\Sigma \langle S, A, t, \epsilon \rangle$ que nous prolongeons en une trace r de $\Sigma \langle S, A, t, \epsilon \rangle$ dont nous montrons qu'elle est fortement équitable. Pour la base de la construction, choisissons $r^{\omega} = p$ de sorte que r soit construit jusqu'au point $n-1$. Dans la construction de r , nous utilisons une file d'attente f qui contient toujours une fois et une seule tout élément de α et qui par hypothèse est donc finie. Notons f_ℓ la valeur de cette file au point ℓ de la construction. Initialement f_{n-1} contient toutes les actions de α dans un ordre quelconque.

Supposons que nous ayons construit r jusqu'au point ℓ et défini f_ℓ . Si r_ℓ n'a pas de successeur pour t alors la trace r est finie et donc fortement équitable, ce qui termine la démonstration. Sinon, il existe une action b activable en r_ℓ . Si aucune des actions activables en r_ℓ n'appartient à f_ℓ , nous choisissons $f_{\ell+1} = f_\ell$, $r_\ell = b$ et $r_{\ell+1}$ un élément quelconque de S tel que $t_{r_\ell}(r_\ell, r_{\ell+1})$ soit vrai. Sinon il existe une action de f_ℓ activable en r_ℓ . Dans ce cas, nous choisissons r_ℓ comme étant l'action activable de f_ℓ la plus près de la tête de la file (toutes les actions devant r_ℓ dans f_ℓ s'il y en a) n'étant donc pas activables en r_ℓ). Nous choisissons $r_{\ell+1}$ quelconque tel que $t_{r_\ell}(r_\ell, r_{\ell+1})$ soit vrai et $f_{\ell+1}$ comme étant f_ℓ à la différence que r_ℓ a été déplacé en queue de la file d'attente. Pour achever cette démonstration, il suffit de montrer que si par cette construction, nous obtenions une trace r infinie alors elle est fortement équitable. En effet, dans le cas contraire il existerait une action a de α qui est activable infiniment souvent et jamais activée au delà d'un point i et donc du point $j = \sup^+ \{m, i\}$ de r . Pour chaque entier ℓ pour lequel a est activable, r_ℓ précède a dans f_ℓ . Or ceci n'est possible qu'un nombre fini de fois.

□

2.6.5 FERMETURE D'UNE SEMANTIQUE PAR FUSIONS

En général, les évolutions possibles de l'exécution à partir d'un état d'un programme ne dépendent pas de la manière dont cet état a été atteint. Cette condition s'exprime par le fait que la sémantique du programme est fermée par fusions c'est-à-dire que tout préfixe fini d'une trace (terminé par un état s) peut se prolonger par tout suffixe (commençant par s) d'une trace :

L'extension d'une sémantique $\langle S, A, \Sigma \rangle$ par fusions est $E_{fus}(\langle S, A, \Sigma \rangle) = \langle S, A, \{p \wedge q : p \in \Sigma^{<\omega} \langle S, A \rangle \wedge q \in \Sigma^{<\omega} \langle S, A \rangle \wedge \exists p', q' \in \Sigma. (p \mapsto p' \wedge q \mapsto q')\} \rangle$

Exemple

L'extension de la sémantique $\langle \{0,1\}, \{a,b\}, \{0 \xrightarrow{b} 1, 0 \xrightarrow{a} 0 \xrightarrow{b} 1\} \rangle$ par fusions est la sémantique 2.1.2-1 c'est-à-dire $\langle \{0,1\}, \{a,b\}, \{0 \xrightarrow{b} 1, 0 \xrightarrow{a} 0 \xrightarrow{b} 1, 0 \xrightarrow{a} 0 \xrightarrow{a} 0 \xrightarrow{b} 1\} \rangle$.

□

Observons que E_{fus} est un opérateur monotone et extensif sur $\langle \text{Sem} \langle \mathcal{L}, \mathcal{A} \rangle, \subseteq \rangle$, mais n'est pas idempotent. D'où la définition suivante :

La fermeture d'une sémantique $\langle S, A, \Sigma \rangle$ par fusions est $F_{fus}(\langle S, A, \Sigma \rangle)$ où F_{fus} est le plus petit opérateur de fermeture sur $\text{Sem} \langle \mathcal{L}, \mathcal{A} \rangle$ plus grand ou égal à E_{fus} . (plus petit et plus grand étant compris par rapport à l'extension point par point de $\subseteq$).

Exemple

La fermeture par fusions de la sémantique 2.1.2-1 est la sémantique

2.1.2-2.

□

Lemme 2.6.5~1

$$(1) \quad \underline{F}_{fus} = \bigcup_{n \geq 0} \underline{E}_{fus}^n$$

$$(2) \quad \underline{E}_{fus} \circ \underline{F}_{fus} = \underline{F}_{fus}$$

Démonstration

Comme $\text{Sem} \langle \mathcal{P}, \mathcal{A} \rangle$ est un treillis complet pour $\subseteq$ et $\underline{E}_{fus}$ est monotone et extensif, le théorème 4.3, son corollaire 4.4-1 et la définition 1.13 dans Cousot - Cousot [79] impliquent que $\underline{F}_{fus}(\langle S, A, \Sigma \rangle)$ est la limite de la séquence $X_0 = \langle S, A, \Sigma \rangle$, $X_\delta = \underline{E}_{fus}(X_{\delta-1})$ si δ est un ordinal successeur et $X_\delta = \bigcup_{\alpha < \delta} X_\alpha$ si α est un ordinal limite. Par définition de $\underline{E}_{fus}$, X_δ est de forme $\langle S, A, \Sigma_\delta \rangle$ avec $(\alpha \leq \beta) \Rightarrow (\Sigma_\alpha \subseteq \Sigma_\beta)$. Pour montrer que la limite est atteinte pour $\delta = \omega$, il suffit de montrer que $\Sigma_{\omega+1} = \{p \wedge q : p \in \Sigma^{\leq \omega} \langle S, A \rangle \wedge q \in \Sigma^{\leq \omega} \langle S, A \rangle \wedge (\exists p', q' \in \bigcup_{i < \omega} \Sigma_i. p \mapsto p' \wedge q \mapsto q')\} \subseteq \Sigma_\omega = \bigcup_{k < \omega} \Sigma_k = \bigcup_{k < \omega} \{p \wedge q : p \in \Sigma^{\leq \omega} \langle S, A \rangle \wedge q \in \Sigma^{\leq \omega} \langle S, A \rangle \wedge (\exists p', q' \in \bigcup_{i < k} \Sigma_i. p \mapsto p' \wedge q \mapsto q')\}$. Nous avons bien $(\exists p', q' \in \bigcup_{i < \omega} \Sigma_i) \Rightarrow (\exists i, j \in \omega. p' \in \Sigma_i \wedge q' \in \Sigma_j) \Rightarrow (\exists i, j \in \omega. p', q' \in \Sigma_{\sup\{i, j\}}) \Rightarrow (\exists k \in \omega. p', q' \in \Sigma_k)$ car $\Sigma_i \subseteq \Sigma_{\sup\{i, j\}}$ et $\Sigma_j \subseteq \Sigma_{\sup\{i, j\}}$.

□

La relation induite sur les systèmes de transition est l'identité

car :

Lemme 2.6.5~2

$$\underline{F}_{fus}(\langle S, A, \Sigma \langle S, A, E, E \rangle \rangle) = \langle S, A, \Sigma \langle S, A, E, E \rangle \rangle$$

Démonstration

Il suffit de remarquer que $\underline{E}_{fus}(\langle S, A, \Sigma \langle S, A, E, E \rangle \rangle) = \langle S, A, \Sigma \langle S, A, E, E \rangle \rangle$ et d'appliquer le lemme 2.6.5~1.1.

□

Nous utiliserons la propriété suivante :

Lemme 2.6.5v3

$$(1) \quad \underline{E}_{fus} \circ \underline{W}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) = \underline{W}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle)$$

$$(2) \quad \underline{F}_{fus} \circ \underline{W}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) = \underline{W}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle)$$

$$(3) \quad \underline{E}_{fus} \circ \underline{S}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) = \underline{S}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle)$$

$$(4) \quad \underline{F}_{fus} \circ \underline{S}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) = \underline{S}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle)$$

Démonstration

(1), (3) : Si $P = p^{\wedge} p'$ et $Q = q^{\wedge} q$ sont des traces équitables de $\Sigma \langle S, A, t, \epsilon \rangle$ alors $p^{\wedge} q$ est une trace de $\Sigma \langle S, A, t, \epsilon \rangle$ qui est équitable car sinon une action est au delà de p toujours ou infiniment souvent activable et jamais activée dans q donc dans Q .

(2) : Par récurrence, nous avons $\underline{E}_{fus}^m \circ \underline{W}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) = \underline{W}_{fair} \langle \alpha \rangle (\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle)$ et le résultat dérive de 2.6.5v1.1. Idem pour (4).

□

2.6.6 REDUCTION D'UNE SEMANTIQUE PAR ELIMINATION DES TRACES PREFIXES STRICTS

En général, si l'exécution d'un programme peut se poursuivre à partir d'un certain état, alors elle doit se poursuivre. Cette condition correspond à l'hypothèse habituelle que les calculs progressent à une vitesse non nulle ou encore qu'il n'y a pas de blocages (pannes) dus à un agent extérieur. Cette condition s'exprime par le fait que la sémantique du programme est réduite par élimination des traces préfixes stricts c'est-à-dire qu'il n'existe pas de trace qui soit préfixe strict d'une autre trace.

La réduction d'une sémantique $\langle S, A, \Sigma \rangle$ par élimination des traces préfixes stricts est $\text{Retps}(\langle S, A, \Sigma \rangle) = \langle S, A, \{p \in \Sigma : \forall q \in \Sigma. (p \rightarrow q) \Rightarrow (p = q)\} \rangle$

Exemple

$$\text{Retps}(\langle \{0\}, \{a\}, \{0, 0 \xrightarrow{a} 0\} \rangle) = \langle \{0\}, \{a\}, \{0 \xrightarrow{a} 0\} \rangle$$

$$\text{Retps}(\langle \{0\}, \{a\}, \{0, 0 \xrightarrow{a} 0, 0 \xrightarrow{a} 0 \xrightarrow{a} 0\} \rangle) = \langle \{0\}, \{a\}, \{0 \xrightarrow{a} 0 \xrightarrow{a} 0\} \rangle$$

□

L'opérateur Retps est réductif, idempotent mais n'est pas monotone pour $\subseteq$ comme le montre le contre-exemple ci-dessus.

La relation induite sur les systèmes de transition est l'identité car :

Lemme 8.6.6 v1

$$\text{Retps}(\langle S, A, \Sigma \langle S, A, T, E \rangle \rangle) = \langle S, A, \Sigma \langle S, A, T, E \rangle \rangle$$

Observons que si $\langle S, A, \Sigma' \rangle = \text{Retps}(\langle S, A, \Sigma \rangle)$ alors Σ et Σ' ont mêmes traces infinies et par conséquent, de manière évidente, nous avons :

Lemme 2.6.6~2

$$\text{Retps} \circ \text{Wfair} \langle \alpha \rangle (\langle s, A, \Sigma \langle s, A, t, E \rangle \rangle) = \text{Wfair} \langle \alpha \rangle (\langle s, A, \Sigma \langle s, A, t, E \rangle \rangle)$$

$$\text{Retps} \circ \text{Sfair} \langle \alpha \rangle (\langle s, A, \Sigma \langle s, A, t, E \rangle \rangle) = \text{Sfair} \langle \alpha \rangle (\langle s, A, \Sigma \langle s, A, t, E \rangle \rangle)$$

2.6.7 FERMETURE D'UNE SEMANTIQUE PAR LIMITES

Certaines sémantiques ont la propriété que certaines traces d'exécution peuvent être suivies pendant un temps fini arbitrairement long mais pas pendant un temps infini.

Exemple

Pour tout $n \geq 0$, la sémantique 2.1.2-2 offre la possibilité d'exécuter n fois l'action a (puis l'action b) mais il est impossible d'exécuter l'action a un nombre infini de fois.

□

Dans le cas contraire, la sémantique est fermée par limites c'est-à-dire que si tous les préfixes finis d'une trace infinie peuvent être suivis par une exécution alors la trace infinie est un calcul légitime :

$$\text{Flim}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \cup \{p \in \Sigma^\omega \langle S, A \rangle : \forall m \in \omega. \exists q \in \Sigma. p^{\leq m} \mapsto q\} \rangle$$

Exemple

La fermeture par limites de la sémantique 2.1.2-2 est la sémantique

2.1.2-3.

□

Lemme 2.6.7 v1

Flim est un opérateur de fermeture supérieure sur $\langle \text{Sem} \langle \mathcal{A}, \mathcal{A} \rangle, \subseteq \rangle$

Démonstration

Flim est évidemment monotone et extensif. Pour montrer l'idempotence posons $L(\Sigma) = \{p \in \Sigma^\omega \langle S, A \rangle : \forall m \in \omega. \exists q \in \Sigma. p^{\leq m} \mapsto q\}$. Pour montrer que $\text{Flim}(\text{Flim}(\langle S, A, \Sigma \rangle)) = \langle S, A, \Sigma \cup L(\Sigma) \cup L(\Sigma \cup L(\Sigma)) \rangle = \text{Flim}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \cup L(\Sigma) \rangle$ il faut montrer $L(\Sigma) = L(\Sigma \cup L(\Sigma))$ soit $L(L(\Sigma)) \subseteq L(\Sigma)$ car $L(\Sigma \cup L(\Sigma)) = L(\Sigma) \cup L(L(\Sigma))$.

Si $p \in L(L(\Sigma))$ alors $\forall m \in \omega. \exists q \in L(\Sigma). p \stackrel{\leq m}{\rightarrow} q$ et donc $\exists q' \in \Sigma. p \stackrel{\leq \omega}{\rightarrow} q'$ et donc $p \in L(\Sigma)$.

□

La relation induite sur les systèmes de transition est l'identité car :

Lemme 2.6.7~2

$$\text{Flim}_{\omega}(\langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle) = \langle S, A, \Sigma \langle S, A, t, \epsilon \rangle \rangle$$

Observons que la fermeture par limites d'une sémantique n'introduit pas de nouveaux préfixes finis de traces et par conséquent deux sémantiques ayant mêmes fermetures par limites ont mêmes préfermetures par préfixes finis (la réciproque étant fausse):

Lemme 2.6.7~3

$$(1) \text{Pref}_{\omega}^{\omega} \circ \text{Flim}_{\omega} = \text{Pref}_{\omega}^{\omega}$$

$$(2) [\text{Flim}_{\omega}(\langle S_1, A_1, \Sigma_1 \rangle) = \text{Flim}_{\omega}(\langle S_2, A_2, \Sigma_2 \rangle)] \iff [\text{Pref}_{\omega}^{\omega}(\langle S_1, A_1, \Sigma_1 \rangle) = \text{Pref}_{\omega}^{\omega}(\langle S_2, A_2, \Sigma_2 \rangle)]$$

Démonstration

(1) Comme Flim_{ω} est extensif et $\text{Pref}_{\omega}^{\omega}$ monotone, nous avons $\text{Pref}_{\omega}^{\omega}(\langle S, A, \Sigma \rangle) \subseteq \text{Pref}_{\omega}^{\omega}(\text{Flim}_{\omega}(\langle S, A, \Sigma \rangle))$. Si $\langle S', A', \Sigma' \rangle = \text{Pref}_{\omega}^{\omega}(\text{Flim}_{\omega}(\langle S, A, \Sigma \rangle))$ et $r \in \Sigma'$ alors r est le préfixe fini d'une trace de Σ ou bien le préfixe fini d'une trace $p \in \Sigma^{\leq \omega} \langle S, A \rangle$ telle que si $m < \omega$, $\exists q \in \Sigma. p \stackrel{\leq m}{\rightarrow} q$. Comme $r = p^{\leq m}$, nous en déduisons que r est encore un préfixe fini d'une trace de Σ .

(2, $\Rightarrow$) Si $\text{Flim}_{\omega}(\langle S_1, A_1, \Sigma_1 \rangle) = \text{Flim}_{\omega}(\langle S_2, A_2, \Sigma_2 \rangle)$ alors $\text{Pref}_{\omega}^{\omega} \circ \text{Flim}_{\omega}(\langle S_1, A_1, \Sigma_1 \rangle) = \text{Pref}_{\omega}^{\omega} \circ \text{Flim}_{\omega}(\langle S_2, A_2, \Sigma_2 \rangle)$ et donc d'après le lemme 2.6.7~3.1, nous avons $\text{Pref}_{\omega}^{\omega}(\langle S_1, A_1, \Sigma_1 \rangle) = \text{Pref}_{\omega}^{\omega}(\langle S_2, A_2, \Sigma_2 \rangle)$.

(2, $\Leftarrow$) Le contre-exemple est $S_1 = S_2 = \{A\}$, $A_1 = A_2 = \{a\}$, $\Sigma_1 = \{A \xrightarrow{a} A, A \xrightarrow{a} A \xrightarrow{a} A \xrightarrow{a} A \xrightarrow{a} A\}$ et $\Sigma_2 = \{A \xrightarrow{a} A \xrightarrow{a} A, A \xrightarrow{a} A \xrightarrow{a} A \xrightarrow{a} A \xrightarrow{a} A\}$

□

Toutefois deux sémantiques ayant mêmes ensembles de traces finies et mêmes préfermetures par préfixes finis ont même fermeture par limites :

Lemme 2.6.7 ~ 4

$$\begin{aligned} & [(\Sigma_1 \cap \Sigma^{\omega} \langle S_1, A_1 \rangle) = (\Sigma_2 \cap \Sigma^{\omega} \langle S_2, A_2 \rangle) \wedge \text{Pref}^{\omega} \langle S_1, A_1, \Sigma_1 \rangle = \text{Pref}^{\omega} \langle S_2, A_2, \Sigma_2 \rangle] \\ \Rightarrow & [\text{Flim} \langle S_1, A_1, \Sigma_1 \rangle = \text{Flim} \langle S_2, A_2, \Sigma_2 \rangle] \end{aligned}$$

Démonstration

Posons $L(\Sigma) = \{p \in \Sigma^{\omega} \langle S, A \rangle : \forall m \in \omega. \exists q \in \Sigma. p^{\leq m} \rightarrow q\}$. Nous observons que $\text{Pref}^{\omega} \langle S_1, A_1, \Sigma_1 \rangle = \text{Pref}^{\omega} \langle S_2, A_2, \Sigma_2 \rangle$ entraîne que $S_1 = S_2$, $A_1 = A_2$, et $L(\Sigma_1) = L(\Sigma_2)$. Nous avons aussi $\Sigma \cap \Sigma^{\omega} \langle S, A \rangle \subseteq L(\Sigma)$. Le lemme dérive alors du fait que $\text{Flim} \langle S, A, \Sigma \rangle = \Sigma \cup L(\Sigma) = (\Sigma \cap \Sigma^{\omega} \langle S, A \rangle) \cup L(\Sigma)$.

□

Comme conséquence, nous obtenons que la fermeture par limites de la réduction aux traces équitables (pour un nombre fini d'actions) d'une sémantique engendrée par un système de transition est cette sémantique :

Lemme 2.6.7 ~ 5

Si $\text{card}(\alpha) < \omega$ alors

$$\begin{aligned} (1) \quad \text{Flim} \circ \text{Wfair} \langle \alpha \rangle \langle S, A, \Sigma \langle S, A, t, E \rangle \rangle &= \langle S, A, \Sigma \langle S, A, t, E \rangle \rangle \\ (2) \quad \text{Flim} \circ \text{Sfair} \langle \alpha \rangle \langle S, A, \Sigma \langle S, A, t, E \rangle \rangle &= \langle S, A, \Sigma \langle S, A, t, E \rangle \rangle \end{aligned}$$

Démonstration

Par définition, $\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle$, $\text{Wfair} \langle \alpha \rangle \langle S, A, \Sigma \langle S, A, t, E \rangle \rangle$ et $\text{Sfair} \langle \alpha \rangle \langle S, A, \Sigma \langle S, A, t, E \rangle \rangle$ ont même ensembles de traces finies. Il suffit alors d'appliquer les lemmes 2.6.4 ~ 2, 2.6.7 ~ 4 et 2.6.7 ~ 2.

□

Le résultat suivant portant sur la composition de fermetures d'une sémantique par fusions puis par limites sera utilisé ultérieurement pour caractériser les sémantiques engendrées par un système de transition :

Lemme 2.6.7 ~ 6

$$(1) \quad \underline{E}_{fus} \circ \underline{F}_{lim} \circ \underline{F}_{fus} = \underline{F}_{lim} \circ \underline{F}_{fus}$$

$$(2) \quad \underline{F}_{fus} \circ \underline{F}_{lim} \circ \underline{F}_{fus} = \underline{F}_{lim} \circ \underline{F}_{fus}$$

$$(3) \quad \underline{F}_{lim} \circ \underline{F}_{fus} \text{ est un opérateur de fermeture supérieure sur } \langle \underline{Sem} \langle \mathcal{P}, \mathcal{A} \rangle, \subseteq \rangle$$

Démonstration

(1) Posons $\langle s, A, \Sigma_1 \rangle = \underline{F}_{fus}(\langle s, A, \Sigma \rangle)$, $\langle s, A, \Sigma_2 \rangle = \underline{F}_{lim}(\langle s, A, \Sigma_1 \rangle) = \langle s, A, \Sigma_1 \cup L(\Sigma_1) \rangle$ où $L(\Sigma) = \{ p \in \Sigma^\omega \langle s, A \rangle : \forall m \in \omega. \exists q \in \Sigma. p^{\leq m} \rightarrow q \}$, $\langle s, A, \Sigma_3 \rangle = \underline{E}_{fus}(\langle s, A, \Sigma_2 \rangle)$. Pour démontrer que $\Sigma_3 = \Sigma_2$, il suffit de démontrer que $\Sigma_3 \subseteq \Sigma_2$ car $\underline{E}_{fus}$ est extensive pour $\subseteq$. Si $p \in \Sigma_3$ alors p est de la forme $r^{\wedge} q$ avec $R = (r^{\wedge} r') \in \Sigma_2$ et $Q = (q^{\wedge} q) \in \Sigma_2$. Comme $\Sigma_2 = \Sigma_1 \cup L(\Sigma_1)$, quatre cas sont à considérer :

- $R \in \Sigma_1$ et $Q \in \Sigma_1$: d'après le lemme 2.6.5 ~ 2 et la définition de Σ_1 , nous avons $\underline{E}_{fus}(\langle s, A, \Sigma_1 \rangle) = \langle s, A, \Sigma_1 \rangle$ et donc $(r^{\wedge} r') \in \Sigma_1$ et $(q^{\wedge} q) \in \Sigma_1$; nous en déduisons $p = r^{\wedge} q \in \Sigma_1 \subseteq \Sigma_2$.
- $R \in L(\Sigma_1)$ et $Q \in \Sigma_1$: il existe $r'' \in \Sigma^{\leq \omega} \langle s, A \rangle$ telle que $(r^{\wedge} r'') \in \Sigma_1$, ce qui ramène au cas précédent.

- $R \in \Sigma_1$ et $Q \in L(\Sigma_1)$: soit $i (=|\Sigma_1|)$ le rang de $p_{|\Sigma_1|}$ dans p et donc dans R de sorte que $p_i = R_i$. Comme $R \in \Sigma_1$, il existe $L \in \Sigma_1$ telle que $R^{\leq m} \rightarrow L$ pour $m \in |\mathbb{N}|$. En particulier $\forall m \leq i. p^{\leq m} \rightarrow L \in \Sigma_1$ car $p^{\leq m} = R^{\leq m}$. Comme $Q \in L(\Sigma_1)$, c'est une trace infinie et donc p l'est également. Par définition de $L(\Sigma_1)$, nous avons $\forall m \in \omega. \exists L \in \Sigma_1. q^{\leq m} \rightarrow L$. Quand m est strictement plus grand que le rang j de q dans Q , nous avons $L = ((q^{\wedge} q)^{\leq m} \wedge l) \in \Sigma_1$ soit $(q^{\wedge} q^{\leq m-j-1} \wedge l) \in \Sigma_1$, avec $l \in \Sigma^{\leq \omega} \langle s, A \rangle$. Comme $\underline{E}_{fus}(\langle s, A, \Sigma_1 \rangle) = \langle s, A, \Sigma_1 \rangle$, nous en déduisons $(r^{\wedge} q^{\leq m-j-1} \wedge l) \in \Sigma_1$ soit $(p^{\leq m+i-j} \wedge l) \in \Sigma_1$. En particulier quand $n = (m+i-j) > i$ soit $m > j$, $\exists L \in \Sigma_1. p^{\leq m} \rightarrow L$ et donc $p \in L(\Sigma_1) \subseteq \Sigma_2$.

- $R \in L(\Sigma_1)$ et $Q \in L(\Sigma_1)$: même raisonnement que précédemment car $R \in L(\Sigma_1)$ implique $\exists L \in \Sigma_1. R^{\leq m} \rightarrow L$.

(2) D'après (1) et par récurrence $\underline{E}_{fus}^n \circ \underline{F}_{lim} \circ \underline{F}_{fus} = \underline{F}_{lim} \circ \underline{F}_{fus}$ et d'après le lemme 2.6.5 v1, nous en déduisons $\underline{F}_{fus} \circ \underline{F}_{lim} \circ \underline{F}_{fus} = \underline{F}_{lim} \circ \underline{F}_{fus}$.

(3) $\underline{F}_{lim}$ et $\underline{F}_{fus}$ étant des opérateurs de fermeture, il reste à montrer l'idempotence qui résulte de (2).

□

2.6.8 RETRACTION D'UNE SEMANTIQUE PAR TRANSITIONS, SEMANTIQUE CLOSE

Dans une preuve de correction d'un programme, nous cherchons souvent à éviter les raisonnements sur les traces d'exécution d'une sémantique $\langle S, A, \Sigma \rangle$ et à les remplacer par des raisonnements sur le système de transition $\langle S, A, E, \Sigma \rangle$ que cette sémantique engendre. Cette simplification ne se justifie que si nous pouvons montrer qu'une preuve relative à la rétraction par transitions $R_{\text{tran}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \langle S, A, E, \Sigma \rangle \rangle$ de la sémantique $\langle S, A, \Sigma \rangle$ peut remplacer la preuve relative à $\langle S, A, \Sigma \rangle$. Nous étudions les propriétés de l'opérateur R_{tran} qui nous serviront pour de telles justifications.

Définition 2.6.8:1

La rétraction par transitions d'une sémantique $\langle S, A, \Sigma \rangle$ est la sémantique engendrée par le système de transition engendré par cette sémantique :

$$R_{\text{tran}} \in (\text{Sem} \langle \mathcal{S}, \mathcal{A} \rangle \rightarrow \text{Sem} \langle \mathcal{S}, \mathcal{A} \rangle)$$

$$R_{\text{tran}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \langle S, A, E \langle S, A, \Sigma \rangle, E \langle S, A, \Sigma \rangle \rangle$$

Exemples

La rétraction par transitions de la sémantique $\langle \{0\}, \{a\}, \{0 \xrightarrow{a} 0\} \rangle$ est $\langle \{0\}, \{a\}, \{0 \xrightarrow{a} 0 \dots 0 \xrightarrow{a} 0 \dots\} \rangle$. Les sémantiques 2.1.2-1, 2.1.2-2 et 2.1.2-3 ont toutes pour rétraction par transitions la sémantique 2.1.2-3.

□

R_{tran} est une rétraction (monotone et idempotent) sur $\langle \text{Sem} \langle \mathcal{S}, \mathcal{A} \rangle, \subseteq \rangle$ mais ce n'est pas un opérateur extensif (comme le montre l'exemple ci-dessus). Toutefois nous utiliserons le résultat suivant :

Théorème 2.6.8~1

Pref $\circ$ Rtran est une fermeture supérieure sur $\langle \text{Sem} \langle \mathcal{P}, \mathcal{A} \rangle, \subseteq \rangle$

Démonstration

Pref et Rtran étant monotones et idempotents, la composition l'est également. Il reste à montrer que si $\langle S, A, \Sigma_1 \rangle = \text{Pref} \circ \text{Rtran}(\langle S, A, \Sigma \rangle)$ alors $\Sigma \subseteq \Sigma_1$. C'est évident car si $p \in \Sigma$, nous avons $\varepsilon(p)$ et $\forall i \in \mathbb{N} \cdot t_{p_i}(p_i, p_{i+1})$ où $\langle S, A, t, E \rangle$ est le système de transition engendré par $\langle S, A, \Sigma \rangle$. Par conséquent, p est préfixe d'une trace de $\Sigma \langle S, A, t, E \rangle$ et donc $p \in \Sigma_1$ par définition de Pref.

□

Une sémantique et sa rétraction par transitions n'étant en général pas comparables (et en particulier, pas $\subseteq$ -comparables), le raisonnement sur une sémantique n'est pas équivalent au raisonnement sur le système de transition qu'elle engendre.

Exemple

Nous ne pouvons pas démontrer que toutes les traces des sémantiques 2.1.2-1 ou 2.1.2-2 sont finies en raisonnant sur le système de transition 2.3-1 qu'elles engendrent pour la raison que celui-ci engendre la trace infinie $0 \xrightarrow{a} 0 \dots 0 \xrightarrow{a} 0 \dots$.

□

Nous pouvons caractériser les cas où une sémantique et sa rétraction par transitions sont $\subseteq$ -comparables comme suit :

Théorème 2.6.8 n°2

$$(1) \quad [\langle s, A, \Sigma \rangle \in \underline{Rtran}(\langle s, A, \Sigma \rangle)] \Rightarrow [\langle s, A, \Sigma \rangle = \underline{Retps}(\langle s, A, \Sigma \rangle)]$$

$$(2) \quad [\langle s, A, \Sigma \rangle \in \underline{Rtran}(\langle s, A, \Sigma \rangle)] \Leftarrow [\langle s, A, \Sigma \rangle = \underline{Retps}(\langle s, A, \Sigma \rangle) \wedge \langle s, A, \Sigma \rangle = \underline{Efus}(\langle s, A, \Sigma \rangle)]$$

Démonstration

(1) Par l'absurde supposons $\langle s, A, \Sigma \rangle \in \underline{Rtran}(\langle s, A, \Sigma \rangle)$ et $\underline{Retps}(\langle s, A, \Sigma \rangle) \neq \langle s, A, \Sigma \rangle$. Comme $\underline{Retps}$ est réductif, $\exists p, q \in \Sigma. p \twoheadrightarrow q \wedge p \neq q$. Si p était une trace infinie, $p \twoheadrightarrow q$ entraînerait $p = q$, donc p est une trace finie. Comme $\langle s, A, \Sigma \rangle \in \underline{Rtran}(\langle s, A, \Sigma \rangle)$, p est une trace engendrée par le système de transition $\langle s, A, t, E \rangle$ engendré par $\langle s, A, \Sigma \rangle$ et donc $p|_{|p|}$ est un état de blocage pour t . Par ailleurs, comme $p \twoheadrightarrow q$ et $p \neq q$, nous avons $(|p|+1) \in |q|$ et donc par définition de t il vient $t_{q|_{|p|}}(q|_{|p|}, q|_{|p|+1})$, en contradiction avec $q|_{|p|} = p|_{|p|}$ d'après $p \twoheadrightarrow q$.

(2) Soit $\langle s, A, \Sigma \rangle = \underline{Rtran}(\langle s, A, \Sigma \rangle)$. Par l'absurde supposons $\langle s, A, \Sigma \rangle = \underline{Efus}(\langle s, A, \Sigma \rangle)$, $\langle s, A, \Sigma \rangle = \underline{Retps}(\langle s, A, \Sigma \rangle)$ et $\exists P \in \Sigma. P \notin \Sigma_t$.

D'après le lemme 2.6.8 n°1 et par définition de Σ_t , P est préfixe d'une trace q de Σ_t . P est un préfixe fini sinon nous aurions eu $P = q$ et $P \in \Sigma_t$. On a donc $P \in \underline{Pref}^{\omega}(\langle s, A, \Sigma_t \rangle)$.

$\langle s, A, \Sigma \rangle = \underline{Efus}(\langle s, A, \Sigma \rangle)$ et le lemme 2.6.8 n°7 entraînent $P \in \underline{Pref}^{\omega}(\langle s, A, \Sigma \rangle)$. On a donc $\exists p \in \Sigma, P \in \Sigma. P \twoheadrightarrow p$, en contradiction avec $\langle s, A, \Sigma \rangle = \underline{Retps}(\langle s, A, \Sigma \rangle)$.

□

Lemme 2.6.8 v3

$$[\underline{Rtran}(\langle S, A, \Sigma \rangle) \in \langle S, A, \Sigma \rangle]$$

 $\Leftrightarrow$

$$[\underline{Relps} \circ \underline{Efus}(\langle S, A, \Sigma \rangle) \in \langle S, A, \Sigma \rangle \quad \wedge \quad \underline{Flim} \circ \underline{Efus}(\langle S, A, \Sigma \rangle) = \underline{Efus}(\langle S, A, \Sigma \rangle)]$$

Démonstration

($\Rightarrow$) Posons $\langle S, A, \Sigma_e \rangle = \underline{Rtran}$, $\langle S, A, \Sigma_e \rangle = \underline{Efus}(\langle S, A, \Sigma \rangle)$ et supposons $\Sigma_e \in \Sigma$. Nous avons $\Sigma \in \Sigma_e$ car $\underline{Efus}$ est extensif.

Une trace de la sémantique $\underline{Relps} \circ \underline{Efus}(\langle S, A, \Sigma \rangle)$ est une trace de la forme $(p \wedge q) \in \Sigma_e$ avec $p \leftrightarrow p' \in \Sigma$ et $q' \rightarrow q \in \Sigma$ et qui n'est pas préfixe strict dans Σ_e . Par définition de $\underline{Rtran}$, nous observons que $p \wedge q$ est préfixe d'une trace $\pi \in \Sigma_e \in \Sigma \in \Sigma_e$. Comme $p \wedge q$ n'est pas préfixe strict dans Σ_e , nous avons $(p \wedge q) = \pi$ et par conséquent $(p \wedge q) \in \Sigma$.

Pour tout $p \in \Sigma^{\omega}(\langle S, A \rangle)$, nous avons $\forall m \in \omega. \exists q \in \Sigma_e. p^{<m} \rightarrow q$ qui entraîne $p \in \Sigma_e \in \Sigma_e$ et donc $\underline{Flim}(\langle S, A, \Sigma_e \rangle) = \langle S, A, \Sigma_e \rangle$.

($\Leftarrow$) Pour la réciproque, soient $\langle S, A, \Sigma_e \rangle = \underline{Efus}(\langle S, A, \Sigma \rangle)$, $\langle S, A, \Sigma_e \rangle = \underline{Relps}(\langle S, A, \Sigma_e \rangle)$ et $\langle S, A, t, \varepsilon \rangle$ le système de transition engendré par la sémantique $\langle S, A, \Sigma \rangle$.

Montrons d'abord que pour tous $p \in \Sigma^{\omega}(\langle S, A, t, \varepsilon \rangle)$, $i \in |p|$, $q \in \Sigma_e$, $p^{<i} \rightarrow q$, nous avons $\exists q' \in \Sigma_e. p^{<i} \rightarrow q'$. Posons $q^0 = q$. Si la trace q^0 de Σ_e est préfixe strict dans Σ_e , elle se prolonge en une trace q^1 de Σ_e telle que $p^{<i} \rightarrow q^1$, qui à nouveau si elle est préfixe strict, se prolonge en une trace q^2 de Σ_e telle que $p^{<i} \rightarrow q^2$. Si cette construction s'arrête après $n+1$ pas, nous obtenons une trace q^n de Σ_e telle que $p^{<i} \rightarrow q^n$, qui n'est pas préfixe strict et qui est donc élément de Σ_e . Dans ce cas nous choisissons $q' = q^n$. Si nous obtenons une suite infinie q^j , $j \geq 0$ de traces de Σ_e telles que $p^{<i} \rightarrow q^j$ et $(j < j') \Rightarrow ((q^j)^{<j} = (q^{j'})^{<j} \wedge q^j \neq q^{j'})$. Comme $\underline{Flim}(\langle S, A, \Sigma_e \rangle) = \langle S, A, \Sigma_e \rangle$ la trace limite q' définie par $q'^{<i} = p^{<i}$, $q'_j = q^j_j$ pour $j \geq i$ et $q'_j = q^0_j$ pour $j < i$ appartient à Σ_e . Etant infinie, elle n'est pas préfixe strict et appartient donc à Σ_e .

Montrons maintenant que pour tous $p \in \Sigma^{\omega}(\langle S, A, t, \varepsilon \rangle)$, $i \in |p|$, nous avons $\exists q' \in \Sigma_e. p^{<i} \rightarrow q'$.

Si $i=0$ alors $p \in \Sigma \langle S, A, t, \epsilon \rangle$ implique $\epsilon(p_0)$ et donc par définition de ϵ , $\exists r \in \Sigma. r_0 = p_0$ de sorte que $\exists r \in \Sigma. p^{\leq 0} \mapsto r$. Comme $\Sigma \in \Sigma_e$, nous déduisons du lemme ci-dessus l'existence de $q^0 \in \Sigma_\kappa$ tel que $p^{\leq 0} \mapsto q^0$.

Supposons par hypothèse d'induction pour $i \leq |p|$ et $i \neq 0$ l'existence de $q^{i-1} \in \Sigma_\kappa$ tel que $p^{\leq i-1} \mapsto q^{i-1}$. Par définition de t , $\exists q' \in \Sigma, j \in |q'|. p_{i-1} = q'_{j-1} \wedge p_{i-1} = q'_{j-1} \wedge p_i = q'_j$. Nous en déduisons que $(p^{\leq i-1} \wedge q^{\geq j}) \in \Sigma_e$ et donc l'existence de $q^i \in \Sigma_\kappa$ tel que $p^{\leq i} \mapsto q^i$.

Si $p \in \Sigma \langle S, A, t, \epsilon \rangle$ est finie alors nous avons montré qu'il existe $q^i \in \Sigma_\kappa$ tel que $i = (|p|-1)$ et $p^{\leq i} \mapsto q^i$. Comme $p \in \Sigma \langle S, A, t, \epsilon \rangle$, p_i est un état de blocage et donc $q^i = p^{\leq i}$ car sinon nous aurions $(p^{\leq i-1} \wedge q^{\geq i}) \in \Sigma$ et donc $t_{q_i}(p_i, q_{i+1}^i)$ puisque $\langle S, A, t, \epsilon \rangle$ est engendré par $\langle S, A, \Sigma \rangle$. Nous en déduisons $p = p^{\leq i} = q^i \in \Sigma$. Soit p est infinie et $\text{Flim}(\langle S, A, \Sigma_e \rangle) = \langle S, A, \Sigma_e \rangle$ de sorte que $p \in \Sigma_e$ car $\forall i \in \mathbb{N}. \exists q^i \in \Sigma_\kappa \subseteq \Sigma_e$. Comme p est infinie et $p \in \Sigma_e$, nous en déduisons $p \in \Sigma_\kappa \subseteq \Sigma$.

□

Le cas particulier où une sémantique est rétractée par transitions ou close est important car alors $\langle S, A, t \langle S, A, \Sigma \rangle, \epsilon \langle S, A, \Sigma \rangle \rangle$ engendre exactement Σ et tout raisonnement sur les traces Σ du programme peut se ramener à un raisonnement sur le système de transition $\langle S, A, t \langle S, A, \Sigma \rangle, \epsilon \langle S, A, \Sigma \rangle \rangle$ (au pire de manière triviale en engendrant Σ explicitement à partir de ce système de transition). Ceci conduit à

Définition 2.6.8:2

$$[\langle S, A, \Sigma \rangle \text{ est } \underline{\text{close}}] \iff [\underline{\text{Rtran}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle]$$

Nous pouvons caractériser les sémantiques closes par les faits que

- (α) le comportement futur de l'exécution dépend seulement de l'état courant qui a été atteint et non de la façon dont il a été atteint,
- (β) le blocage de l'exécution en un état ne dépend que de cet état et
- (γ) les limites des comportements finis sont des comportements infinis acceptables :

Théorème 2.6.8~4

$[\langle S, A, \Sigma \rangle \text{ est close}]$

$\Leftrightarrow$

$[\underline{E}_{\text{fin}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle \wedge \underline{R}_{\text{tps}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle \wedge \underline{F}_{\text{lim}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle]$

Démonstration

($\Rightarrow$) Lemmes 2.6.5~2, 2.6.6~1 et 2.6.7~2. ($\Leftarrow$) Lemmes 2.6.8~2 et 2.6.8~3.

□

(Emerson [80] a obtenu un résultat similaire avec une notion différente de sémantique utilisant des traces incomplètes). Essayons maintenant de caractériser l'opérateur $\underline{R}_{\text{tran}}$:

Lemme 2.6.8~5

$[\underline{R}_{\text{tps}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle \wedge \underline{E}_{\text{fin}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle] \Rightarrow [\underline{R}_{\text{tran}}(\langle S, A, \Sigma \rangle) = \underline{F}_{\text{lim}}(\langle S, A, \Sigma \rangle)]$

Démonstration

Si $\underline{R}_{\text{tps}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle$ et $\underline{E}_{\text{fin}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle$ alors d'après 2.6.8~2, nous avons $\langle S, A, \Sigma \rangle \in \underline{R}_{\text{tran}}(\langle S, A, \Sigma \rangle)$ et donc par monotonie $\underline{F}_{\text{lim}}(\langle S, A, \Sigma \rangle) \subseteq \underline{F}_{\text{lim}} \circ \underline{R}_{\text{tran}}(\langle S, A, \Sigma \rangle)$ ou $\underline{F}_{\text{lim}} \circ \underline{R}_{\text{tran}}(\langle S, A, \Sigma \rangle) = \underline{R}_{\text{tran}}(\langle S, A, \Sigma \rangle)$ d'après le Théorème 2.6.8~4 donc $\underline{F}_{\text{lim}}(\langle S, A, \Sigma \rangle) \subseteq \underline{R}_{\text{tran}}(\langle S, A, \Sigma \rangle)$.

$\underline{Flim} \circ \underline{Efus} \circ \underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle)$ est égal à $\underline{Flim} \circ \underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle)$ d'après le lemme 2.6.7~6.1 soit $\underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle)$ car d'après 2.6.7~1, $\underline{Flim}$ est idempotent, soit enfin $\underline{Efus} \circ \underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle)$ en appliquant à nouveau 2.6.7~6.1. D'autre part, $\underline{Retps} \circ \underline{Efus} \circ \underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle) = \underline{Retps} \circ \underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle) \subseteq \underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle)$ d'après 2.6.7~6.1 et le fait que $\underline{Retps}$ soit réductif. D'après le lemme 2.6.8~3, nous en déduisons que $\underline{Rtran} (\underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle)) \subseteq \underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle)$. Comme $\underline{Flim}$ et $\underline{Efus}$ sont ε -extensives et $\underline{Rtran}$ ε -monotone, nous avons $\underline{Rtran} (\langle S, A, \Sigma \rangle) \subseteq \underline{Rtran} (\underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle))$ et donc par transitivité $\underline{Rtran} (\langle S, A, \Sigma \rangle) \subseteq \underline{Flim} \circ \underline{Efus} (\langle S, A, \Sigma \rangle)$. Mais $\langle S, A, \Sigma \rangle = \underline{Efus} (\langle S, A, \Sigma \rangle)$ et le lemme 2.6.5~1 impliquent $\langle S, A, \Sigma \rangle = \underline{Efus} (\langle S, A, \Sigma \rangle)$. On en déduit $\underline{Rtran} (\langle S, A, \Sigma \rangle) \subseteq \underline{Flim} (\langle S, A, \Sigma \rangle)$.

□

La rétraction par transitions de la sémantique équitable (pour un nombre fini d'actions) engendrée par un système de transition est exactement la sémantique engendrée par ce système de transition :

Théorème 2.6.8~6

si $\text{card}(\alpha) < \omega$ alors

$$(1) \quad \underline{Rtran} \circ \underline{Wfair}(\alpha) \circ \underline{Rtran} = \underline{Rtran}$$

$$(2) \quad \underline{Rtran} \circ \underline{Sfair}(\alpha) \circ \underline{Rtran} = \underline{Rtran}$$

Démonstration

Nous avons $\underline{Retps} \circ \underline{Wfair}(\alpha) \circ \underline{Rtran} (\langle S, A, \Sigma \rangle) = \underline{Wfair}(\alpha) \circ \underline{Rtran} (\langle S, A, \Sigma \rangle)$ d'après 2.6.6~2 et $\underline{Efus} \circ \underline{Wfair}(\alpha) \circ \underline{Rtran} (\langle S, A, \Sigma \rangle) = \underline{Wfair}(\alpha) \circ \underline{Rtran} (\langle S, A, \Sigma \rangle)$ d'après 2.6.5~3.1 donc d'après 2.6.8~5, $\underline{Rtran} \circ \underline{Wfair}(\alpha) \circ \underline{Rtran} (\langle S, A, \Sigma \rangle)$ est égal à $\underline{Flim} \circ \underline{Efus} \circ \underline{Wfair}(\alpha) \circ \underline{Rtran} (\langle S, A, \Sigma \rangle)$ et donc à $\underline{Flim} \circ \underline{Wfair}(\alpha) \circ \underline{Rtran} (\langle S, A, \Sigma \rangle)$ d'après le lemme 2.6.5~3.2 soit encore $\underline{Rtran} (\langle S, A, \Sigma \rangle)$ d'après le lemme 2.6.7~5.1. La preuve est similaire pour (2).

□

Si une sémantique est fermée par fusions, elle a même préfixes finis que sa rétraction par transitions :

Lemme 2.6.8v7

$$[E_{fus}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle] \Rightarrow [\text{Pref}^{\omega} \circ \text{Rtran}(\langle S, A, \Sigma \rangle) = \text{Pref}^{\omega}(\langle S, A, \Sigma \rangle)]$$

Démonstration

($\Leftarrow$) Posons $\langle S, A, \Sigma_1 \rangle = \text{Rtran}(\langle S, A, \Sigma \rangle)$. Soient $p \in \Sigma_1$, $i \in |p|$ de sorte que $p^{\leq i}$ est un préfixe fini de p . Montrons qu'il est aussi préfixe fini d'une trace de Σ . $p^{\leq 0} = p_0$ est préfixe fini d'une trace de Σ car par définition de Σ_1 , nous avons $\varepsilon(p_0)$ qui implique $\exists p' \in \Sigma$. $p'_0 = p_0$. Supposons maintenant que $p^{\leq i'}$, $i' < i$ est un préfixe fini d'une trace de Σ . Donc il existe une trace $q \in \Sigma$. $q^{\leq i'} = p^{\leq i'}$. D'autre part, p étant une trace de Σ_1 , elle est engendrée par le système de transition $\langle S, A, t, \varepsilon \rangle$ engendré par $\langle S, A, \Sigma \rangle$, et nous avons $t_{\varepsilon i'}(p_{i'}, p_{i'+1})$ qui implique d'après 2.3:1 que $\exists \pi \in \Sigma$, $j \in |\pi|$. $(\pi_j = p_{i'}, \wedge \pi_j = p_{i'} \wedge \pi_{j+1} = p_{i'+1})$. $\langle S, A, \Sigma \rangle$ étant fermée par fusions, la trace $p^{\leq i'} \xrightarrow{\pi_j} p_{i'+1} = \pi_{j+1} \xrightarrow{\pi_{j+1}} \pi_{j+1}$ est aussi une trace de Σ et $p^{\leq i'+1}$ en est un préfixe fini. Finalement $p^{\leq i}$ est préfixe fini d'une trace de Σ .

($\Rightarrow$) $\langle S, A, \Sigma \rangle \subseteq \text{Pref}^{\omega} \circ \text{Rtran}(\langle S, A, \Sigma \rangle)$ d'après 2.6.8v1. Pref^{ω} étant monotone, nous avons $\text{Pref}^{\omega}(\langle S, A, \Sigma \rangle) \subseteq \text{Pref}^{\omega} \circ \text{Pref}^{\omega} \circ \text{Rtran}(\langle S, A, \Sigma \rangle) = \text{Pref}^{\omega} \circ \text{Rtran}(\langle S, A, \Sigma \rangle)$.

□

2.7 SPECIFICATION D'UNE SEMANTIQUE A L'AIDE D'UN SYSTEME DE TRANSITION

Comme nous chercherons à ramener les preuves de programmes relatives à une sémantique $\langle S, A, \Sigma \rangle$ à des preuves relatives au système de transition $\langle S, A, t\langle S, A, \Sigma \rangle, \varepsilon\langle S, A, \Sigma \rangle \rangle$ qu'elle engendre, il est naturel de chercher à spécifier la sémantique $\langle S, A, \Sigma \rangle$ à l'aide d'un système de transition $\langle S, A, t, \varepsilon \rangle$ aussi proche que possible de $\langle S, A, t\langle S, A, \Sigma \rangle, \varepsilon\langle S, A, \Sigma \rangle \rangle$.

Dans le cas d'une sémantique close (cf. 2.7.1) ces deux systèmes de transition sont équivalents.

Dans le cas d'une sémantique $\langle S, A, \Sigma \rangle$ non close (cf. 2.7.2) les traces de Σ sont préfixes des traces engendrées par $\langle S, A, t\langle S, A, \Sigma \rangle, \varepsilon\langle S, A, \Sigma \rangle \rangle$ de sorte que cette sémantique $\langle S, A, \Sigma \rangle$ peut être spécifiée par un système de transition et une condition sur les préfixes des traces qu'il engendre (cf. 2.7.1.1). Nous montrons également que cette sémantique non close $\langle S, A, \Sigma \rangle$ est engendrée à une concordance près par un système de transition $\langle S^\#, A^\#, t^\#, \varepsilon^\# \rangle$ qui inclut un contrôleur surveillant l'exécution des programmes (cf. 2.7.1.2). Nous étudions ensuite (cf. 2.7.3) le cas particulier important d'une sémantique non close réduite par élimination des traces préfixes stricts et fermée par fusions. Dans ce cas Σ est inclus dans l'ensemble des traces engendrées par $\langle S, A, t\langle S, A, \Sigma \rangle, \varepsilon\langle S, A, \Sigma \rangle \rangle$ de sorte que la sémantique $\langle S, A, \Sigma \rangle$ peut alors être spécifiée par un système de transition et une condition sur les traces qu'il engendre (cf. 2.7.3.1) ou bien comme précédemment par concordance avec une sémantique close (cf. 2.7.3.2).

2.7.1 SPECIFICATION D'UNE SEMANTIQUE CLOSE A L'AIDE DU SYSTEME DE TRANSITION QUI L'ENGENDRE

Les sémantiques closes sont caractérisées par le théorème 2.6.8~4 et peuvent d'après la définition 2.6.8:2 être définies par le système de transition qui les engendre exactement.

Exemple 2.7.1-1

- (1) La sémantique 2.1.2-3 est engendrée par l'automate 2.3-1.
 - (2) La sémantique des programmes séquentiels (du style programmes PASCAL) est réduite par élimination des traces préfixes stricts et fermée par fusions et limites et peut donc d'après le théorème 2.6.8~4 être définie par un système de transition.
 - (3) Il en va de même pour les programmes asynchrones pour lesquels aucune hypothèse d'équité à l'exécution n'est faite. Ceci correspond aux hypothèses que chaque processus est exécuté par un processeur qui calcule à une vitesse indéterminée qui peut être nulle (quand le processeur tombe en panne) et que ces processeurs ne peuvent pas tous rester en panne indéfiniment. De ce fait, si l'exécution peut globalement progresser alors elle progressera fatalement (grâce à un processus exécuté sur un processeur qui n'est pas indéfiniment en panne). Formellement ceci s'exprime par le fait que la sémantique est réduite par élimination des traces préfixes stricts.
- De plus, comme les vitesses relatives des processeurs sont inconnues (parce que les pannes sont imprévisibles ou bien parce que les processeurs calculent à des vitesses différentes sans être synchronisés), l'état suivant un état donné ne dépend pas de la façon dont celui-ci a été atteint. Formellement ceci s'exprime par le fait que la sémantique est fermée par fusions.
- Enfin, l'exécution de toute opération qui peut être reportée pour un temps arbitrairement long peut également être reportée indéfiniment, ce qui fait que la sémantique est fermée par limites.

□

2.7.2 SPECIFICATION D'UNE SEMANTIQUE NON CLOSE

Dans le cas général, deux méthodes peuvent être utilisées pour spécifier une sémantique à l'aide d'un système de transition :

2.7.2.1 Spécification par un système de transition et une condition sur les préfixes des traces qu'il engendre

D'après le théorème 2.6.8v1, nous avons toujours $\langle S, A, \Sigma \rangle \in \text{Pref} \circ \text{Rtran}(\langle S, A, \Sigma \rangle)$. Il est donc toujours possible de spécifier la sémantique $\langle S, A, \Sigma \rangle$ par le système de transition $\langle S, A, t, \epsilon \rangle$ qu'elle engendre et une condition sur les préfixes des traces engendrées par ce système de transition pour éliminer les préfixes parasites.

Exemple

La sémantique $S = \{0\}$, $A = \{a\}$, $\Sigma = \{0 \xrightarrow{a} 0, 0 \xrightarrow{a} 0 \xrightarrow{a} 0\}$ peut être spécifiée par l'automate $\langle S, A, t, \epsilon \rangle$ schématisé par :



et la restriction aux préfixes de longueur deux ou trois des traces qu'il engendre :

$$\Sigma = \{p : \exists q \in \Sigma \langle S, A, t, \epsilon \rangle. (p \rightarrow q \wedge 2 \leq |p| \leq 3)\}$$

□

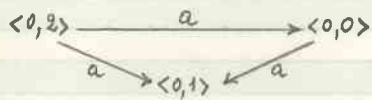
2.7.2.2 Spécification par concordance avec une sémantique close

Pour faire des preuves sur une sémantique non close $\langle S, A, \Sigma \rangle$, il n'est pas équivalent de raisonner sur le système de transition qu'elle engendre. Toutefois, il est toujours possible de trouver une sémantique close $\langle S^*, A^*, \Sigma^* \rangle$ qui concorde avec la sémantique non close $\langle S, A, \Sigma \rangle$ à une relation entre états

et actions près. Par conséquent les preuves relatives à la sémantique non close $\langle S, A, \Sigma \rangle$ peuvent se faire, à une relation entre états et actions près, sur le système de transition $\langle S^*, A^*, E^*, \epsilon^* \rangle$ engendré par cette sémantique close $\langle S^*, A^*, \Sigma^* \rangle$. Il est donc toujours possible de raisonner sur des systèmes de transition.

Exemple

La sémantique $\langle \{0\}, \{a\}, \{0 \xrightarrow{a} 0, 0 \xrightarrow{a} 0 \xrightarrow{a} 0\} \rangle$ peut être définie par la sémantique engendrée par le système de transition représenté par l'automate :



à la concordance τ_a entre états et τ_a entre actions près définies par :

$$\tau_a(\langle s, m \rangle, s') = [s = s']$$

$$\tau_a(a, a') = [a = a']$$

□

Montrons qu'il est toujours possible de se restreindre au cas simple où la concordance entre sémantiques est définie à une fonction $e^* \in (S^* \rightarrow S)$ entre états près, (cf. 2.5.3.1) :

Théorème 2.7.2.2v1

Pour toute sémantique $\langle S, A, \Sigma \rangle$, il existe une sémantique close $\langle S^*, A^*, \Sigma^* \rangle$ et une fonction $e^* \in (S^* \rightarrow S)$ entre états telle que $\langle S, A, \Sigma \rangle$ dérive de $\langle S^*, A^*, \Sigma^* \rangle$ à e^* près, c'est-à-dire :

$$\approx \langle e^* \rangle (\langle S^*, A^*, \Sigma^* \rangle) = \langle S, A, \Sigma \rangle$$

Démonstration

$\langle s^*, A^*, \Sigma^* \rangle$ peut toujours être choisie comme la sémantique engendrée par le système de transition $\langle s^*, A, t^*, \varepsilon^* \rangle$ défini par :

$$s^* = \Sigma \times \omega$$

$$t_a^*(\langle p, i \rangle, \langle p', i' \rangle) = [(i+1) \in |p| \wedge a = p_i \wedge p' = p \wedge i' = i+1]$$

$$\varepsilon^*(\langle p, i \rangle) = [i=0]$$

avec

$$e^*(\langle p, i \rangle) = p_i$$

□

En général, nous nous intéressons à des programmes exécutables par des machines et par conséquent la sémantique $\langle s^*, A^*, \Sigma^* \rangle$ dérive de $\text{Rtran}(\langle s, A, \Sigma \rangle)$ par inclusion d'un contrôleur (scheduler) surveillant l'exécution des programmes.

Observons en effet, que toute sémantique $\langle s, A, \Sigma \rangle$ peut être définie par un système de transition $\langle s, A, t, \varepsilon \rangle$ et un contrôleur $\langle s^c, A, t^c, \varepsilon^c \rangle$ de sorte que $\langle s, A, \Sigma \rangle$ concorde avec la sémantique engendrée par $\langle s^*, A, t^*, \varepsilon^* \rangle$ (où $s^* = s \times s^c$, $t_a^*(\langle s, s^c \rangle, \langle s', s'^c \rangle) = [t_a(s, s') \wedge t_a^c(s^c, s'^c)]$ et $\varepsilon^*(\langle s, s^c \rangle) = [\varepsilon(s) \wedge \varepsilon^c(s^c)]$) à une correspondance $e^* \in (s^* \rightarrow s)$ près telle que $e^*(\langle s, s^c \rangle) = s$. L'objet de l'agent extérieur $\langle s^c, A, t^c, \varepsilon^c \rangle$ est de contrôler l'initialisation et les transitions du système $\langle s, A, t, \varepsilon \rangle$. En pratique, on ne s'intéresse qu'au cas où ces fonctions et relations $t, t^c, \varepsilon, \varepsilon^c$ sont calculables.

2.7.3 SPECIFICATION D'UNE SEMANTIQUE NON CLOSE REDUITE PAR ELIMINATION DES TRACES PREFIXES STRICTS ET FERMEE PAR FUSIONS

Nous considérons maintenant le cas assez fréquent d'une sémantique qui n'est pas close mais sans traces préfixes stricts et fermée par fusions.

Exemple 2.7.3-1

(1) L'automate 2.2-1 engendre la sémantique 2.1.2-3 qui, réduite aux traces faiblement équitables pour $\langle \{a, b\} \rangle$ est la sémantique 2.1.2-2 qui n'est pas fermée par limites puisque sa fermeture par limites est précisément la sémantique 2.1.2-3, d'après le lemme 2.6.7 v 5.1. La sémantique 2.1.2-2 n'est pas close mais elle est fermée par fusions et réduite par élimination des traces préfixes stricts.

(2) Il en va de même pour les programmes parallèles avec hypothèse d'exécution faiblement équitable des processus. Ceci correspond à l'idée que les processus sont exécutés par des processeurs différents dont aucun ne tombe indéfiniment en panne de sorte que les processus peuvent s'exécuter à des vitesses différentes mais aucun processus toujours prêt ne doit attendre indéfiniment. Cette hypothèse d'équité faible a (comme le montre l'exemple 2.7.3-1.1 ci-dessus) pour conséquence que la sémantique correspondante n'est pas fermée par limites.

De plus, comme les processeurs parallèles calculent à des vitesses différentes sans priorités, tous les entremêlements possibles des opérations effectuées par chaque processeur sont réalisables de sorte que l'état suivant un état donné ne dépend pas de la façon dont cet état donné a été atteint. Formellement ceci s'exprime par le fait que la sémantique est fermée par fusions.

Enfin, si l'exécution peut globalement progresser (un processeur au moins n'étant pas bloqué) alors elle progressera fatalement (car aucun des processeurs n'a une vitesse de calcul nulle). Par conséquent, la sémantique est réduite par élimination des traces préfixes stricts.

□

Notons par rapport au cas général une simplification des méthodes (2.7.2.1 et 2.7.2.2) de spécification de sémantiques non closes à l'aide de systèmes de transitions :

2.7.3.1 Spécification par un système de transition et une condition sur les traces qu'il engendre

Lorsque $R_{\text{tps}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle$ et $E_{\text{fus}}(\langle S, A, \Sigma \rangle) = \langle S, A, \Sigma \rangle$ et $\langle S, A, \Sigma \rangle$ n'est pas close, on a $\Sigma \subset \Sigma \langle S, A, \langle S, A, \Sigma \rangle, \langle S, A, \Sigma \rangle \rangle$ d'après le théorème 2.6.8v2 et la définition 2.6.8:2. Dans ce cas, il est donc possible de spécifier la sémantique $\langle S, A, \Sigma \rangle$ par le système de transition $\langle S, A, \langle S, A, \Sigma \rangle, \langle S, A, \Sigma \rangle \rangle$ qu'elle engendre et une condition sur les traces engendrées par ce système de transition pour éliminer les traces parasites, c'est-à-dire celles de $(\Sigma \langle S, A, \langle S, A, \Sigma \rangle, \langle S, A, \Sigma \rangle \rangle \setminus \Sigma)$.

Exemples 2.7.3.1-1

(1) La sémantique 2.1.2-2 peut se définir comme étant la sémantique engendrée par l'automate 2.2-1 avec la restriction aux traces finies.

(2) La sémantique des programmes parallèles avec hypothèse d'exécution faiblement équitable des processus est fermée par fusions, réduite par élimination des traces préfixes stricts mais pas fermée par limites (2.7.2-1.2). En associant une action différente à chaque processus (action qui devient le nom du processus), nous pouvons spécifier cette sémantique $\langle S, A, \Sigma \rangle$ à l'aide du système de transition $\langle S, A, \langle S, A, \Sigma \rangle, \langle S, A, \Sigma \rangle \rangle$ et d'une condition sur les traces de la sémantique $R_{\text{tran}}(\langle S, A, \Sigma \rangle)$ ainsi définie pour éliminer les traces non équitables. Cette condition s'exprime naturellement par l'opérateur de fermeture inférieure $\text{wfin}(\alpha)$ où α est l'ensemble des noms de processus.

□

Quand nous utilisons cette méthode, nous définissons d'abord un système de transition $\langle S, A, t, E \rangle$ puis une condition C sur les traces pour obtenir la sémantique $\langle S, A, \{p \in \Sigma^* \langle S, A, t, E \rangle : C(p)\} \rangle$. Pour faire des preuves, nous chercherons à faire des raisonnements utilisant un système de transition plutôt que des traces. Il est possible d'utiliser $\langle S, A, t, E \rangle$ ou le système de transition qui engendre $\text{Rtran}(\langle S, A, \{p \in \Sigma^* \langle S, A, t, E \rangle : C(p)\} \rangle)$. Ces deux systèmes de transition sont en général différents. Il se pose alors le problème de savoir s'il est plus facile de spécifier la sémantique $\langle S, A, \Sigma \rangle$ par un moyen quelconque et d'en déduire le système de transition $\langle S, A, t \langle S, A, \Sigma \rangle, E \langle S, A, \Sigma \rangle \rangle$ ou si à l'inverse il vaut mieux commencer par spécifier un système de transition $\langle S, A, t, E \rangle$ et en déduire $\langle S, A, \Sigma \rangle$ par restriction de la sémantique $\langle S, A, \Sigma \langle S, A, t, E \rangle \rangle$. Sans pouvoir apporter de réponse générale à ce problème, une réponse est possible (ces deux systèmes de transition sont équivalents) dans chacun des cas particuliers que nous traitons.

Exemple 2.7.3.1-2

Le théorème 2.6.8v6 montre que dans le cas de programmes parallèles avec hypothèse d'exécution faiblement ou fortement équitable, il revient au même de spécifier la sémantique puis d'en déduire le système de transition ou de spécifier le système de transition puis d'en déduire la sémantique. Cette seconde solution est donc généralement retenue car un système de transition qui définit un pas du calcul est plus simple à spécifier qu'une sémantique qui définit une suite de pas de calcul.

□

2.7.3.2 Spécification par concordance avec une sémantique close

La technique est la même que dans le cas général:

Exemple 2.7.3.2-1

Nous pouvons définir la sémantique 2.1.2-2, version faiblement équitable de 2.1.2-3, en ajoutant un contrôleur à l'automate 2.2-1 comme suit :

$$S^\# = \{0,1\} \times \omega$$

$$A^\# = \{a,b\}$$

$$t_a^\#(\langle \Delta, m \rangle, \langle \Delta', m' \rangle) = [\Delta = \Delta' = 0 \wedge m' = m-1], \quad t_b^\#(\langle \Delta, m \rangle, \langle \Delta', m' \rangle) = [\Delta = 0 \wedge \Delta' = 1]$$

$$\varepsilon^\#(\langle \Delta, m \rangle) = [\Delta = 0]$$

et en définissant la fonction $e^\#(\langle \Delta, m \rangle) = \Delta$.

□

Exemple 2.7.3.2-2

Nous pouvons définir la réduction aux traces faiblement équitables $\text{Wfair}(\langle S, A, \Sigma \langle S, A, t, \varepsilon \rangle \rangle)$ d'une sémantique $\langle S, A, \Sigma \langle S, A, t, \varepsilon \rangle \rangle$ engendrée par un système de transition $\langle S, A, t, \varepsilon \rangle$, en ajoutant un contrôleur au système de transition $\langle S, A, t, \varepsilon \rangle$ comme suit :

$$S^\boxtimes = (A \rightarrow \omega) \times S$$

$$t^\boxtimes(\langle m, \Delta \rangle, \langle m', \Delta' \rangle) = \left[\exists k \in A. \begin{array}{l} t_k(\Delta, \Delta') \\ \wedge \left([m_k > 0 \wedge m'_k < m_k \wedge \forall j \in (A \setminus k). m'_j = m_j] \right. \right. \\ \left. \left. [\forall j \in A. ((\forall \Delta' \in S. \neg t_j(\Delta, \Delta') \vee m_j = 0) \wedge m'_j > 0)] \right) \right] \end{array}$$

$$\varepsilon^\boxtimes(\langle m, \Delta \rangle) = \varepsilon(\Delta)$$

et par concordance à la fonction $f^\boxtimes$ des états près, définie par :

$$f^\boxtimes(\langle m, \Delta \rangle) = \Delta$$

(Observons que le contrôleur organise l'exécution en reprises. Au début de la reprise, est déterminé le nombre maximal m_k de fois l'action $k \in A$ sera activée pendant la reprise. Si l'action k est activée dans l'état Δ ,

le nombre maximum d'activations dans la reprise décroît strictement pour k et reste inchangé pour les autres actions. Une nouvelle reprise peut commencer dans l'état s si toute action $k \in A$ n'est pas activable dans l'état s ou a été activée au moins une fois dans la reprise précédente).

Lemme 2.7.3.2-2^o1

$$\simeq \langle f_s^\square \rangle (\langle S^\square, A, \Sigma \langle S^\square, A, t^\square, \varepsilon^\square \rangle \rangle) = \underset{\sim}{W}_{\text{fair}} (\langle S, A, \Sigma \langle S, A, t, \varepsilon \rangle \rangle)$$

Démonstration

Montrons que si $p^\square \in \Sigma \langle S^\square, A, t^\square, \varepsilon^\square \rangle$ alors $f_s^\square(p^\square)$ est une trace faiblement équitable de $\Sigma \langle S, A, t, \varepsilon \rangle$. C'est évident si $p^\square$ est une trace finie. Sinon, il existe $m \in (w \rightarrow (A \rightarrow w))$ et $p \in (w \rightarrow S)$ tels que $\forall i \in w. (p_i^\square = \langle m_i, p_i \rangle)$. Supposons que $p = f_s^\square(p^\square)$ ne soit pas faiblement équitable pour A , de sorte que $\exists k \in A, i \in w. \forall j \geq i. (\exists s \in S. t_k(p_j, s) \wedge \neg t_k(p_j, p_{j+1}))$. Dans une reprise, la somme des $m_{j,k}$ pour $k \in A$ décroît strictement à chaque pas, de sorte qu'aucune reprise ne peut être infinie. En particulier, la reprise à laquelle " i appartient" doit se terminer en $i' \geq i$. Au début de la reprise suivante suivante, nous avons $m_{(i'+1),k} > 0$ et les $m_{i,k}$ ne sont pas modifiés pendant cette reprise puisque l'action k n'est jamais activée. Quand cette dernière reprise se termine en $i'' \geq i'$, nous avons $\exists s \in S. t_k(p_{i''}, s) \wedge m_{i'',k} = m_{(i'+1),k} > 0$ en contradiction avec la définition de $t^\square$.

Il reste à montrer que si p est une trace de $\Sigma \langle S, A, t, \varepsilon \rangle$, faiblement équitable pour A , nous pouvons construire $m \in (|p| \rightarrow (A \rightarrow w))$ tel que $p^\square$ définie par $(|p^\square| = |p| \wedge \forall i \in |p|. p_i^\square = \langle m_i, p_i \rangle)$ (et donc telle que $f_s^\square(p^\square) = p$) appartienne à $\Sigma \langle S^\square, A, t^\square, \varepsilon^\square \rangle$.

Si $|p| = 1$, posons $m_{0,k} = 0$

Si $1 < |p| < w$, posons $m_{i,k} = \sum_{j=i}^{|p|-2} (t_k(p_j, p_{j+1}) \rightarrow 1/0)$

Si $|p| = w$, alors grâce à l'hypothèse d'équité faible, nous pouvons définir $s \in (w \rightarrow w)$ par

$$\delta_0 = 0 \text{ et } \delta_{i+1} = 1 \{j \in \omega : \forall k \in m. [(\forall l \geq \delta_i. \forall a \in s. \neg t_k(p_l, a)) \vee (\exists l \in \omega. \delta_i \leq l < j \wedge t_k(p_l, p_{l+1}))]\}$$

de sorte que toutes les actions qui ne sont pas continuellement bloquées après δ_i sont activées au moins une fois entre δ_i et δ_{i+1} . Nous définissons

$\eta \in (\omega \rightarrow \omega)$ tel que η_i est le δ_{j+1} tel que $\delta_j \leq i < \delta_{j+1}$. Posons

$$m_{i_k} = \sum_{j=i}^{\eta_i} (t_k(p_j, p_{j+1}) \rightarrow 1/0).$$



2.8 EXEMPLE DE DEFINITION DE LA SEMANTIQUE D'UN LANGAGE DE PROGRAMMATION

Nous définissons la sémantique d'un langage que nous utiliserons ultérieurement comme exemple.

Les programmes (P_r) peuvent être séquentiels (P_s), parallèles asynchrones (P_{pa}), parallèles communiquant par envois de messages sur rendez-vous (P_{pc}), parallèles faiblement équitables (P_{pw}) ou parallèles synchronisés par sémaphores (P_{ps})

$$P_r \rightarrow P_s \mid P_{pa} \mid P_{pc} \mid P_{pw} \mid P_{ps}$$

Après avoir défini la syntaxe de ces programmes, nous en définissons la sémantique. Pour cela nous associons à tout programme P_r syntaxiquement correct un système de transition $\langle S[P_r], A[P_r], E[P_r], \varepsilon[P_r] \rangle$ qui est spécifié par induction sur la syntaxe du programme.

La sémantique des programmes séquentiels, parallèles asynchrones et parallèles communicants est close. Elle peut donc être définie comme la sémantique engendrée par ce système de transition, (cf. 2.7.1). La sémantique des programmes parallèles faiblement équitables et celle des programmes avec sémaphores est réduite par élimination de traces préfixes stricts, fermée par fusion mais n'est pas close. Elle sera définie par élimination des traces non équitables de l'ensemble des traces engendrées par ce système de transitions (cf. 2.7.3.1) ou bien à une fonction des états pès en incorporant un contrôleur dans ce système de transition (cf. 2.7.3.2).

2.8.1 PROGRAMMES SEQUENTIELS

Un programme séquentiel est une suite de commandes exécutées en séquence. Une commande peut être nulle, une affectation, une affectation aléatoire ou une composition alternative ou itérative de commandes. Chaque commande est précédée et suivie par une étiquette qui n'apparaît qu'une seule fois dans le programme et qui sert uniquement à désigner des points de contrôle du programme. Pour simplifier, les déclarations de types et de variables sont omises.

2.8.1.1 Syntaxe

Soient $\mathcal{L}$, $\mathcal{V}$, $\mathcal{E}$ et $\mathcal{B}$ des ensembles non vides donnés, respectivement d'étiquettes, de variables, d'expressions et d'expressions booléennes dont nous omettrons la syntaxe pour simplifier.

Les ensembles $\mathcal{P}_s$ de programmes séquentiels et $\mathcal{C}$ de commandes séquentielles sont définis par la syntaxe suivante :

$$\mathcal{P}_s \rightarrow \mathcal{L}_0 : \mathcal{C}_0 ; \dots ; \mathcal{L}_{m-1} : \mathcal{C}_{m-1} ; \mathcal{L}_m : \quad (m > 0)$$

$$\mathcal{C} \rightarrow \underline{\text{skip}} \mid \mathcal{V} := \mathcal{E} \mid \mathcal{V} := ? \mid \text{if } \mathcal{B} \text{ then } \mathcal{P}_s \text{ else } \mathcal{P}_s \text{ fi} \mid \text{while } \mathcal{B} \text{ do } \mathcal{P}_s \text{ od}$$

(if $\mathcal{B}$ then $\mathcal{P}_s$ fi est l'abréviation de if $\mathcal{B}$ then $\mathcal{P}_s$ else skip fi)

Soient N une chaîne terminale dérivant du non-terminal $\mathcal{N}$, α_i des chaînes éventuellement vides et N_i des chaînes terminales non vides dérivant respectivement des non-terminaux $\mathcal{N}_i$, nous utiliserons la notation $N \equiv \alpha_0 N_0 \dots \alpha_m N_m \alpha_{m+1}$ pour $\exists \alpha_0, \dots, \alpha_{m+1} \cdot (\dots N = \alpha_0 N_0 \dots \alpha_m N_m \alpha_{m+1} \dots)$, le quantificateur étant convenablement placé pour lier toutes les occurrences des $\alpha_0, \dots, \alpha_{m+1}$ dans la formule où ils sont utilisés.

Cette notation sera utilisée pour exprimer des conditions de contexte sur les programmes comme par exemple qu'une étiquette ne peut

apparaître plus d'une fois dans un programme :

$$\forall P_1 \in \mathcal{P}. (P_1 \equiv \alpha L_1 : \beta L_2 : \delta) \Rightarrow (L_1 \neq L_2)$$

2.8.1.2 Sémantique

2.8.1.2.1 Etats

Un état d'un programme séquentiel P_s est une paire $\langle \text{état de contrôle, état mémoire} \rangle$. L'état de contrôle est similaire à un compteur de programme. Il est représenté par une étiquette L qui désigne un point unique du programme. Supposons que les variables prennent leurs valeurs dans un domaine $\mathcal{D}$ (que nous ne spécifions pas pour simplifier). L'état mémoire M est un élément de $\mathcal{M} = (\mathcal{V} \rightarrow \mathcal{D})$ c'est-à-dire une fonction totale de l'ensemble $\mathcal{V}$ des variables dans l'ensemble $\mathcal{D}$ des valeurs des variables.

Formellement, l'ensemble $S[P_s]$ des états du programme P_s est :

$$S[P_s] = \{L \in \mathcal{L} : P_s \equiv \alpha L : \beta\} \times \mathcal{M}$$

2.8.1.2.2 Actions

N'ayant pas besoin dans la suite de nommer les actions qui sont exécutées par les programmes séquentiels, nous définissons :

$$A[P_s] = \{\underline{a}\}$$

où $\{\underline{a}\}$ est l'action unique que nous omettrons dans la suite.

2.8.1.2.3 Etats initiaux

Les états initiaux d'un programme séquentiel P_s sont caractérisés par:

$$\varepsilon[P_s] \in (S[P_s] \rightarrow \{\text{tt}, \text{ff}\})$$

$$\varepsilon[P_s](\langle L, M \rangle) = [P_s \equiv L : \alpha]$$

2.8.1.2.4 Relation de transition

La relation de transition $t[P_s]$ entre un état $\langle L, M \rangle$ du programme séquentiel P_s et ses successeurs $\langle L', M' \rangle$ (s'il en existe) est définie par:

$$t[P_s] \in ((S[P_s] \times S[P_s]) \rightarrow \{\text{tt}, \text{ff}\})$$

$$t[P_s](\langle L, M \rangle, \langle L', M' \rangle) = [\text{cond}[P_s](L, L')(M) \wedge \text{succ}[P_s](L)(M, M')]$$

$\langle L', M' \rangle$ est l'état successeur de $\langle L, M \rangle$ si et seulement si l'exécution d'un pas du programme séquentiel P_s au point de contrôle L dans l'état mémoire M a pour effet de déplacer le contrôle en L' (tel que $\text{cond}[P_s](L, L')(M)$ est vrai) et de changer l'état mémoire en M' (tel que $\text{succ}[P_s](L)(M, M')$ est vrai).

L'état mémoire ne peut être modifié que par une commande d'affectation. Une affectation aléatoire $V := ?$ affecte une valeur quelconque de $\mathcal{D}$ à la variable V . Une affectation $V := E$ affecte à V la valeur $\llbracket E \rrbracket(M)$ de l'expression E dans l'état mémoire M à la variable V . Pour simplifier, la sémantique $\llbracket E \rrbracket : (\mathcal{E} \rightarrow (\mathcal{V} \rightarrow \mathcal{D}) \rightarrow \mathcal{D})$ des expressions n'est pas spécifiée.

$$\begin{aligned} \text{succ}[P_s](L)(M, M') = & (P_s \equiv \alpha L : V := E; \beta \Rightarrow \\ & \forall W \in (\mathcal{V} \sim \mathcal{V}). [M'(W) = M(W)] \wedge M'(V) = \llbracket E \rrbracket(M) \\ & | (P_s \equiv \alpha L : V := ?; \beta \Rightarrow \\ & \quad \forall W \in (\mathcal{V} \sim \mathcal{V}). [M'(W) = M(W)] \\ & | M' = M) \end{aligned}$$

Pour définir le contrôle nous supposons donnée la sémantique $B \in (\mathcal{B} \rightarrow ((\mathcal{V} \rightarrow \mathcal{D}) \rightarrow \{\text{tt}, \text{ff}\}))$ des expressions booléennes telle que $B[B](M)$ soit la valeur de l'expression booléenne B dans l'état mémoire M . $E[E]$ et $B[B]$ sont des fonctions partielles pour rendre compte des erreurs possibles à l'exécution. La relation de transition cond $\llbracket Ps \rrbracket (L, L')(M)$ entre l'état de contrôle L et son successeur L' dans l'état mémoire M est défini par cas comme suit :

$$\begin{aligned} \text{cond} \llbracket Ps \rrbracket (L, L')(M) = & \\ & [\\ & \quad [\\ & \quad \vee \quad Ps \equiv \alpha L: \text{skip}; L': \beta \\ & \quad \vee \quad Ps \equiv \alpha L: v := E; L': \beta \quad \wedge \quad M \in \text{dom}(E[E]) \\ & \quad \vee \quad Ps \equiv \alpha L: v := ?; L': \beta \\ & \quad \vee \quad Ps \equiv \alpha L: \text{else } Ps' \text{ fi}; L': \beta \\ & \quad \vee \quad Ps \equiv \alpha L: \text{fi}; L': \beta \\ & \quad] \\ & \vee [\\ & \quad [(\\ & \quad \vee \quad Ps \equiv \alpha L: \text{if } B \text{ then } L': \beta \\ & \quad \vee \quad Ps \equiv \alpha L: \text{while } B \text{ do } L': \beta \\ & \quad \vee \quad Ps \equiv \alpha \text{while } B \text{ do } L': I_0; \dots; L_{m-1}: I_{m-1}; L: \text{od}; \beta \\ & \quad) \\ & \quad \wedge \quad M \in \text{dom}(B[B]) \wedge B[B](M) \\ & \quad] \\ & \vee [\\ & \quad [(\\ & \quad \vee \quad Ps \equiv \alpha L: \text{if } B \text{ then } Ps' \text{ else } L': \beta \\ & \quad \vee \quad Ps \equiv \alpha L: \text{while } B \text{ do } Ps' \text{ od}; L': \beta \\ & \quad \vee \quad Ps \equiv \alpha \text{while } B \text{ do } L_0: I_0; \dots; L_{m-1}: I_{m-1}; L: \text{od}; L': \beta \\ & \quad) \\ & \quad \wedge \quad M \in \text{dom}(B[B]) \wedge \neg B[B](M) \\ & \quad] \\ & \quad] \\ &] \end{aligned}$$

Par exemple, si le contrôle est au point L avant d'exécuter une boucle while ou après avoir exécuté son corps alors le contrôle passe au point L' qui désigne la première commande de son corps si le test est bien défini et vrai, le contrôle sort de la boucle si le test est bien défini et faux et le programme s'arrête en L (qui n'a pas de successeur) si le test n'est pas défini (ce qui produit une erreur à l'exécution).

2.8.1.2.5 Traces

La sémantique d'un programme séquentiel $P_s \in \mathcal{P}_s$ est :
 $\langle S[P_s], A[P_s], \Sigma \langle S[P_s], A[P_s], t[P_s], e[P_s] \rangle \rangle$

2.8.1.3 Exemple

Le programme séquentiel suivant calcule 2^m quand $m \geq 0$:

```

0:   P := 1;
1:   while N > 0 do
2:       P := 2 x P;
3:       N := N - 1;
4:   od;
5:

```

et sera utilisé pour illustrer quelques méthodes de preuve.

2.8.2 PROGRAMMES PARALLELES ASYNCHRONES

2.8.2.1 Syntaxe

Un programme parallèle asynchrone P_{pa} est composé d'un prélude séquentiel P_s , suivi de processus séquentiels $\llbracket P_{ra_0} \rrbracket \dots \llbracket P_{ra_{m-1}} \rrbracket$ partageant des variables globales communes et se déroulant en parallèle de manière asynchrone, suivis d'un postlude séquentiel $P_{s'}$:

$$P_{pa} \longrightarrow P_s \llbracket P_{ra_0} \rrbracket \dots \llbracket P_{ra_{m-1}} \rrbracket ; P_{s'} \quad (m \geq 1)$$

Les processus asynchrones $P_{ra_i}, i \in m$ sont des listes de commandes asynchrones étiquetées :

$$P_{ra} \longrightarrow L_0 : C_{a_0} ; \dots ; L_{m-1} : C_{a_{m-1}} ; L_m : \quad (m \geq 1)$$

Chaque commande asynchrone est une commande séquentielle (cf. 2.8.1.1) ou bien une liste de commandes séquentielles P_s exécutées de manière indivisible, ce que nous notons $\{P_s\}$:

$$C_a \longrightarrow \underline{\text{skip}} \mid \underline{v} := E \mid \underline{v} := ? \mid \underline{\text{if } B \text{ then } P_{ra_1} \text{ else } P_{ra_2} \text{ fi}} \mid \\ \underline{\text{while } B \text{ do } P_{ra} \text{ od}} \mid \{P_s\}$$

2.8.2.2 Sémantique

2.8.2.2.1 Etats

Un état d'un programme $P_{pa} = P_s \llbracket P_{ra_0} \rrbracket \dots \llbracket P_{ra_{m-1}} \rrbracket ; P_{s'}$ est une paire $\langle \text{état de contrôle, état mémoire} \rangle$ où l'état mémoire affecte des valeurs dans $\mathbb{D}$ aux variables partagées V . L'état de contrôle est une étiquette quand le contrôle est dans le prélude P_s ou le postlude $P_{s'}$ ou un tuple $\langle L_0, \dots, L_{m-1} \rangle$ de points de contrôle qui décrivent respectivement l'état de

contrôle des processus $Pra_0, \dots, Pra_{m-1}$ qui se déroulent en parallèle.
Formellement nous définissons :

$$S[Ppa] = (\{L \in \mathcal{L} : Ps \equiv \alpha L : \beta \vee Ps' \equiv \alpha L : \beta\} \cup \prod_{i \in m} \{L \in \mathcal{L} : Pra_i \equiv \alpha L : \beta\}) \times \mathcal{M}$$

2.8.2.2.2 Actions

Les actions du programme $Ps[Pra_0 \parallel \dots \parallel Pra_{m-1}] ; Ps'$ sont $\underline{p}_m, 0, \dots, m-1, \underline{p}'$ correspondant respectivement à un pas dans le prélude, le processus Pra_i et le postlude :

$$A[Ppa] = \{\underline{p}, \underline{p}'\} \cup m$$

2.8.2.2.3 Etats initiaux

L'exécution du programme commence au point d'entrée du prélude dans un état mémoire quelconque :

$$E[Ppa] \in (S[Ppa] \rightarrow \{\underline{t}, \underline{ff}\})$$

$$E[Ppa](\lambda) = [\exists L \in \mathcal{L}, M \in \mathcal{M}. \lambda = \langle L, M \rangle \wedge Ppa \equiv L : \alpha]$$

2.8.2.2.4 Relation de transition

La relation de transition

$$t[Ppa] \in (A[Ppa] \rightarrow ((S[Ppa] \times S[Ppa]) \rightarrow \{\underline{t}, \underline{ff}\}))$$

est définie par cas :

- L'exécution du prélude et du postlude est purement séquentielle :

$$t \llbracket Ppa \rrbracket_{\#} (\langle L, M \rangle, \langle L', M' \rangle) = [Ppa \equiv Ps \llbracket Pra_0 \parallel \dots \parallel Pra_{m-1} \rrbracket; Ps' \wedge t \llbracket Ps \rrbracket (\langle L, M \rangle, \langle L', M' \rangle)]$$

$$t \llbracket Ppa \rrbracket_{\#}' (\langle L, M \rangle, \langle L', M' \rangle) = [Ppa \equiv Ps \llbracket Pra_0 \parallel \dots \parallel Pra_{m-1} \rrbracket; Ps' \wedge t \llbracket Ps \rrbracket' (\langle L, M \rangle, \langle L', M' \rangle)]$$

- Les exécutions des processus parallèles commencent simultanément :

$$t \llbracket Ppa \rrbracket_{\#} (\langle L, M \rangle, \langle \langle L_0, \dots, L_{m-1} \rangle, M' \rangle) = [Ppa \equiv \alpha L: \llbracket L_0: \alpha_0 \parallel \dots \parallel L_{m-1}: \alpha_{m-1} \rrbracket; Ps' \wedge M' = M]$$

- Nous rendons compte de l'exécution parallèle des processus par un mélange nondéterministe d'exécutions d'actions atomiques. Chaque pas du programme consiste donc à exécuter de manière indivisible une action atomique d'un processus Pra_i tandis que les autres processus Pra_j , $j \in (m \setminus i)$ n'évoluent pas. L'exécution d'une action atomique indivisible correspond à l'évaluation d'une affectation ou d'un test d'une commande séquentielle (cf. 2.7.1.2.4) ou encore à l'exécution d'une liste de commandes séquentielles $\{Ps\}$ en exclusion mutuelle. Formellement :

$$t \llbracket Ppa \rrbracket_i (\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle \langle L'_0, \dots, L'_{m-1} \rangle, M' \rangle) =$$

$$[Ppa \equiv Ps \llbracket Pra_0 \parallel \dots \parallel Pra_{m-1} \rrbracket; Ps' \wedge \forall j \in (m \setminus i). L'_j = L_j \wedge$$

$$(t \llbracket Pra_i \rrbracket (\langle L_i, M \rangle, \langle L'_i, M' \rangle) \vee (Pra_i \equiv \alpha L_i: \{Ps\}; L'_i \beta \wedge Ps' \equiv L: \alpha L: \wedge t \llbracket Ps \rrbracket^* (\langle L, M \rangle, \langle L', M' \rangle)))]$$

- L'exécution de la commande parallèle se termine quand tous les processus ont terminé leur exécution :

$$t \llbracket Ppa \rrbracket_{\#} (\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle L, M' \rangle) = [Ppa \equiv Ps \llbracket \alpha_0 L_0: \parallel \dots \parallel \alpha_{m-1} L_{m-1}: \rrbracket; L: \beta \wedge M' = M]$$

2.8.2.2.5 Traces

Nous définissons

$$\Sigma \llbracket Ppa \rrbracket = \Sigma \langle s \llbracket Ppa \rrbracket, A \llbracket Ppa \rrbracket, \varepsilon \llbracket Ppa \rrbracket, t \llbracket Ppa \rrbracket \rangle$$

ce qui correspond à une exécution parallèle des processus sans aucune hypothèse d'équité. Par conséquent, l'exécution du programme

$B := \text{true}; \parallel B := \text{false} \parallel \text{while } B \text{ do skip od} \parallel$

peut ne jamais se terminer si à chaque pas de l'exécution le second processus est activé tandis que le premier processus est toujours en attente.

2.8.2.3 Exemple

Le programme parallèle asynchrone suivant calcule 2^m pour $m \geq 0$:

```

0:  [
    11:  P1 := 1;
    12:  while N > 1 do
    13:    & 131:  N := N - 1;
           132:  P1 := 2 * P1;
           133:  &;
    14:  od;
    15:
    ||
    21:  P2 := 1;
    22:  while N > 1 do
    23:    & 231:  N := N - 1;
           232:  P2 := 2 * P2;
           233:  &;
    24:  od;
    25:
    I;
1:  if N = 0 then P := P1 * P2 else P := 2 * P1 * P2 fi;
2:

```

(Remarquons que seule la commande d'affectation $N := N-1$ doit être atomique. Aucune condition d'équité n'est nécessaire sur l'exécution des processus car le calcul peut être entièrement fait par l'un des processus).

2.8.3 PROGRAMMES PARALLELES COMMUNICANTS

2.8.3.1 Syntaxe

Pour décrire des programmes parallèles distribués où les processus se synchronisent et communiquent par des commandes d'envoi et de réception de messages sur rendez-vous, nous utilisons une variante de CSP (Hoare[78]) où les commandes de communication utilisent des canaux au lieu du nom des autres processus. Par simplicité également nous n'utiliserons pas de variables locales aux processus mais des variables globales (qui ne peuvent être ni consultées ni modifiées par les autres processus) :

$$Ppc \longrightarrow P_s [P_{c_0} \parallel \dots \parallel P_{c_{m-1}}] ; P_s' \quad (m \geq 1)$$

Un processus est une liste séquentielle de commandes :

$$Prc \longrightarrow L_0 : Cc_0 ; \dots ; L_{m-1} : Cc_{m-1} ; L_m : \quad (m \geq 1)$$

chaque commande étant une commande asynchrone (cf. 2.8.2.1) ou un envoi de message sur rendez-vous, une réception de message sur rendez-vous ou encore une commande alternative qui permet de sélectionner une communication parmi plusieurs alternatives. Soit $\mathcal{C}_h$ un ensemble de canaux, nous avons :

$$Cc \longrightarrow \text{skip} \mid \mathcal{V} := \mathcal{C} \mid \mathcal{V} := ? \mid \text{if } \mathcal{B} \text{ then } Prc_1 \text{ else } Prc_2 \text{ fi} \\ \text{while } \mathcal{B} \text{ do } Prc \text{ od} \mid \{ P_s \} \mid \mathcal{C}_h ! \mathcal{E} \mid \mathcal{C}_h ? \mathcal{V} \mid \\ \text{se } \mathcal{A}lt_0 \text{ or } \dots \text{ or } \mathcal{A}lt_n \text{ es}$$

$$\mathcal{A}lt \longrightarrow \mathcal{B} ; \mathcal{C}_h ! \mathcal{E} \text{ then } Prc \mid \mathcal{B} ; \mathcal{C}_h ? \mathcal{V} \text{ then } Prc$$

2.8.3.2 Sémantique

Hoare n'ayant fait aucune hypothèse d'équité sur l'exécution des programmes CSP, la sémantique des programmes parallèles communicants est similaire à celle des programmes asynchrones sauf en ce qui concerne les commandes de communication. Dans la suite du paragraphe, nous avons :

$$P_{pc} \equiv P_s \llbracket P_{rc_0} \parallel \dots \parallel P_{rc_{m-1}} \rrbracket; P_{s'}$$

2.8.3.2.1 Etats

Comme en 2.8.2.2.1, nous définissons :

$$S \llbracket P_{pc} \rrbracket = (\{L \in \mathcal{L} : P_s \equiv \alpha L : \beta \vee P_{s'} \equiv \alpha L : \beta\} \cup \prod_{i \in m} \{L \in \mathcal{L} : P_{rc_i} \equiv \alpha L : \beta\}) \times \mathcal{M}$$

2.8.3.2.2 Actions

Les actions du programme $P_s \llbracket P_{rc_0} \parallel \dots \parallel P_{rc_{m-1}} \rrbracket; P_{s'}$ sont $\#_s, 0, \dots, m-1, \#_{s'}$ correspondant respectivement à un pas dans le prélude, le processus P_{rc_i} et le postlude comme en 2.8.2.2.2, plus l'ensemble des canaux intervenant dans le programme

$$A \llbracket P_{pc} \rrbracket = \{\#_s, \#_{s'}\} \cup m \cup \{\underline{ch} \in \mathcal{Ch} : P_{pc} \equiv \alpha \underline{ch} : \beta\}$$

2.8.3.2.3 Etats initiaux

Comme en 2.8.2.2.3, nous définissons :

$$E \llbracket P_{pc} \rrbracket \in (S \llbracket P_{pc} \rrbracket \rightarrow \{\#, \#_{s'}\})$$

$$E \llbracket P_{pc} \rrbracket (\lambda) = [\exists L \in \mathcal{L}, M \in \mathcal{M}. \lambda = \langle L, M \rangle \wedge P_{pc} \equiv L : \alpha]$$

2.8.3.2.4 Relation de transition

L'exécution d'un pas d'un programme parallèle communicant est similaire à celle d'un programme asynchrone sauf en ce qui concerne les commandes de communication. Par rapport à 2.8.2.2.4, nous rajoutons donc à la relation de transition un cas qui correspond à l'exécution d'une communication :

$$\begin{aligned}
 \vdash [Ppc]_{ch} (\langle L_0, \dots, L_{m-1} \rangle, M, \langle L'_0, \dots, L'_{m-1} \rangle, M') = \\
 & [Ppc \equiv Ps [Prc_0 \parallel \dots \parallel Prc_{m-1}]; Ps' \wedge \\
 & \exists i, j \in m. (Prc_i \equiv \alpha L_i : ch!E; L'_i : \beta \\
 & \vee (Prc_i \equiv \alpha L_i : \underline{ae} Alt_0 \underline{or} \dots \underline{or} Alt_{\ell-1} \underline{ea} \wedge \exists \ell \in \ell. \\
 & \quad Alt_\ell \equiv B_1; ch!E \text{ then } L'_i : \eta \wedge M \in \text{dom}(B[B_1, I] \wedge B[B_1, I](M))) \\
 & \wedge M \in \text{dom}(E[E]) \\
 & \wedge (Prc_j \equiv \gamma L_j : ch?v; L'_j : \delta \\
 & \vee (Prc_j \equiv \gamma L_j : \underline{ae} Alt'_0 \underline{or} \dots \underline{or} Alt'_p \underline{ea} \wedge \exists q \in p. \\
 & \quad Alt'_q \equiv B_2; ch?v \text{ then } L'_j : \xi \wedge M \in \text{dom}(B[B_2, I] \wedge B[B_2, I](M))) \\
 & \wedge \forall k \in (m \setminus \{i, j\}). L'_k = L_k \\
 & \wedge M'(V) = E[E](M) \wedge \forall W \in (V \cup V). (M'(W) = M(W))]
 \end{aligned}$$

Nous rajoutons également le cas correspondant à la sortie d'une commande alternative :

$$\begin{aligned}
 \vdash [Ppc]_i (\langle L_0, \dots, L_{m-1} \rangle, M, \langle L'_0, \dots, L'_{m-1} \rangle, M') = \\
 & [Ppc \equiv Ps [Prc_0 \parallel \dots \parallel Prc_{m-1}]; Ps' \wedge \forall j \in (m \setminus i). L'_j = L_j \wedge \\
 & (\vdash [Prc_i] (\langle L_i, M \rangle, \langle L'_i, M' \rangle) \\
 & \vee (Prc_i \equiv \alpha L_i : \nmid Ps'' \nmid; L'_i : \beta \wedge Ps'' \equiv L : \gamma; L' : \wedge \vdash [Ps'']^* (\langle L, M \rangle, \langle L', M' \rangle)) \\
 & \vee (Prc_i \equiv \alpha \underline{ae} Alt_0 \underline{or} \dots \underline{or} \beta L_i : \underline{or} \dots \underline{or} Alt_{\ell-1} \underline{ea}; L'_i : \gamma \wedge M' = M))]
 \end{aligned}$$

2.8.3.2.5 Traces

La sémantique d'un programme parallèle communiquant P_{pc} est :

$$\langle S[P_{pc}], A[P_{pc}], \Sigma \langle S[P_{pc}], A[P_{pc}], E[P_{pc}], E[P_{pc}] \rangle \rangle$$

2.8.3.3 Exemples

Exemple 2.8.3.3-1

Le programme parallèle suivant réalise une section critique.
Remarquons que le troisième processus joue le rôle de contrôleur.
(Ayant défini (avec les notations Pascal) le type $\text{signal} = (\text{any})$ et la variable $\text{Any} : \text{signal}$, nous écrivons $P! \text{Any}$ (respectivement $P? \text{Any}$) pour la transmission (respectivement réception) d'un signal).


```

0: 11:  while true do
    12:      P! any ;
    13:      V! any ;
    14:  od ;
    15:
11: 21:  while true do
    22:      P! any ;
    23:      V! any ;
    24:  od ;
    25:
12: 31:  while true do
    32:      P! Any ;
    33:      V! Any ;
    34:  od ;
    35:
13: II;
14:

```

Remarquons que l'entrée dans une section critique donnée n'est pas fatale.

Par contre, elle l'est dans la version suivante où l'hypothèse d'équité faible est nécessaire. (Nous codons la valeur du sémaphore et le contenu de la file d'attente comme suit :

$s=0$ le sémaphore est ouvert

$s=i$ le sémaphore est fermé. Il a été franchi par le processus i , $i=1,2$ et ensuite le processus $\tilde{i}$ n'a pas demandé à le franchir (où nous notons $\tilde{i}=2$ et $\tilde{\tilde{i}}=1$)

$s=i\tilde{i}$ le sémaphore est fermé. Il a été franchi par le processus i et ensuite le processus $\tilde{i}$ a demandé à le franchir).

```

0:  S := 0;
1:  [ 11:  while true do
      12:      if S=0 then S:=1 else S:=12;  {demande d'entrée en section critique}
      13:      P1 ! Any;                    {autorisation d'entrée en section critique}
      14:      V ! Any;                    {sortie de la section critique}
      15:  od;
      16:  ||
      21:  while true do
      22:      if S=0 then S:=2 else S:=21;
      23:      P2 ! Any;
      24:      V ! Any;
      25:  od;
      26:  ||
      31:  while true do
      32:      se (S=1); P1?Any then skip;
      33:      or (S=21); V?Any then S:=2; P2?Any;
      34:      or (S=2); P2?Any then skip;
      35:      or (S=12); V?Any then S:=1; P1?Any;
      36:      or ((S=1) v (S=2)); V?Any then S:=0;
      37:      es;
      38:  od;
      39:  ];
2:

```

Exemple 2.8.3.3.2

Le programme suivant (Hoare[73]) réalise la transmission de messages d'un producteur vers un consommateur via un tampon pour régulariser les flux; les variables sont du type suivant :

```

type signal = (any);
N : 1..maxint;
output, Input : array[0..N-1] of type-messages;
Buffer        : array[0..9] of type-messages;
I, In, Out, J : integer;
Pe, Ft, Te    : boolean;
PTm, TCm      : channel type-messages;
PTs, TCS      : channel signal;
Any           : signal;

```

```

0:  [[
    {Producteur:} 11:
                        I := 0;
    12:
                        while I <> N do
    13:
                            PTm ! output[I];
    14:
                            I := I+1;
    15:
                        od;
    16:
                            PTs ! any;
    17:

```

```

||
{Tampon} 201:  Im:=0;  Out:=0;  Pe:=false;  Ft:=false;
202:
203:  while not Ft do
204:      ae  $\neg Pe \wedge Im < Out + 10$ ;  PTm ? Buffer[Im mod 10] then
205:          Im := Im + 1;
206:      or  Out < Im;  TCm ! Buffer[Out mod 10] then
207:          Out := Out + 1;
208:      or   $\neg Pe$ ;  PTA ? Any then
209:          Pe := true;
210:      or  Pe  $\wedge$  (Im = Out);  TCs ! any then
211:          Ft := true;
212:      es;
213:  od;

```

```

||
{Consommateur} 31:  J:=0;  Te:=false;
32:
33:  while  $\neg Te$  do
34:      ae true;  TCm ? Input[J] then
35:          J := J + 1;
36:      or  true;  TCs ? Any then
37:          Te := true;
38:      es;
39:  od;

```

1: II;

2.8.4 PROGRAMMES PARALLELES FAIBLEMENT EQUITABLES

2.8.4.1 Syntaxe

Pour simplifier nous adoptons la même syntaxe que pour les programmes asynchrones :

$$P_{pw} \rightarrow P_{pa}$$

2.8.4.2 Sémantique

La sémantique d'un programme parallèle faiblement équitable peut être définie de manière équivalente (cf. 2.4.3.2)

- comme la restriction $\mathcal{W}_{fair}(\langle S[P_{pw}], A[P_{pw}], \Sigma \langle S[P_{pw}], A[P_{pw}], t[P_{pw}], e[P_{pw}] \rangle \rangle$ aux traces faiblement équitables de la sémantique des programmes parallèles

- par concordance avec la sémantique close $\langle S[P_{pw}]^{\#}, A[P_{pw}]^{\#}, \Sigma \langle S[P_{pw}]^{\#}, A[P_{pw}]^{\#}, t[P_{pw}]^{\#}, e[P_{pw}]^{\#} \rangle \rangle$ à la fonction $f_s^{\#}$ entre états près.

2.8.4.3 Exemple

Soit $F \in (\mathbb{Z} \rightarrow \mathbb{Z})$ un opérateur sur les entiers. S'il possède un point fixe alors l'exécution du programme parallèle faiblement équitable suivant se termine avec une valeur finale p de P telle que $F(p) = p$.

Le premier processus cherche un point fixe parmi les entiers strictement négatifs et le deuxième processus cherche parmi les entiers positifs ou nuls. L'hypothèse d'équité faible est donc nécessaire pour garantir la terminaison :

```

0:   $S_1 := \text{true} ; S_2 := \text{true}$ 
1:  [
10:     $L := -1;$ 
11:    while  $S_1 \wedge S_2$  do
12:      if  $F(L) = L$  then
13:         $S_1 := \text{false};$ 
14:      else
15:         $L := L - 1;$ 
16:      fi;
17:    od;
18:  ]
19:  ||
20:     $H := 0;$ 
21:    while  $S_1 \wedge S_2$  do
22:      if  $F(H) = H$  then
23:         $S_2 := \text{false};$ 
24:      else
25:         $H := H + 1;$ 
26:      fi;
27:    od;
28:  ]
29:  if  $\neg S_1$  then
30:     $P := L;$ 
31:  else if  $\neg S_2$  then
32:     $P := H;$ 
33:  fi;
34:
35:

```

2.8.5 PROGRAMMES PARALLELES SYNCHRONES

De nombreuses solutions ont été proposées pour synchroniser des processus parallèles partageant des données communes. En général, elles peuvent s'implanter à l'aide de la notion primitive de sémaphores (Dijkstra [68]). C'est la solution que nous avons retenue comme exemple en raison de sa généralité et de sa relative simplicité.

2.8.5.1 Syntaxe

La syntaxe des programmes parallèles synchrones

$$Pps \longrightarrow Ps [Prs_0 \parallel \dots \parallel Prs_{m-1}]; Ps' \quad (m \geq 1)$$

ne diffère de la syntaxe des programmes parallèles asynchrones que par le fait que les processus synchrones

$$Prs \longrightarrow L_0 : Es_0 ; \dots ; L_{m-1} : Es_{m-1} ; L_m : \quad (m \geq 1)$$

sont des listes de commandes synchrones étiquetées. Ces commandes synchrones peuvent être une commande séquentielle ou bien une opération $\underline{p}$ ou $\underline{v}$ sur un sémaphore. Soit $\mathcal{S} \subseteq \mathcal{V}$ un ensemble non vide de variables sémaphores, nous avons

$$Es \longrightarrow \underline{\text{skip}} \mid \underline{v} := E \mid \underline{v} := ? \mid \underline{\text{if } B \text{ then } Prs_1 \text{ else } Prs_2 \text{ fi}} \mid \\ \underline{\text{while } B \text{ do } Prs \text{ od}} \mid \underline{p}(\mathcal{S}) \mid \underline{v}(\mathcal{S})$$

Comme pour les autres variables, les déclarations de sémaphores (Sem $\mathcal{S}$ init $\mathcal{A}_0$) sont laissées implicites. Nous supposons cependant qu'elles associent à tout sémaphore $Se \in \mathcal{S}$, une valeur initiale entière Is_{em}(Se) (notée $\mathcal{A}_0$ ci-dessus).

Les conditions de contexte que les sémaphores n'apparaissent pas ailleurs que dans les commandes p et v et que les autres variables ne peuvent pas apparaître dans ces instructions sont habituelles :

$$\begin{aligned} & ([(Pps \equiv \alpha E \beta \wedge E \equiv \alpha' v \beta') \vee (Pps \equiv \alpha v := \beta) \vee (Pps \equiv \alpha B \beta \wedge B \equiv \alpha' v \beta')] \Rightarrow v \notin Se) \\ \wedge & [(Pps \equiv \alpha p(v) \beta) \vee (Pps \equiv \alpha v(v) \beta)] \Rightarrow v \in Se \end{aligned}$$

2.8.5.2 Sémantique

De très nombreuses définitions opérationnelles de la sémantique des sémaphores ont été proposées. Des différences souvent légères entre ces définitions entraînent parfois de subtiles différences de comportement pour les programmes qui les utilisent. La définition que nous proposons se veut fidèle à la définition originale de Dijkstra [68]. D'un point de vue pratique, on pourrait objecter que l'utilisation de stratégies de gestion des processus en attente plus souples que les files d'attente sont utiles, mais Hebermann [72] montre que les sémaphores de Dijkstra avec files d'attente strictes permettent d'implanter les autres stratégies.

Dans la suite du paragraphe, nous posons $Pps \equiv Ps \parallel Prs_0 \parallel \dots \parallel Prs_{m-1} \parallel ; Ps'$.

2.8.5.2.1 Etats

La différence avec les états des programmes parallèles asynchrones (cf. 2.8.2.2.1) est qu'à tout sémaphore $Se \in \mathcal{S} \subseteq \mathcal{V}$ est associé une valeur entière et une file d'attente de processus. Pour simplifier, nous supposons $\mathbb{Z} \subseteq \mathbb{D}$ et représentons la file d'attente par une séquence (eventuellement vide) d'éléments de m :

$$C[Pps] = \{L \in \mathcal{L} : Ps \equiv \alpha L : \beta \vee Ps' \equiv \alpha L : \beta\} \cup \bigcup_{i \in m} \{L \in \mathcal{L} : Pr_{\alpha_i} \equiv \alpha L : \beta\}$$

$$S[Pps] = C[Pps] \times \mathcal{M} \times (\mathcal{V} \rightarrow \mathcal{M}^{\omega})$$

2.8.5.2.2 Actions

Les actions du programme parallèle synchrone $Pps \equiv Ps [Pr_{\alpha_0} \parallel \dots \parallel Pr_{\alpha_{m-1}}] ; Ps'$ sont :

- $\#$ correspondant à l'exécution d'un pas du prélude Ps
- $\#'$ correspondant à l'exécution d'un pas du postlude Ps'
- i correspondant à l'exécution d'un pas du processus Pr_{α_i} , $i \in m$ (autre qu'une opération sur un sémaphore)

La commande $\#(Se)$ donne lieu à deux actions possibles, à savoir la mise en attente (généralement suivie d'un réveil par un autre processus exécutant un $\#(Se)$) ou le passage du sémaphore :

$\#(Se, i)$ correspond à l'exécution de la commande $\#(Se)$ par le processus Pr_{α_i} qui provoque la mise en attente de ce processus devant le sémaphore Se .

$\#(Se, i)$ correspond à l'exécution de la commande $\#(Se)$ par le processus Pr_{α_i} et au passage de ce sémaphore par ce processus

La commande $\#(Se)$ donne lieu à deux actions possibles selon qu'il y ait ou non un processus en attente qu'il faut réveiller :

$\#(Se, i)$ le processus Pr_{α_i} exécute une commande $\#(Se)$ qui libère le sémaphore alors qu'aucun autre processus n'était en attente sur ce sémaphore

$\#(Se, i, j)$ le processus Pr_{α_i} exécute une commande $\#(Se)$ qui libère le sémaphore et permet au processus Pr_{α_j} qui était en attente de le passer.

Nous avons donc :

$$A[Pps] = \{\#, \#'\} \cup m \cup \{\#(Se, i), \#(Se, i), \#(Se, i), \#(Se, i, j) : Se \in \mathcal{S} \wedge i, j \in m \wedge i \neq j\}$$

2.8.5.2.3 Etats initiaux

Dans un état initial le contrôle est au début du programme. Les sémaphores $se \in \mathcal{S}$ ont la valeur initiale $\underline{Isem}(se)$ (définie par une déclaration, ...) et la file d'attente associée est vide :

$$\varepsilon[\![Pps]\!](\Delta) = [\![\exists L \in \mathcal{L}, M \in \mathcal{M}_0, Q \in (\mathcal{S} \rightarrow \mathbb{N}^{<\omega}) . (A = \langle L, M, Q \rangle \wedge Pps \equiv L : \alpha \wedge \forall se \in \mathcal{S} . (M(se) = \underline{Isem}(se) \wedge Q(se) = \langle \rangle))]\!]$$

2.8.5.2.4 Relation de transition

Pour les actions $\sharp, \sharp'$ et $q, \dots, m-1$ la relation de transition est similaire à celle des programmes parallèles asynchrones (cf. 2.8.2.2.4, la valeur et la file d'attente des sémaphores n'étant pas modifiées par ces actions). Il convient d'ajouter quatre cas correspondant à l'exécution d'une commande sur un sémaphore :

$$\begin{aligned} t[\![Pps]\!]_{\omega(se,i)}(\langle\langle L_0, \dots, L_{n-1} \rangle, M, Q \rangle, \langle\langle L'_0, \dots, L'_{n-1} \rangle, M', Q' \rangle) = \\ [Pps \equiv Ps[\![P_{\tau 0} \parallel \dots \parallel P_{\tau n-1}]\!]; Ps' \wedge P_{\tau i} \equiv \alpha L_i : \sharp(se); \beta \wedge M(se) \leq 0 \wedge \\ \forall l \in |Q(se)| . Q(se)_l \neq i \wedge \forall k \in m . L'_k = L_k \wedge M' = M \wedge Q'(se) = (Q(se) \hat{\langle i \rangle}) \wedge \\ \forall sm \in (\mathcal{S} \setminus se) . Q'(sm) = Q(sm).] \end{aligned}$$

(Le processus $P_{\tau i}$ exécute $\sharp(se)$ alors que le sémaphore a une valeur négative ou nulle et que le processus $P_{\tau i}$ n'est pas dans la file d'attente. Le processus $P_{\tau i}$ entre dans la file d'attente et le contrôle reste aux mêmes points du programme).

$$\begin{aligned} t[\![Pps]\!]_{\sharp(se,i)}(\langle\langle L_0, \dots, L_{n-1} \rangle, M, Q \rangle, \langle\langle L'_0, \dots, L'_{n-1} \rangle, M', Q' \rangle) = \\ [Pps \equiv Ps[\![P_{\tau 0} \parallel \dots \parallel P_{\tau n-1}]\!]; Ps' \wedge P_{\tau i} \equiv \alpha L_i : \sharp(se); L'_i : \beta \wedge M(se) > 0 \wedge \\ \forall k \in (m \setminus i) . L'_k = L_k \wedge M'(se) = M(se) - 1 \wedge \forall v \in (\mathcal{S} \setminus se) . M'(v) = M(v) \wedge Q' = Q] \end{aligned}$$

(Le processus Prs_i exécute $\varphi(Se)$ alors que le sémaphore a une valeur strictement positive. Le contrôle du processus passe donc la commande $\varphi(Se)$ et la valeur du sémaphore diminue de 1).

$$t[Pps]_{\varphi(Se,i)}(\langle\langle L_0, \dots, L_{m-1} \rangle, M, Q \rangle, \langle\langle L'_0, \dots, L'_{m-1} \rangle, M', Q' \rangle) =$$

$$[Pps \equiv Ps[Prs_0 \parallel \dots \parallel Prs_{m-1}]; Ps' \wedge Prs_i \equiv \alpha L_i : \varphi(Se); L'_i : \beta \wedge Q(Se) = \langle \rangle \wedge \\ \forall k \in (m \setminus i). L'_k = L_k \wedge M'(Se) = M(Se) + 1 \wedge \forall v \in (\mathcal{V} \setminus Se). M'(v) = M(v) \wedge Q' = Q]$$

(Le processus Prs_i exécute la commande $\varphi(Se)$ alors qu'aucun autre processus n'est en attente sur ce sémaphore. Le contrôle de Prs_i franchit la commande et la valeur du sémaphore est augmentée de 1).

$$t[Pps]_{\varphi\varphi(Se,i,j)}(\langle\langle L_0, \dots, L_{m-1} \rangle, M, Q \rangle, \langle\langle L'_0, \dots, L'_{m-1} \rangle, M', Q' \rangle) =$$

$$[Pps \equiv Ps[Prs_0 \parallel \dots \parallel Prs_{m-1}]; Ps' \wedge Prs_i \equiv \alpha L_i : \varphi(Se); L'_i : \beta \wedge Q(Se) = \langle j \rangle \wedge Q'(Se) \wedge \\ Prs_j \equiv \alpha' L'_j : \varphi\varphi(Se); L'_j : \beta' \wedge \forall k \in (m \setminus \{i, j\}). L'_k = L_k \wedge M' = M \wedge \forall Sm \in (\mathcal{S} \setminus \{Se\}). Q'(Sm) = Q(Sm)]$$

(Le processus Prs_j exécute la commande $\varphi\varphi(Se)$ alors que le processus Prs_j est en tête de la file d'attente sur ce sémaphore en attente d'exécution de $\varphi(Se)$. De manière atomique le processus Prs_j quitte la file d'attente et les processus Prs_i et Prs_j franchissent simultanément les commandes respectives $\varphi(Se)$ et $\varphi\varphi(Se)$).

2.8.5.2.5 Traces

Dans une implantation de programmes parallèles avec sémaphores sur monoprocesseur, le contrôleur ("scheduler") synchronisant les opérations sur les sémaphores a la main régulièrement au moyen d'interruptions d'horloge, quand aucune opération sur les sémaphores n'est exécutée.

De manière plus abstraite, cette hypothèse d'équité faible se traduit par le fait que si le contrôle est devant une commande $\sharp(se)$ (auquel cas l'une des deux actions $\underline{w}(se,i)$ ou $\sharp(se,i)$ est toujours exécutable) alors fatalement le processus correspondant sera mis dans la file d'attente (par exécution de l'action $\underline{w}(se,i)$) ou franchira la commande $\sharp(se)$ (par exécution de l'action $\sharp(se,i)$). De même, si le contrôle d'un processus est devant une commande $\underline{v}(se)$ (auquel cas l'une des actions $\underline{v}(se,i)$ ou $\underline{vp}(se,i,j), i \neq j$ est toujours exécutable) alors fatalement le processus franchira la commande $\underline{v}(se)$ (par exécution de l'une des actions $\underline{v}(se,i)$ ou $\underline{vp}(se,i,j)$).

Plus formellement, nous définissons l'ensemble réduit d'actions

$$Ar[Pps] = \{\sharp, \sharp'\} \cup m \cup \{\sharp(se,i), \underline{v}(se,i) : se \in \mathcal{S}_e \wedge i \in m\}$$

et la correspondance $f_a \in (A[Pps] \rightarrow Ar[Pps])$ par cas :

$$\begin{aligned} f_a(\underline{w}(se,i)) &= f_a(\sharp(se,i)) = \sharp(se,i) & \text{si } se \in \mathcal{S}_e, i \in m \\ f_a(\underline{v}(se,i)) &= f_a(\underline{vp}(se,i,j)) = \underline{v}(se,i) & \text{si } se \in \mathcal{S}_e, i, j \in m \\ f_a(x) &= x & \text{si } x \text{ est } \sharp, \sharp' \text{ ou } i \in m \end{aligned}$$

Les traces p de $\Sigma[Pps]$ sont les traces engendrées par le système de transition associé à Pps (et appartenant donc à $\Sigma\langle S[Pps], A[Pps], t[Pps], \varepsilon[Pps] \rangle$) dont, l'image $\alpha\langle f_a \rangle(p)$, après avoir renommé les actions par f_a , est faiblement équitable pour $Ar[Pps]$:

$$\Sigma[Pps] = \{p \in \Sigma\langle S[Pps], A[Pps], t[Pps], \varepsilon[Pps] \rangle :$$

$$\alpha\langle f_a \rangle(p) \in \bigcap_{\text{fin}} \langle Ar[Pps] \rangle (\alpha\langle f_a \rangle(\Sigma\langle S[Pps], A[Pps], t[Pps], \varepsilon[Pps] \rangle))\}$$

2.8.5.2.6 Propriétés de la sémantique des sémaphores

Pour nous convaincre que cette définition de la sémantique des sémaphores correspond bien à l'idée intuitive que nous en avons, nous démontrons des propriétés classiques d'invariance et d'équité des sémaphores. Ces propriétés sont fréquemment utilisées mais tenues pour vraies sans

démonstration ou en utilisant des arguments informels.

Nous commençons par démontrer des lemmes préliminaires :

Le premier lemme exprime que si la valeur initiale du sémaphore est positive ou nulle alors il est toujours vrai en cours d'exécution du programme que si la valeur du sémaphore est positive ou nulle alors la file est vide et que si la valeur du sémaphore est nulle alors la file contient (au plus) m processus deux à deux différents :

Lemme 2.8.5.2.6 v1

$\forall p \in \Sigma[\text{Pps}], j \in |p|, L \in C[\text{Pps}], M \in \mathcal{N}, \varphi \in (\mathcal{V}_e \rightarrow m^{\omega}), S_e \in \mathcal{V}_e.$

$[p_j = \langle L, M, \varphi \rangle \wedge \text{Isem}(S_e) \geq 0]$

$\Rightarrow [(|Q(S_e)| = 0 \wedge M(S_e) \geq 0) \vee (0 \leq |Q(S_e)| \leq m \wedge M(S_e) = 0 \wedge \forall k, k' \in |Q(S_e)|. (k \neq k' \Rightarrow Q(S_e)_k \neq Q(S_e)_{k'}))]$

Démonstration

Par induction sur $j \in |p|$. Si $j=0$ alors $\varepsilon[\text{Pps}](p_0)$, $p_0 = \langle L, M, \varphi \rangle$ et $\text{Isem}(S_e) \geq 0$ entraînent $Q(S_e) = \langle \rangle$ donc $|Q(S_e)| = 0$ et $M(S_e) = \text{Isem}(S_e) \geq 0$. Si la propriété est vraie pour $p_j = \langle L, M, \varphi \rangle$ et $j+1 \in |p|$, démontrons que par définition de $\varepsilon[\text{Pps}]_{p_j}$, la propriété reste vraie pour $p_{j+1} = \langle L', M', \varphi' \rangle$. Si l'action p_j est $\#$, $\#'$ ou iem c'est évident car $M'(S_e) = M(S_e)$ et $\varphi'(S_e) = \varphi(S_e)$. Si $p_j = w(S_e, i)$ alors nous avons $M(S_e) \leq 0$ et $\forall e \in |Q(S_e)|. Q(S_e)_e \neq i$ et donc par hypothèse d'induction $0 \leq |Q(S_e)| < m$ et $M(S_e) = 0$ et les éléments de $Q(S_e)$ sont deux à deux différents. Comme $\varphi'(S_e) = Q(S_e)^{\wedge i}$ et $M'(S_e) = M(S_e)$, nous en déduisons que $1 \leq Q'(S_e) \leq m$, $M'(S_e) = 0$ et les éléments de $Q'(S_e)$ sont deux à deux différents. Si $p_j = \#(S_e, i)$ alors $M(S_e) > 0$ et donc par hypothèse d'induction $|Q(S_e)| = 0$. Nous avons également $M'(S_e) = M(S_e) - 1$ et $\varphi'(S_e) = Q(S_e)$ donc nous en déduisons $M'(S_e) \geq 0$ et $|Q'(S_e)| = 0$. Si $p_j = w(S_e, i)$ nous avons $|Q(S_e)| = 0$, $M'(S_e) = M(S_e) + 1$ et $\varphi'(S_e) = Q(S_e)$ et donc $|Q'(S_e)| = 0$ et $M'(S_e) > 0$. Si $p_j = w(S_e, i, k)$ alors $|Q(S_e)| > 0$ et donc $M(S_e) = 0$. Comme $M'(S_e) = M(S_e)$ et $0 \leq |Q'(S_e)| < |Q(S_e)|$, nous en déduisons $0 \leq |Q'(S_e)| < m$, $M'(S_e) = 0$ et les éléments de $Q'(S_e)$ sont

deux à deux différents puisque ceux de $q(se)$ le sont par hypothèse d'induction.

□

Le lemme suivant montre que si un processus est dans la file d'attente d'un sémaphore se alors son point de contrôle est devant une commande $p(se)$.

Lemme 2.8.5.2.6 v2

$\forall Pps \in \mathcal{P}ps, m \in \omega, p \in \Sigma[Pps], \langle L_0, \dots, L_{m-1} \rangle \in \mathcal{L}^m, M \in \mathcal{M}, Q \in (\mathcal{V}_e \rightarrow m^{\omega}), se \in \mathcal{V}_e.$

$m \in m, j \in |p|.$

$$\begin{aligned} & [Pps \equiv Ps[P_{r0} \parallel \dots \parallel P_{r_{m-1}}]; Ps' \wedge p_j = \langle \langle L_0, \dots, L_{m-1} \rangle, M, Q \rangle \wedge \exists i \in |Q(se)|. Q(se)_i = m] \\ \Rightarrow & [P_{r_m} \equiv \alpha L_m : p(se); \beta] \end{aligned}$$

Démonstration

Par induction sur $j \in |p|$. Le lemme est vrai pour $j=0$ car $\in[Pps](p_0)$ implique $Q(se)=0$. Si le lemme est vrai pour $p_j = \langle \langle L_0, \dots, L_{m-1} \rangle, M, Q \rangle$ et $j+1 \in |p|$ alors il reste vrai pour $p_{j+1} = \langle \langle L'_0, \dots, L'_{m-1} \rangle, M', Q' \rangle$.

- Si $\forall k \in |Q(se)|. Q(se)_k \neq m$ et $\exists k \in |Q'(se)|. Q'(se)_k = m$ alors par définition de $t[Pps]$, nous avons $p_j = \underline{w}(se, m)$ et $P_{r_m} \equiv \alpha L_m : p(se); \beta$

- Si $\exists k \in |Q(se)|. Q(se)_k = m$ alors par hypothèse d'induction, nous avons $P_{r_m} \equiv \alpha L_m : p(se); \beta$. Par définition de $t[Pps]$, p_j n'est pas p , p' , $\underline{w}(se, m)$, $p(se, m)$ (car nous aurions $M(se) > 0$ et d'après le lemme 2.8.5.2.6 v1, ceci impliquerait $Q(se) = \langle \rangle$, $\underline{v}(se, m)$ ou $\underline{w}(se, m, i')$ (car nous aurions $P_{r_m} \equiv \alpha' L_m : \underline{v}(se); \beta'$ contraire à l'hypothèse que L_m n'apparaît qu'une fois dans P_{r_m}). Si p_j est $\underline{v}(se, m, i)$, $p(se, m, i)$, $\underline{v}(se, m, i)$, $\underline{w}(se, m, i, i')$ avec $sm \in (\mathcal{V}_e \cup se)$, nous avons $i \neq m$ et $i' \neq m$ et donc $L'_m = L_m$ par définition de $t[Pps]$. Appliquant l'hypothèse d'induction, nous en déduisons que $P_{r_m} = \alpha L'_m : p(se); \beta$. Enfin si $p_j = \underline{w}(se, i, m)$, par définition de $t[Pps]$ nous avons $Q(se) = m Q'(se)$ et donc d'après 2.8.5.2.6 v1, $\forall k \in |Q'(se)|. Q'(se)_k \neq m$, ce qui implique le lemme de façon triviale.

□

Le lemme suivant exprime que le nombre d'exécutions d'une commande $p(se)$ conduisant à la mise en attente du processus qui l'exécute est égal au nombre d'exécutions d'une commande $v(se)$ conduisant au réveil d'un processus en attente plus le nombre de processus en attente dans la file d'attente du sémaphore se :

Etant donnée une sémantique $\langle S, A, \Sigma \rangle$, nous définissons $\sigma \in (2^A \rightarrow ((\Sigma \times \omega) \rightarrow \omega))$

$$\sigma_\alpha(p, j) = \sum_{k \in (j+1|p|)} (p_k \in \alpha \rightarrow 1|0) \quad (\text{avec } \Sigma \phi = 0)$$

de sorte que $\sigma_\alpha(p, j)$ est le nombre de fois qu'une action appartenant à α est exécutée entre p_0 et p_j .

Lemme 2.8.5.2.6 v3

$\forall p \in \Sigma[Pps], j \in |p|, L \in C[Pps], M \in \mathcal{M}, Q \in (\mathcal{P} \rightarrow \mathcal{M}^{\omega}), se \in \mathcal{P}.$

$$[p_j = \langle L, M, Q \rangle] \Rightarrow [\sigma\{\underline{w}(se, i) : i \in m\}(p, j) = \sigma\{\underline{w}(se, i, i') : i, i' \in m\}(p, j) + |Q(se)|]$$

Démonstration

Par induction sur $j \in |p|$. Si $j=0$, nous avons $e \in [Pps](p_j)$ et donc $|Q(se)|=0$ de sorte que $\sigma\{\underline{w}(se, i) : i \in m\}(p, j) = \sigma\{\underline{w}(se, i, i') : i, i' \in m\}(p, j) = |Q(se)|=0$. Si par hypothèse d'induction la propriété est vraie pour $p_j = \langle L, M, Q \rangle$, $j+1 \in |p|$ et $p_{j+1} = \langle L', M', Q' \rangle$ alors si p_j est p , p' , $i \in m$, $\underline{w}(sm, i)$, $p(sm, i)$, $v(sm, i)$, $\underline{w}(se, i, i')$ pour $sm \in (\mathcal{P} \cup se)$, $p(se, i)$ ou $v(se, i)$, nous avons $\sigma\{\underline{w}(se, i) : i \in m\}(p, j+1) = \sigma\{\underline{w}(se, i) : i \in m\}(p, j)$, $\sigma\{\underline{w}(se, i, i') : i, i' \in m\}(p, j+1) = \sigma\{\underline{w}(se, i, i') : i, i' \in m\}(p, j)$ et $Q'=Q$. Si $p_j = \underline{w}(se, k)$ alors $\sigma\{\underline{w}(se, i) : i \in m\}(p, j+1) = \sigma\{\underline{w}(se, i) : i \in m\}(p, j) + 1$, $\sigma\{\underline{w}(se, i, i') : i, i' \in m\}(p, j+1) = \sigma\{\underline{w}(se, i, i') : i, i' \in m\}(p, j)$ et $|Q'(se)| = |Q(se)| + 1$. Si $p_j = \underline{w}(se, k, k')$ alors $\sigma\{\underline{w}(se, i) : i \in m\}(p, j) = \sigma\{\underline{w}(se, i) : i \in m\}(p, j+1)$, $\sigma\{\underline{w}(se, i, i') : i, i' \in m\}(p, j+1) = \sigma\{\underline{w}(se, i, i') : i, i' \in m\}(p, j) + 1$ et $|Q'(se)| = |Q(se)| - 1$.

□

Le lemme suivant exprime que le nombre d'exécutions d'une commande $p(se)$ sans attente du processus qui l'exécute est égal au nombre d'exécutions de la commande $v(se)$ alors qu'aucun processus n'était en attente dans la file de se plus la valeur initiale moins la valeur courante du sémaphore se .

Lemme 2.8.5.2.6 ~ 4

$\forall p \in \Sigma[Pps], j \in |p|, L \in C[Pps], M \in \mathcal{M}, Q \in (\mathcal{V} \rightarrow \mathbb{N}^{\leq \omega}), se \in \mathcal{V}.$

$\Rightarrow [p_j = \langle L, M, Q \rangle]$

$[\sigma \{ p(se, i) : i \in m \} (p, j) = \sigma \{ v(se, i) : i \in m \} (p, j) + \underline{I_{sem}}(se) - M(se)]$

Démonstration

Par induction sur $j \in |p|$. Pour $j=0$, nous avons $\sigma \{ p(se, i) : i \in m \} (p, 0) = \sigma \{ v(se, i) : i \in m \} (p, 0) = 0$ et $\varepsilon[Pps](p_0)$ entraîne que $M(se) = \underline{I_{sem}}(se)$. Si la propriété est vraie pour $p_j = \langle L, M, Q \rangle$, $j+1 \in |p|$ et $p_{j+1} = \langle L', M', Q' \rangle$ alors si p_j est p , p' , $i \in m$, $\underline{w}(sm, i)$, $p(sm, i)$, $v(sm, i)$, $\underline{wp}(sm, i, i')$ avec $sm \neq se$, $\underline{w}(se, i)$ ou $\underline{wp}(se, i, i')$ alors $M' = M$ et $\sigma \{ p(se, i) : i \in m \} (p, j+1) = \sigma \{ p(se, i) : i \in m \} (p, j)$ et $\sigma \{ v(se, i) : i \in m \} (p, j+1) = \sigma \{ v(se, i) : i \in m \} (p, j)$. Si p_j est $p(se, i)$ alors $\sigma \{ p(se, i) : i \in m \} (p, j+1) = \sigma \{ p(se, i) : i \in m \} (p, j) + 1$, $\sigma \{ v(se, i) : i \in m \} (p, j+1) = \sigma \{ v(se, i) : i \in m \} (p, j)$ et $M'(se) = M(se) - 1$. Si p_j est $v(se, i)$ alors $\sigma \{ p(se, i) : i \in m \} (p, j+1) = \sigma \{ p(se, i) : i \in m \} (p, j)$, $\sigma \{ v(se, i) : i \in m \} (p, j+1) = \sigma \{ v(se, i) : i \in m \} (p, j) + 1$ et $M'(se) = M(se) + 1$ et dans les deux cas, nous déduisons de l'hypothèse d'induction que $\sigma \{ p(se, i) : i \in m \} (p, j+1) = \sigma \{ v(se, i) : i \in m \} (p, j+1) + \underline{I_{sem}}(se) - M'(se)$.

□

Nous en déduisons le théorème d'Hebermann[72] qui s'énonce informellement comme suit :

"Si la valeur initiale du sémaphore se est positive ou nulle alors le nombre de fois où la commande $p(se)$ a été franchie est égal au minimum du nombre de fois où la commande $p(se)$ a été exécutée et du nombre de fois où la commande $v(se)$ a été exécutée plus la valeur initiale du sémaphore".

Théorème 2.8.5.2.6 v5

$\forall p \in \Sigma[[\text{tps}]], j \in |p|, se \in \mathcal{S}_e.$

$$\underline{I_{sem}}(se) \geq 0$$

$\Rightarrow$

$$\sigma\{\#(se, i), \#(se, i, i') : i, i' \in m\}(p, j) = \min \left\{ \sigma\{\underline{w}(se, i), \#(se, i) : i \in m\}(p, j), \right. \\ \left. \sigma\{\underline{v}(se, i), \#(se, i) : i, i' \in m\}(p, j) + \underline{I_{sem}}(se) \right\}$$

Démonstration

Posons $p_j = \langle L, M, Q \rangle$. D'après le lemme 2.8.5.2.6 v1 deux cas sont à considérer :

- Si $|Q(se)| = 0$ alors $M(se) \geq 0$ et donc 2.8.5.2.6 v3 entraîne que

$$\sigma\{\#(se, i), \#(se, i, i') : i, i' \in m\}(p, j) = \sigma\{\#(se, i) : i \in m\}(p, j) + \sigma\{\#(se, i, i') : i, i' \in m\}(p, j) = \\ \sigma\{\#(se, i) : i \in m\}(p, j) + \sigma\{\underline{w}(se, i) : i \in m\}(p, j) = \sigma\{\#(se, i), \underline{w}(se, i) : i \in m\}(p, j). \text{ D'autre part } \\ 2.8.5.2.6 v4 \text{ entraîne que } \sigma\{\#(se, i), \#(se, i, i') : i, i' \in m\}(p, j) \leq \\ \sigma\{\underline{v}(se, i), \#(se, i, i') : i, i' \in m\}(p, j) + \underline{I_{sem}}(se) \text{ et donc } \sigma\{\#(se, i), \#(se, i, i') : i, i' \in m\}(p, j) \text{ est } \\ \text{égal à l'infimum de ces deux quantités.}$$

- Si $|Q(se)| > 0$ alors $M(se) = 0$ et donc 2.8.5.2.6 v3 entraîne que

$$\sigma\{\#(se, i), \#(se, i, i') : i, i' \in m\}(p, j) < \sigma\{\#(se, i), \underline{w}(se, i) : i \in m\}(p, j) \text{ tandis que } 2.8.5.2.6 v4 \\ \text{entraîne que } \sigma\{\#(se, i), \#(se, i, i') : i, i' \in m\}(p, j) = \sigma\{\underline{v}(se, i), \#(se, i, i') : i, i' \in m\}(p, j) + \underline{I_{sem}}(se) \text{ et donc } \\ \sigma\{\#(se, i), \#(se, i, i') : i, i' \in m\}(p, j) \text{ est égal à l'infimum de ces deux quantités.}$$

□

La première des propriétés d'équité des pénales est qu'aucun processus ne peut être bloqué devant une commande $\underline{v}(se)$. En termes d'actions et pour des traces infinies, nous avons que si une des actions $\underline{v}(se, m)$ ou $\#(se, m, i)$, $i \in (m \cup m)$ est exécutable alors une de ces actions sera fatalement exécutée.

Théorème 2.8.5.2.6 v 6

$\forall p \in \Sigma[Pps], \text{ se} \in \mathcal{E}, m \in m, i \in |p|.$

$$\begin{aligned} & [|p| = \omega \wedge \exists \Delta \in S[Pps]. (t[Pps]_{\underline{v}(se, m)}(p_i, \Delta) \vee \exists m' \in m. t[Pps]_{\underline{vp}(se, m, m')}(p_i, \Delta))] \\ \rightarrow & [\exists j \geq i. (p_j = \underline{v}(se, m) \vee \exists m' \in m. p_j = \underline{vp}(se, m, m'))] \end{aligned}$$

Démonstration

Si $\underline{v}(se, m)$ ou $\underline{vp}(se, m, m')$ est exécutable en p_j alors par définition de $t[Pps]$, nous avons $\text{Pro}_m \equiv \alpha L_m : \underline{v}(se); L'_m : \beta$. Si $\underline{v}(se, m)$ et $\underline{vp}(se, m, m')$, $m' \in (m \vee m)$ ne sont jamais exécutées alors le contrôle de Pro_m reste en L_m pour tout p_j , $j \geq i$ (car par définition de $t[Pps]$, seule l'exécution de l'une de ces actions peut le déplacer (en L'_m)). Posant $p_j = \langle \langle L_0, \dots, L_{n-1} \rangle, M, Q \rangle$, nous observons que deux cas sont possibles :

- Si $Q(se) = \langle \rangle$ alors l'action $\underline{v}(se, m)$ est exécutable,
- Si $Q(se) = R Q'(se)$ alors d'après le lemme 2.8.5.2.6 v 2, nous avons $\text{Pro}_R \equiv \alpha' L_R : p(se); \beta'$ et donc d'après la syntaxe $\alpha' L_R : p(se); L'_R : \beta''$ de sorte que l'action $\underline{vp}(se, m, R)$ est exécutable.

Nous avons démontré que l'une des actions $\underline{v}(se, m)$ ou $\underline{vp}(se, m, m')$, $m' \in (m \vee m)$ est toujours exécutable en p_j pour $j \geq i$. D'après l'hypothèse d'équité faible énoncée en 2.8.5.2.5 il n'est pas possible que les deux actions ne soient jamais exécutées au delà de p_i .

□

D'autre part, si la commande $p(se)$ est exécutable infiniment souvent alors elle sera franchie. Cette propriété s'énonce plus précisément en termes d'actions et pour des traces infinies comme suit :

Si infiniment souvent une des actions $\underline{w}(se, m)$, $p(se, m)$ ou $\underline{vp}(se, i, m)$, $i \in (m \vee m)$ est exécutable alors une des actions $p(se, m)$ ou $\underline{vp}(se, i, m)$, $i \in (m \vee m)$ sera fatalement exécutée :

Théorème 2.8.5.2.6~7

$\forall p \in \llbracket Pps \rrbracket, se \in \mathcal{S}_e, m \in m, i \in |p|.$

$[|p| = \omega]$

$$\Rightarrow [[\forall j \geq i. \exists k \geq j, \Delta \in S \llbracket Pps \rrbracket. (t \llbracket Pps \rrbracket_{\underline{w}(se, m)}(p_k, \Delta) \vee t \llbracket Pps \rrbracket_{\underline{p}(se, m)}(p_k, \Delta) \vee \exists m' \in m. t \llbracket Pps \rrbracket_{\underline{up}(se, m', m)}(p_k, \Delta))]]$$

$$\Rightarrow [\exists j \geq i. (\underline{p}_j = \underline{p}(se, m) \vee \exists m' \in m. \underline{p}_j = \underline{up}(se, m', m))]$$

Démonstration

Supposons $|p| = \omega$ et posons $p_i = \langle L^i, M^i, Q^i \rangle$ avec $L^i = \langle L_0^i, \dots, L_{m-1}^i \rangle$ quand le contrôle est dans la commande parallèle de $Pps \equiv Ps \llbracket Pres_0 \parallel \dots \parallel Pres_{m-1} \rrbracket; Ps'$. Supposons que infiniment souvent les actions $\underline{w}(se, m)$, $\underline{p}(se, m)$ ou $\underline{up}(se, m', m)$ avec $m' \in m$ sont exécutables au delà de p_i .

- Si $\exists m' \in m. \underline{up}(se, m', m)$ est exécutable (c'est-à-dire $\exists j \geq i, \Delta \in S \llbracket Pps \rrbracket. t \llbracket Pps \rrbracket_{\underline{up}(se, m', m)}(p_j, \Delta')$) alors il faut (d'après la définition de $t \llbracket Pps \rrbracket$) que le processus $Pres_m$ soit en tête de la file d'attente du sémaphore se ($Q^j(se) = m$ ou $\Delta' = \langle L', M', Q' \rangle$). Si $\forall m' \in m, \underline{up}(se, m', m)$ n'était jamais exécuté ultérieurement et que seule cette action permet à $Pres_m$ de sortir de la file de se , $Pres_m$ restera toujours en tête de la file de $Q^k(se), k \geq j$. D'autre part le théorème 2.8.5.2.6~6 montre que $\underline{up}(se, m', m)$ étant exécutable en p_j , l'action $\underline{v}(se, m')$ ou $\underline{up}(se, m', m'')$ sera fatalement exécutée en $p_k, k \geq j$. Pour que $\underline{v}(se, m')$ puisse être exécuté il faudrait par définition de $t \llbracket Pps \rrbracket$ que la file de se soit vide ($Q^k(se) = \langle \rangle$) contraire au fait que $Pres_m$ est en tête ($Q^k(se)_0 = m$). C'est donc $\underline{up}(se, m', m'')$ qui est exécuté et donc par définition de $t \llbracket Pps \rrbracket_{\underline{up}(se, m', m'')}, m''$ est en tête de la file de se ($Q^k(se)_0 = m''$) et donc $m'' = m$, ce qui montre que $\underline{up}(se, m', m)$ est fatalement exécuté.

- Si $\forall m' \in m. \underline{up}(se, m', m)$ n'est pas exécutable (c'est-à-dire $\forall j \geq i, \Delta \in S \llbracket Pps \rrbracket. \neg t \llbracket Pps \rrbracket_{\underline{up}(se, m', m)}(p_j, \Delta')$) alors infiniment souvent l'une des actions $\underline{w}(se, m)$ ou $\underline{p}(se, m)$ est exécutable au delà de p_i . Il existe un $p_j, j \geq i$ tel que l'une des actions $\underline{w}(se, m)$ ou $\underline{p}(se, m)$ est exécutable. En ce point, par définition

de $t[\text{Pps}]_{\underline{w}(se,m)}$ et $t[\text{Pps}]_{\underline{p}(se,m)}$ le processus Pro_m n'est pas dans la file du sémaphore se ou cette file est vide, $\forall l \in |\varphi^j(se)|. \varphi^j(se)_l \neq m$. Deux cas sont alors possibles :

- Si Pro_m entre ultérieurement dans la file d'attente du sémaphore se , c'est-à-dire $\exists k > j, l \in |\varphi^k(se)|. \varphi^k(se)_l = m$; tant que m reste dans la file d'attente, les actions $\underline{w}(se,m)$ et $\underline{p}(se,m)$ ne sont pas exécutables et comme infiniment souvent l'une d'elle doit l'être, il faut que Pro_m sorte de la file d'attente de se après y être entré. Par définition de $t[\text{Pps}]$, seule une action $\underline{wp}(se, m', m)$ peut faire sortir m de la file de se et donc une action de ce type est fatalement exécutée'.

- Si Pro_m n'entre jamais dans la file d'attente du sémaphore se , c'est-à-dire $\forall k > j, l \in |\varphi^k(se)|. \varphi^k(se)_l \neq m$. Par définition de $t[\text{Pps}]$, il n'est pas possible que les actions $\underline{w}(se,m)$ et $\underline{wp}(se, m', m)$, m'én soient exécutées car elles auraient pour effet de faire entrer ou sortir le processus Pro_m de la file d'attente de se . Comme $\underline{w}(se,m)$ ou $\underline{p}(se,m)$ est exécutable en p_j , nous avons $\text{Pro}_m \equiv \alpha L_m^j : \underline{p}(se); \beta$. Puisque le contrôle de Pro_m se déplace de L_m^j au delà de p_j , il faudrait qu'une action $\underline{wp}(se, m', m)$ ou $\underline{p}(se,m)$ soit exécutée'. La première alternative n'est pas possible, la seconde termine la preuve. Supposons que l'action $\underline{p}(se,m)$ ne soit jamais exécutée' au delà de p_j . Nous observons alors que $\forall k > j, \text{Pro}_m \equiv \alpha L_m^k : \underline{p}(se); \beta$ avec $L_m^k = L_m^j$. Par conséquent, $\forall k > j$, si $M^k(se) \leq 0$ alors $\underline{w}(se,m)$ est exécutable tandis que si $M^k(se) > 0$ alors $\underline{p}(se,m)$ est exécutable. Au delà de p_j , l'une des deux actions $\underline{w}(se,m)$ ou $\underline{p}(se,m)$ est toujours exécutable et d'après la définition des traces $\Sigma[\text{Pps}]$ donnée en 2.8.5.2.5, l'une de ces actions sera fatalement exécutée'. Comme il ne peut s'agir de $\underline{w}(se,m)$ (car Pro_m n'entre jamais dans la file d'attente de se), il s'agit de $\underline{p}(se,m)$.

□

Les recherches actuelles sur les preuves de propriétés de fatalité de programmes parallèles avec sémaphores comme Lehman, Pnueli-Stavi [24] sont basées sur des hypothèses d'équité des sémaphores similaires à celle donnée dans les théorèmes 2.8.5.2.6v6 et 2.8.5.2.6v7. Ces méthodes ne peuvent pas être complètes pour la raison que certaines traces satisfaisant ces hypothèses d'équité ne sont pas des traces de $\Sigma[Pps]$. C'est le cas en particulier parce que les propriétés d'équité énoncées dans les théorèmes 2.8.5.2.6v6 et 2.8.5.2.6v7 n'interdisent pas que l'attente pour passer un sémaphore ne soit pas bornée.

Au contraire, pour la sémantique de Dijkstra [68], nous avons la propriété suivante, que nous démontrons :

"Si un processus $Proc_m$ est en attente devant une commande $p(se)$ en position r dans la file d'attente du sémaphore se , alors $r+1$ commandes $r(se)$ et un nombre fini borné de commandes $p(se)$ s'exécutent avant que l'attente ne prenne fin."

Théorème 2.8.5.2.6v8

$\forall Pps \in \mathcal{Pps}, new, p \in \Sigma[Pps], Q \in (|p| \rightarrow (\mathcal{L} \rightarrow m^{\leq \omega})), j, k \in |p|, se \in \mathcal{L}, r \in \omega.$

$[Pps \equiv Ps[Proc_0 \parallel \dots \parallel Proc_{m-1}]; Ps' \wedge j < k \wedge \exists m \in m. \forall l \in |p|. (j \leq l \leq k) \Rightarrow$

$(\exists \langle L_0^l, \dots, L_{m-1}^l \rangle \in \mathcal{L}^n, M^l \in (\mathcal{V} \rightarrow \mathcal{W}) : p_l = \langle \langle L_0^l, \dots, L_{m-1}^l \rangle, M^l, Q^l \rangle \wedge$

$Proc_m \equiv \alpha L_m^l : p(se); L_m^k : \beta \wedge Q^l(se)_r = m]$

$\Rightarrow$

$[$
 $\sigma\{\alpha(se, i) : i \in m\}(p, k) - \sigma\{\alpha(se, i) : i \in m\}(p, j) = 0$
 $\wedge$
 $\sigma\{\alpha p(se, i, i') : i, i' \in m\}(p, k) - \sigma\{\alpha p(se, i, i') : i, i' \in m\}(p, j) = r+1$
 $\wedge$
 $\sigma\{p(se, i) : i \in m\}(p, k) - \sigma\{p(se, i) : i \in m\}(p, j) = 0$
 $\wedge$
 $\sigma\{\omega(se, i) : i \in m\}(p, k) - \sigma\{\omega(se, i) : i \in m\}(p, j) \leq m - |Q^l(se)| + r]$

Démonstration

Nous construisons $R \in (|P| \rightarrow \omega)$ tel que $R_j = \kappa$ et $\forall l \in |P|, j \leq l \leq k$ nous avons $Q^l(Se)_{R_j} = m$, $\sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, l) - \sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, j) = R_j - R_l$, $\sigma\{\downarrow(Se, i) : i \in m\}(p, l) - \sigma\{\downarrow(Se, i) : i \in m\}(p, j) = 0$ et $\sigma\{\#(Se, i) : i \in m\}(p, l) - \sigma\{\#(Se, i) : i \in m\}(p, j) = 0$

Par hypothèse, nous avons $Q^j(Se)_\kappa = m$ de sorte que les propriétés ci-dessus sont vraies pour $j = l$ en posant $R_j = \kappa$. Ayant construit R_l pour $l < k$, nous définissons R_{l+1} comme suit : si $\#_l = \uparrow\uparrow(Se, i, i')$ alors par définition de $t[Pps]_{\uparrow\uparrow(Se, i, i')}$ nous avons $Q^l(Se) = i' Q^{l+1}(Se)$ avec $Pres_{i'} \equiv \alpha' L_{i'}^l : \#(Se) ; L_{i'}^{l+1} : \beta'$. D'après l'hypothèse que les étiquettes ne figurent qu'une seule fois dans le programme Pps , nous avons $L_{i'}^l \neq L_{i'}^{l+1}$ et donc $m \neq i'$ car $L_m^l = L_m^{l+1}$. Posons $R_{l+1} = R_l - 1$ et donc $Q^{l+1}(Se)_{R_{l+1}} = Q^l(Se)_{R_{l+1}+1} = Q^l(Se)_{R_l} = m$. De plus $\sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, l+1) = \sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, l) + 1$ et donc $\sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, l+1) - \sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, j) = R_j - R_{l+1} + 1 = R_j - R_{l+1}$. Nous ne pouvons pas avoir $\#_l = \downarrow(Se, i)$ (car il faudrait $Q^l(Se) = \langle \rangle$) et donc $\sigma\{\downarrow(Se, i) : i \in m\}(p, l+1) = \sigma\{\downarrow(Se, i) : i \in m\}(p, l)$. De même, nous ne pouvons pas avoir $\#_l = \#(Se, i)$ (car il faudrait $M^l(Se) > 0$ et donc d'après le lemme 2.8.5.2.6 v1, nous aurions $|Q(Se)| = 0$ contraire à $Q^l(Se)_{R_l} = m$) et donc $\sigma\{\#(Se, i) : i \in m\}(p, l+1) = \sigma\{\#(Se, i) : i \in m\}(p, l)$. Si $\#_l$ est $\downarrow(Se, i)$, $\downarrow(Sm, i)$, $\#(Sm, i)$, $\downarrow(Sm, i)$, $\uparrow\uparrow(Sm, i, i')$ avec $Sm \in (O \setminus Se)$ nous avons $Q^{l+1}(Se) = Q^l(Se)$ par définition de $t[Pps]$ et la propriété reste vraie en posant $R_{l+1} = R_l$.

Observons que pour $l = k-1$, nous avons $\#_{k-1} = \uparrow\uparrow(Se, i, m)$. En effet par définition de $t[Pps]$ et le fait que $Pres_m \equiv \alpha' L_m^{k-1} : \#(Se) ; L_m^k : \beta$ nous ne pouvons avoir que $\#_{k-1} \in \{\#(Se, m), \uparrow\uparrow(Se, i, m)\}$. Mais $\#_{k-1} = \#(Se, m)$ est impossible car il faudrait $M^{k-1}(Se) > 0$ et donc d'après 2.8.5.2.6 v1 $Q^{k-1}(Se) = \langle \rangle$ en contradiction avec $Q^{k-1}(Se)_{R_{k-1}} = m$. Par définition de $t[Pps]_{\uparrow\uparrow(Se, i, m)}$ et le lemme 2.8.5.2.6 v1 (qui implique que m ne peut pas figurer deux fois dans la file $Q^{k-1}(Se)$), nous avons $R_{k-1} = 0$. Il vient donc $\sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, k) - \sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, j) = \sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, k-1) + 1 - \sigma\{\uparrow\uparrow(Se, i, i') : i, i' \in m\}(p, j) = R_j - R_{k-1} + 1 = R_j + 1 = \kappa + 1$.

Observons qu'entre p_j et p_{R-1} (et donc p_R) les processus qui ont pu exécuter la commande $\sharp(se)$ sont ceux qui n'étaient pas dans la file d'attente $q^{\sharp}(se)$ en p_j ainsi que ceux qui étaient devant Pr_{sm} dans la file d'attente $q^{\sharp}(se)$ et qui sont sortis de cette file avant Pr_{sm} . Nous avons vu que l'exécution d'une telle commande ne peut correspondre qu'à l'action $\omega(se, i)$ qui a pour effet de ranger le processus Pr_{si} après Pr_{sm} dans la file d'attente de se . Comme Pr_{sm} ne sort de la file qu'en p_R le processus ne peut rentrer dans la file qu'au plus une fois, nous en déduisons $\# \{ \omega(se, i) : i \in m \} (p, R) - \# \{ \omega(se, i) : i \in m \} (p, j) = m - |q^{\sharp}(se)| + n$.

□

2.8.5.3 Sémantique libérale

Pour démontrer certaines propriétés des programmes parallèles avec sémaphores, nous pouvons quelquefois utiliser une sémantique libérale qui ne prend pas en compte la file d'attente et correspond à une attente active.

$$sl \llbracket Pps \rrbracket = C \llbracket Pps \rrbracket \times \mathcal{M}$$

$$el \llbracket Pps \rrbracket = [\exists L \in \mathcal{L}, M \in \mathcal{M}. (\Delta = \langle L, M \rangle \wedge Pps \equiv L : \alpha \wedge \forall se \in \mathcal{S}. M(se) = \underline{I_{sem}}(se))]$$

$$\begin{aligned} tl \llbracket Pps \rrbracket_{\omega(se, i)} (\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle \langle L'_0, \dots, L'_{m-1} \rangle, M' \rangle) = \\ [Pps \equiv Ps \llbracket Pr_{s_0} \parallel \dots \parallel Pr_{s_{m-1}} \rrbracket ; Ps' \wedge Pr_{s_i} \equiv \alpha L_i : \sharp(se); \beta \wedge M(se) < 0 \wedge \\ \forall k \in m. L'_k = L_k \wedge M' = M] \end{aligned}$$

$$\begin{aligned} tl \llbracket Pps \rrbracket_{\sharp(se, i)} (\langle \langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle \langle L'_0, \dots, L'_{m-1} \rangle, M' \rangle) = \\ [Pps \equiv Ps \llbracket Pr_{s_0} \parallel \dots \parallel Pr_{s_{m-1}} \rrbracket ; Ps' \wedge Pr_{s_i} \equiv \alpha L_i : \sharp(se); L'_i : \beta \wedge M(se) > 0 \wedge \\ \forall k \in (m \setminus i). L'_k = L_k \wedge M'(se) = M(se) - 1 \wedge \forall v \in (\mathcal{V} \setminus se). M'(v) = M(v)] \end{aligned}$$

$$\begin{aligned}
tl[Pps]_{\underline{w}(se,i)}(\langle\langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle\langle L'_0, \dots, L'_{m-1} \rangle, M' \rangle) = \\
[Pps \equiv Ps [Pro_0 \parallel \dots \parallel Pro_{m-1}]; Ps' \wedge Pro_i \equiv \alpha L_i : \underline{w}(se); L'_i : \beta \wedge \\
\forall k \in (m \setminus i). L'_k = L_k \wedge M'(se) = M(se) + 1 \wedge \forall v \in (V \setminus se). M'(v) = M(v)]
\end{aligned}$$

$$\begin{aligned}
tl[Pps]_{\underline{w}(se,i,j)}(\langle\langle L_0, \dots, L_{m-1} \rangle, M \rangle, \langle\langle L'_0, \dots, L'_{m-1} \rangle, M' \rangle) = \\
[Pps \equiv Ps [Pro_0 \parallel \dots \parallel Pro_{m-1}]; Ps' \wedge Pro_i \equiv \alpha L_i : \underline{w}(se); L'_i : \beta \wedge \\
Pro_j \equiv \alpha' L'_j : \underline{p}(se); L'_j : \beta' \wedge \forall k \in m \setminus \{i, j\}. L'_k = L_k \wedge M' = M]
\end{aligned}$$

Cette sémantique s'étant utilisée pour raisonner sur des ensembles d'états accessibles, nous remarquons que l'action d'attente $\underline{w}(se, i)$ peut être supprimée puisqu'elle ne change pas l'état courant. Nous observons également que si l'exécution de l'action $\underline{w}(se, i, j)$ conduit de l'état s à l'état s' alors il est également possible que les actions $\underline{w}(se, i)$ puis $\underline{p}(se, j)$ soient exécutées, ce qui conduirait également de s à s' . Pour raisonner sur l'ensemble des états accessibles par la sémantique libérale, les actions $\underline{w}(se, i)$ et $\underline{w}(se, i, j)$ peuvent donc être supprimées :

$$AL[Pps] = \{\underline{p}, \underline{p}'\} \cup m \cup \{\underline{p}(se, i), \underline{w}(se, i) : se \in \mathcal{O} \wedge i \in m\}$$

Aucune propriété d'équité n'étant à prendre en compte dans cette sémantique libérale, nous définissons :

$$\Sigma l[Pps] = \Sigma \langle sl[Pps], AL[Pps], tl[Pps], el[Pps] \rangle$$

La propriété évidente de cette sémantique est qu'à une fonction des états près, l'ensemble des états accessibles d'après la sémantique (exacte) est inclus dans l'ensemble des états accessibles d'après la sémantique libérale :

Théorème 2.8.5.3 v1

si

 $fl \in (S[Pps] \rightarrow SL[Pps])$ est défini par $fl(\langle L, M, Q \rangle) = \langle L, M \rangle$

alors

$$\begin{aligned} \{\Delta \in SL[Pps] : \exists p \in \nu \langle fl \rangle (\Sigma[Pps]), i \in |p|, p_i = \Delta\} \\ = \{\Delta \in SL[Pps] : \exists p \in SL[Pps], i \in |p|, p_i = \Delta\} \end{aligned}$$

2.8.5.4 Exemple

le programme parallèle suivant réalise une section critique :

sem se init 1;

0:

10:

while true do

11:

p(se);

12:

v(se);

13:

od;

14:

||

20:

while true do

21:

p(se);

22:

v(se);

23:

od;

24:

1;

1:

La sémantique exacte aussi bien que la sémantique libérale garantissent que 12 et 22 sont en exclusion mutuelle. La sémantique exacte garantit l'entrée en section critique mais pas la sémantique libérale.

2.9 REFERENCES

- ABRAHAMSON K. [80], "Expressiveness and decidability of logics of processes", Ph.D. Thesis, Univ. of Washington, Seattle, USA, (1980).
- COUSOT P. [78], "Méthodes itératives de construction et d'approximation de points fixes d'opérateurs monotones sur un treillis, analyse sémantique des programmes", Thèse d'Etat, USMG Grenoble, (1978).
- COUSOT P. [79], "Analysis of the behavior of dynamic discrete systems", Rapport de Recherche n°161, IMAG, USMG Grenoble, (Jan. 1979).
- COUSOT P. [81], "Semantic foundations of program analysis", dans "Program flow analysis, theory and applications", S.S. Muchnick & N.J. Jones (Eds.), Prentice-Hall, (1981), 303-342.
- COUSOT P., COUSOT R. [79], "A constructive characterization of the lattices of all retractions, preclosure, quasi-closure and closure operators on a complete lattice", Portugaliae Mathematica, Vol. 38, Fasc. 1-2, (1979), 185-198.
- DIJKSTRA E.W.D. [68], "Cooperating sequential processes", dans "Programming languages", F. Genuys (Ed.), Academic Press, N.Y., (1968), 43-112.
- EMERSON E.A. [81], "Alternative semantics for temporal logics", Research Report TR-182, Dept. of C. Sci., U. of Texas at Austin, (Oct. 1981), 16 p.
- HABERMANN A.N. [72], "Synchronization of communicating processes", CACM 13, 3 (1972), 171-176.
- HOARE C.A.R. [78], "Communicating sequential processes", CACM 21, 8 (1978), 666-677.
- KELLER R.M. [76], "Formal verification of parallel programs", CACM 19, 7 (1976), 371-384.

LAMPORT L. [80], "Sometime" is sometimes "not never", 7th Annual ACM Symp. on Principles of Programming Languages, (1980), 174-185.

LEHMANN D., PNUELI A., STAVI J. [81], "Impartiality, justice and fairness: the ethics of concurrent termination", Proc. 8th Coll. on Automata, Languages and Programming, Lect. Notes in Comp. Sci. 115, Springer Verlag, (1981), 264-277.

MILNE R., STRACHEY C. [76], "A theory of programming language semantics", Chapman & Hall (London) & Wiley (New York), (1976).

PRATT V.R. [79], "Process logic", Proc. 6th ACM Symp. on Principles of Programming Languages, (1979), 93-100.



3. PROPRIETES D'INVARIANCE ET DE FATALITE DES PROGRAMMES

3.1 SPECIFICATION DE PROGRAMMES

Une preuve de programme consiste à démontrer une relation entre une sémantique (définissant "ce que fait l'exécution du programme") et une spécification (définissant "ce que devrait faire l'exécution du programme").

Pour qu'une étude des méthodes de preuve ne dépende pas des techniques choisies pour spécifier les programmes il faut donner une définition abstraite et générale de la spécification de programmes. Nous proposons de définir la spécification d'un programme P_r comme étant une sémantique $\langle S, A, \Sigma \rangle$. De cette façon une preuve de programme consiste à démontrer qu'une relation est vraie entre deux sémantiques : la spécification $\langle S, A, \Sigma \rangle$ et la sémantique opérationnelle $\langle S[P_r], A[P_r], \Sigma[P_r] \rangle$.

Exemple 3.1-1

Etant donné des ensembles S d'états et A d'actions et une assertion $\phi \in (S \rightarrow \{\text{tt}, \text{ff}\})$ sur les états, l'ensemble des traces pour lesquelles ϕ est tout le temps vraie en cours d'exécution est $Z = \{p \in \Sigma[S, A] : \forall i \in |p|. \phi(i)\}$.

(1) La preuve de l'invariance de ϕ pour un programme P_r consiste à démontrer que $\langle S[P_r], A[P_r], \Sigma[P_r] \rangle \subseteq \langle S, A, \Sigma \rangle$ c'est-à-dire essentiellement $\forall p \in \Sigma[P_r], i \in |p|. \phi(p_i)$.

(2) La preuve de préservation de ϕ pour un programme P_r consiste à démontrer que $S[P_r] \subseteq S$, $A[P_r] \subseteq A$ et $\Sigma[P_r] \cap Z \neq \emptyset$ c'est-à-dire essentiellement $\exists p \in \Sigma[P_r]. \forall i \in |p|. \phi(p_i)$.

Etant donné des ensembles S d'états et A d'actions et une assertion $\psi \in (S \rightarrow \{\text{tt}, \text{ff}\})$ sur les états, l'ensemble des traces pour lesquelles ψ est inévitablement vraie en cours d'exécution est $Z = \{p \in \Sigma^* \langle S, A \rangle : \exists i \in |p|. \psi(p_i)\}$.

(3) La preuve de fatalité de ψ pour un programme P_r consiste à démontrer que $\langle S[P_r], A[P_r], Z[P_r] \rangle \subseteq \langle S, A, Z \rangle$ c'est-à-dire essentiellement $\forall p \in Z[P_r]. \exists i \in |p|. \psi(p_i)$.

(4) La preuve de possibilité de ψ pour un programme P_r consiste à démontrer que $S[P_r] \subseteq S$, $A[P_r] \subseteq A$ et $Z[P_r] \cap Z \neq \emptyset$ c'est-à-dire essentiellement $\exists p \in Z[P_r], i \in |p|. \psi(p_i)$.

□

Les spécifications de programmes les plus souvent utilisées concernent les propriétés d'invariance et de fatalité dont nous donnons maintenant des exemples.

3.2 INVARIANCE

Nous donnons une définition de l'invariance conditionnelle qui généralise une notion introduite par Lamport[80]. Nous obtenons comme cas particulier la notion classique d'invariance dont la correction partielle est un cas particulier. Nous distinguons l'invariance relationnelle qui permet d'exprimer une relation entre un état initial et un état courant sur une trace et l'invariance assertionnelle qui permet d'exprimer une assertion sur l'état courant d'une trace de la sémantique du programme. Nous donnons les définitions et quelques exemples

3.2.1 INVARIANCE CONDITIONNELLE

Soient $\langle S, A, \Sigma \rangle$ une sémantique opérationnelle et $\phi, \psi \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$ des relations entre états. ψ est invariante sous condition ϕ pour $\langle S, A, \Sigma \rangle$ si et seulement si

$$\forall p \in \Sigma, i \in |p|. [\forall j \in i. \phi(p_0, p_j)] \rightarrow \psi(p_0, p_i)$$

Exemple

Considérons un programme parallèle asynchrone $Ppa \equiv P_0 \parallel P_1 \parallel P_2$.
 L'assertion $\text{Terminé}_i(s) = [\exists L_0, L_1 \in \mathcal{L}, M \in \mathcal{M}. s = \langle L_0, L_1, M \rangle \wedge P_{i,1} \equiv \text{tt}]$ exprime que l'exécution du processus $P_{i,1}$, $i=0,1$ s'est terminée correctement.
 L'invariance de $\psi(s, s') = \neg \text{Terminé}_1(s')$ sous condition $\phi(s, s') = \neg \text{Terminé}_0(s')$ pour Ppa exprime que tant que l'exécution du processus $P_{i,0}$ n'est pas terminée correctement, l'exécution du processus $P_{i,1}$ ne peut pas se terminer.

□

Nous pouvons imaginer de très nombreuses variantes de cette définition comme celle-ci :

$\psi \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$ est invariante sous condition $\phi \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$ à partir de $e \in (S \rightarrow \{\text{tt}, \text{ff}\})$ si et seulement si

$$\forall p \in \Sigma, j \in |p|, i \in j. [\mathcal{E}(p_i) \wedge \forall e(j \cup i). \phi(p_i, p_{i+1})] \Rightarrow \psi(p_i, p_{j+1})$$

Exemple

Un certain nombre de propriétés du schéma producteur-consommateur suivant peuvent s'exprimer sous la forme ci-dessus :

```

0: 10:  while true do
    11:  produire(PX);
    12:    C! PX;
    13:  od;
    14:
1: 11:  ||
    20:  while true do
    21:    C? CX;
    22:    consommer(CX);
    23:  od;
    24:
    1: 12:  ||;

```

Un message est produit dans l'état $\langle \langle c_1, c_2 \rangle, M \rangle$ si $c_1 = 11$ et l'état successeur est de la forme $\langle \langle 12, c_2' \rangle, M' \rangle$:

$$\pi(\langle \langle c_1, c_2 \rangle, M \rangle, \langle \langle c_1', c_2' \rangle, M' \rangle) = [c_1 = 11 \wedge c_1' = 12]$$

Un message est consommé dans l'état $\langle \langle c_1, c_2 \rangle, M \rangle$ si $c_2 = 22$ et l'état successeur est de la forme $\langle \langle c_1', 23 \rangle, M' \rangle$:

$$\gamma(\langle \langle c_1, c_2 \rangle, M \rangle, \langle \langle c_1', c_2' \rangle, M' \rangle) = [c_2 = 22 \wedge c_2' = 23]$$

En choisissant $\epsilon(s) = \#$, $\phi = \top$ et $\psi = \top$ dans la formule ci-dessus, nous exprimons qu'à partir du moment où l'exécution du programme est commencée, il n'est pas possible de consommer tant qu'il n'y a pas eu production.

En choisissant $\epsilon(\langle c_1, c_2 \rangle, M) = [c_2 = 23]$, $\phi = \top$ et $\psi = \top$ dans la formule ci-dessus, nous exprimons qu'après une consommation il n'est pas possible de consommer à nouveau sans qu'il y ait eu production entre temps.

□

3.2.2 INVARIANCE RELATIONNELLE

Le cas particulier où $\phi(s, s') = \#$ correspond à l'invariance relationnelle.

$\psi \in (S \times S \rightarrow \{\top, \text{ff}\})$ est invariante pour $\langle s, A, \Sigma \rangle$ si et seulement si

$$\forall p \in \Sigma, i \in |p|. \psi(p_i, p_i)$$

Exemple

La correction partielle est une propriété d'invariance relationnelle.

Etant données une spécification $\Phi \in (\mathcal{X} \rightarrow \{\top, \text{ff}\})$ de l'état mémoire d'entrée et une spécification de sortie $\Psi \in (\mathcal{X} \times \mathcal{X} \rightarrow \{\top, \text{ff}\})$ liant l'état mémoire final à l'état mémoire initial, la correction partielle d'un programme $P \in \mathcal{P}$ pour Φ, Ψ exprime que si l'exécution commence dans un état M satisfaisant Φ et atteint un état de sortie M' alors Ψ lie M et M' . Autrement dit

$$\psi(s, s') = ([\exists L, L', M, M'. s = \langle L, M \rangle \wedge \Phi(M) \wedge s' = \langle L', M' \rangle \wedge P \equiv \alpha L'] \Rightarrow \Psi(M, M'))$$

c'est-à-dire qu'il est toujours vrai en cours d'exécution que si l'état initial satisfait Φ et l'état courant est un état final alors Ψ lie l'état final à l'état initial.

□

3.2.3 INVARIANCE ASSERTIONNELLE

Le cas particulier de l'invariance conditionnelle où $\phi(s, s') = \text{tt}$ et $\psi(s, s') = \Psi(s')$ correspond à l'invariance assertionnelle.

$\psi \in (S \rightarrow \{\text{tt}, \text{ff}\})$ est invariante pour $\langle S, A, \Sigma \rangle$ si et seulement si $\forall p \in \Sigma, i \in |p|. \psi(p_i)$

Exemple

L'exclusion mutuelle est une propriété d'invariance assertionnelle.

Par exemple, nous exprimons que les deux processus du programme 2.8.5.4 ne peuvent jamais être simultanément en section critique par l'invariance de

$$\psi(s) = [\exists L_0, L_1, M. s = \langle \langle L_0, L_1 \rangle, M \rangle \wedge \neg (L_0 = 12 \wedge L_1 = 22)]$$

□

La non-termination, l'absence d'erreurs à l'exécution, l'absence d'interblocages globaux permanents sont d'autres exemples de propriétés de programmes qui peuvent s'exprimer par l'invariance assertionnelle.

3.3 FATALITE

Nous donnons la définition de la fatalité sous invariance, Gabbay-Amueli-Shelah-Stavi[80], avec les cas particuliers de fatalité relationnelle et fatalité assertionnelle.

3.3.1 FATALITE SOUS INVARIANCE

Soient $\langle S, A, \Sigma \rangle$ une sémantique opérationnelle et $\phi, \psi \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$ des relations entre états.

$\psi \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$ est fatalité sous invariance de ϕ pour $\langle S, A, \Sigma \rangle$ si et seulement si

$$\forall p \in \Sigma. \exists i \in |p|. [\forall j \in i. \phi(p_0, p_j) \wedge \psi(p_0, p_i)]$$

Exemple

Nous pouvons exprimer que la valeur d'une variable entière d'un programme n'est strictement positive ou négative avant d'atteindre la valeur zéro par la fatalité de

$$\psi(\Delta, \Delta') = [\exists C', M'. \Delta' = \langle C', M' \rangle \wedge M'(x) = 0]$$

sous invariance de

$$\phi(\Delta, \Delta') = [\exists C, C', M, M'. \Delta = \langle C, M \rangle \wedge \Delta' = \langle C', M' \rangle \wedge M(x) \times M'(x) > 0]$$

□

3.3.2 FATALITE RELATIONNELLE

La fatalité relationnelle correspond au cas particulier de la fatalité sous invariance avec $\phi(\Delta, \Delta') = \text{tt}$.

$\psi \in (S \times S \rightarrow \{\text{tt}, \text{ff}\})$ est fatalité relationnelle pour $\langle S, A, \Sigma \rangle$ si et seulement si

$$\forall p \in \Sigma. \exists i \in |p|. \psi(p_0, p_i)$$

Exemple

La correction totale est une propriété de fatalité relationnelle.

Etant données une spécification d'entrée $\Phi \in (\mathcal{B} \rightarrow \{\text{tt}, \text{ff}\})$ décrivant l'état mémoire initial et une spécification de sortie $\Psi \in (\mathcal{B} \times \mathcal{B} \rightarrow \{\text{tt}, \text{ff}\})$ liant l'état mémoire final à l'état mémoire initial, la correction totale d'un programme $P_2 \in \mathcal{P}_2$ s'exprime par la fatalité de

$$\Psi(A, A') = ([\exists L, M. A = \langle L, M \rangle \wedge \Phi(M)] \Rightarrow [\exists L, M, L', M'. A = \langle L, M \rangle \wedge A' = \langle L', M' \rangle \wedge P_2 \equiv \alpha L' : \wedge \Psi(M, M')])$$

pour la sémantique $\langle S[P_2], A[P_2], \Sigma[P_2] \rangle$.

□

3.3.3 FATALITE ASSERTIONNELLE

La fatalité assertionnelle correspond au cas particulier de la fatalité sous invariance avec $\phi(A, A') = \text{tt}$ et $\psi(A, A') = \Psi(A')$.

$\Psi \in (\mathcal{S} \rightarrow \{\text{tt}, \text{ff}\})$ est fatale pour $\langle S, A, \Sigma \rangle$ si et seulement si

$$\forall p \in \Sigma. \exists i \in |p|. \Psi(p_i)$$

Exemple

L'absence de famine est un exemple de fatalité assertionnelle.

Par exemple, nous exprimons que le premier processus du programme 2.8.5.4 rentre fatalement en section critique s'il en fait la demande par la fatalité de

$$\Psi(A) = [\exists L_0, L_1, M. (A = \langle L_0, L_1 \rangle, M) \wedge L_0 = 12]$$

pour la sémantique

$$\langle S, A, \{p \in \text{Suff}(\Sigma) : \exists L_0, L_1, M. (A = \langle L_0, L_1 \rangle, M) \wedge L_0 = 11\} \rangle$$

□

La termination, la garantie de réponse à un signal peuvent également s'exprimer par la fatalité assertionnelle.

The first part of Radhia Cousot's thesis is available at
<https://pcousot.github.io/publications/RadhiaCousotTheseEsSciences.pdf>

The second part of Radhia Cousot's thesis is available at
<https://pcousot.github.io/publications/RadhiaCousotTheseEsSciences2.pdf>

The third part of Radhia Cousot's thesis is available at
<https://pcousot.github.io/publications/RadhiaCousotTheseEsSciences3.pdf>